

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ
«ФАХОВИЙ КОЛЕДЖ ІНФОРМАЦІЙНИХ СИСТЕМ І ТЕХНОЛОГІЙ
КИЇВСЬКОГО НАЦІОНАЛЬНОГО ЕКОНОМІЧНОГО УНІВЕРСИТЕТУ
ІМЕНІ ВАДИМА ГЕТЬМАНА»**

Освітньо-кваліфікаційний рівень «фаховий молодший бакалавр»

Спеціальність 121 «Інженерія програмного забезпечення»

Освітньо-професійна програма «Розробка програмного забезпечення»

Відділення «Програмування»

**ПОЯСНЮВАЛЬНА ЗАПИСКА
до кваліфікаційної роботи (дипломного проєкту)**

на тему

**Розробка інтелектуальної інформаційної системи управління
особистими фінансами на базі технологій Open Banking**

Студента 4 курсу, групи 472 _____ Владислав ГРЕБЕНЮК

Керівник кваліфікаційної роботи

(дипломного проєкту) _____ Станіслав РЕЗУНЕНКО

**Кваліфікаційна робота (дипломний проєкт) допущений до захисту в
Екзаменаційній комісії**

Завідувач відділення _____ Світлана ЯНЧУК

Голова циклової комісії _____ Оксана ПОМАЗУН

Консультанти:

Економічний розділ _____ Віталій ЧЕРНЕНКО

Розділ з охорони праці _____ Тетяна ПЕТРЕНКО

Нормоконтролер _____ Людмила ЮЩЕНКО

Київ – 2026

РЕФЕРАТ

Кваліфікаційна робота (дипломний проєкт) містить 107 сторінки, 3 таблиці, 34 рисунків, 3 формули, перелік джерел посилань з 23 найменувань, 2 додатки.

РОЗРОБКА ІНТЕЛЕКТУАЛЬНОЇ ІНФОРМАЦІЙНОЇ СИСТЕМИ УПРАВЛІННЯ ОСОБИСТИМИ ФІНАНСАМИ НА БАЗІ ТЕХНОЛОГІЙ OPEN BANKING

Перелік ключових слів: ВЕБСИСТЕМА, ОСОБИСТІ ФІНАНСИ, ШТУЧНИЙ ІНТЕЛЕКТ, OPEN BANKING, REACT, NODE.JS, POSTGRESQL, БАЗА ДАНИХ, СТАТИСТИКА, АНАЛІТИКА.

Об'єктом дослідження є процес автоматизованого управління особистими фінансами.

Метою кваліфікаційної роботи є розроблення інтелектуальної вебсистеми, яка забезпечує безпечну агрегацію, категоризацію та предиктивний аналіз фінансових транзакцій користувача.

Для досягнення поставленої мети було проведено аналіз предметної області, розроблено багаторівневу клієнт-серверну архітектуру, структуру реляційної бази даних та реалізовано інтеграцію з банківськими сервісами та ШІ-моделлю.

Інструменти розроблення кваліфікаційної роботи: IntelliJ IDEA, React, TypeScript, Node.js, Express.js, PostgreSQL (Supabase), Prisma ORM, Nginx, Docker, API Monobank/Wise, Google Gemini API, Microsoft Word.

Результатами виконання роботи є створення готової до експлуатації вебсистеми на власному домені, реалізація автоматичної синхронізації з банками, системи авторизації та криптографічного шифрування ключів, механізму ШІ-категоризації витрат та інтерактивного аналітичного дашборду.

Отримані результати можуть бути використані для подальшого розвитку фінтех-сервісів, PFM-систем та платформ автоматизованого фінансового консалтингу.

Рік виконання кваліфікаційної роботи – 2026.

Рік захисту кваліфікаційної роботи – 2026.

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ
УКРАЇНИ ВІДОКРЕМЛЕНИЙ
СТРУКТУРНИЙ ПІДРОЗДІЛ**

**«ФАХОВИЙ КОЛЕДЖ ІНФОРМАЦІЙНИХ СИСТЕМ І ТЕХНОЛОГІЙ
КИЇВСЬКОГО НАЦІОНАЛЬНОГО ЕКОНОМІЧНОГО УНІВЕРСИТЕТУ
ІМЕНІ ВАДИМА ГЕТЬМАНА»**

Відділення: «Програмування»

Освітньо-кваліфікаційний рівень: фаховий молодший бакалавр

Спеціальність: 121 «Інженерія програмного забезпечення»

ОПП: «Розробка програмного забезпечення»

«ЗАТВЕРДЖУЮ»

Голова циклової комісії

«Програмування» О.М. Помазун

«23» квітня 2026 р.

ЗАВДАННЯ

на виконання кваліфікаційної роботи (дипломного проєкту)

студента Гребенюка Владислава Миколайовича

(прізвище, ім'я, по батькові)

1. Тема кваліфікаційної роботи (дипломного проєкту):

«Розробка інтелектуальної інформаційної системи управління особистими фінансами на базі технологій Open Banking»

Керівник кваліфікаційної роботи (дипломного проєкту) Резуненко Станіслав Андрійович

(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

Тема затверджена Наказом по коледжу від «23» квітня 2026 р. № 39-С

2. Термін виконання кваліфікаційної роботи (дипломного проєкту) : з 25.03.2026 р. до 15.06.2026 р.

3. Вихідні дані до проєкту: Спроєктувати та розробити інтелектуальну інформаційну систему управління особистими фінансами з використанням технологій Open Banking.

4. Зміст пояснювальної записки (перелік питань, що підлягають розробці):

1. Характеристика та аналіз предметної області

2. Моделювання та проєктування програмного продукту

3. Розробка програмного продукту

4. Економічний розділ

5. Охорона праці

5. Перелік обов'язкового графічного матеріалу:

– UML діаграми (прецедентів, компонентів, діяльності);

– фізична модель даних для системи;

– специфікація даних системи;

– екранні форми інтерфейсу;

– результати роботи програмного продукту у вигляді знімків екрану (скріншотів).

КАЛЕНДАРНИЙ ПЛАН

№ п/п	Етапи виконання кваліфікаційної роботи (дипломного проєкту)	Термін виконання етапів
1.	Вибір теми. Отримання завдання	25.03.2026
2.	Збір інформації	23.04.2026
3.	Вивчення варіантів реалізації та вибір варіанту для розробки	23.04.2026 – 20.05.2026
4.	Розробка алгоритму та структури	20.05.2026 – 26.05.2026
5.	Розробка програмного продукту	20.05.2026 – 31.05.2026
6.	Розділ з охорони праці	27.05.2026 – 31.05.2026
7.	Економічний розділ	27.05.2026 – 31.05.2026
8.	Оформлення пояснювальної записки	31.05.2026- 05.06.2026
9.	Нормаконтроль	03.06.2026- 05.06.2026
10.	Перевірка кваліфікаційної роботи (дипломного проєкту) на плагіат	05.06.2026 – 10.06.2026
11.	Відгук керівника кваліфікаційної роботи (дипломного проєкту)	10.06.2026 - 12.06.2026
12.	Рецензування кваліфікаційної роботи (дипломного проєкту)	13.06.2026
13.	Отримання допуску до захисту та здача кваліфікаційної роботи (дипломного проєкту) завідувачому відділення	15.06.2026

Дата видачі завдання: 23.04.2026 р.

Студент _____

Владислав Гребенюк

Керівник
дипломного проєкту _____

Станіслав РЕЗУНЕНКО

ЗМІСТ

ВСТУП.....	6
1 ХАРАКТЕРИСТИКА ТА АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ.....	9
1.1 Характеристика предметної області.....	9
1.2 Аналіз існуючих рішень	11
1.2.1 Spendee (Чехія / Глобальний ринок)	12
1.2.2 Toshl Finance (Словенія / Глобальний ринок).....	13
1.2.3 Додаток Monobank (Україна)	15
1.3 Обґрунтування вибору підходів і технологій для створення вебсистеми	16
2 МОДЕЛЮВАННЯ ТА ПРОЕКТУВАННЯ ПРОГРАМНОГО ПРОДУКТУ	21
2.1 Специфікація вимог до створення	21
2.2 Архітектурні рішення.....	24
2.3 Алгоритм роботи програмного продукту	35
3 РОЗРОБКА ПРОГРАМНОГО ПРОДУКТУ	40
3.1 Обґрунтування вибору інструментальних засобів розроблення	40
3.2 Опис компонентів програмного продукту	42
3.3 Опис роботи програмного продукту.....	51
4 ЕКОНОМІЧНА ЧАСТИНА	60
4.1 Визначення цільової аудиторії ІТ-продукту	60
4.2 Визначення сильних та слабких сторін, можливостей та ризиків створення нового ІТ-продукту	62
4.3 Оцінка прибутковості проєкту створення і обслуговування ІТ-продукту ...	66
4.4 Оцінка економічної доцільності створення нового ІТ-продукту.....	69
5 ОХОРОНА ПРАЦІ.....	74
5.1 Правові та нормативні основи охорони праці в Україні.....	74
5.2 Виробнича санітарія	75
5.3 Виробнича безпека	80
5.4 Пожежна безпека	84
5.5 Безпека в надзвичайних ситуаціях	85
ВИСНОВКИ.....	88
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ	90
ДОДАТКИ.....	92

					227202.472 ЗТП					
Змн.	Арк.	№ докум.	Підпис	Дата	ПОЯСНЮВАЛЬНА ЗАПИСКА			Літ.	Арк.	Аркушів
Розроб.		В.Гребенюк								
Перевір.		С.РЕЗУНЕНКО								
Н. Контр.										
Затверд.								5	107	ПРОГРАМУВАННЯ, група 472

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

AES – симетричний алгоритм блочного шифрування даних.

API – інтерфейс для взаємодії між програмами.

JSON – текстовий формат обміну даними.

JWT – токен для підтвердження авторизації користувача.

LLM – велика мовна модель штучного інтелекту.

MVP – мінімально життєздатний продукт.

NPV – чиста приведена вартість інвестицій.

ORM – технологія зв'язку БД з об'єктним кодом.

PFM – система управління особистими фінансами.

REST – стиль побудови мережових вебсервісів.

ROI – показник рентабельності інвестицій.

SaaS – програмне забезпечення як послуга.

SWOT – метод аналізу сильних і слабких сторін.

UI/UX – інтерфейс і досвід взаємодії користувача.

BaaS – серверна інфраструктура (бекенд) як послуга.

CAPEX – капітальні інвестиції або початкові витрати на розробку.

DDoS – розподілена атака на відмову в обслуговуванні.

IRR – внутрішня норма прибутковості.

NLP – алгоритм обробки природної мови.

OPEX – поточні операційні витрати.

ESG – екологічне, соціальне та корпоративне управління; нефінансові фактори оцінки проєкту (Environmental, Social, and Governance).

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		6

ВСТУП

У сучасному цифровому суспільстві управління особистими фінансами є складною задачею через безліч мікротранзакцій та регулярних підписок. Традиційні додатки вимагають ручного введення даних, що займає багато часу, призводить до помилок та відмови від ведення бюджету. Впровадження концепції відкритого банкінгу (Open Banking) та алгоритмів штучного інтелекту створює умови для повної автоматизації цього процесу. Використання великих мовних моделей для глибокого предиктивного аналізу, автоматичної категоризації транзакцій та надання персоналізованих порад робить розробку інтелектуального вебзастосунку для фінансового контролю надзвичайно актуальною задачею. Вибір моделі Google Gemini як інтелектуального ядра системи зумовлений її високою здатністю до логічного аналізу контексту, якісною підтримкою української мови, надійною генерацією структурованих даних (формат JSON, що є критичним для програмної категоризації витрат), а також оптимальними умовами доступу до API для розгортання MVP-рішень.

Мета роботи полягає у створенні інтелектуальної інформаційної вебсистеми автоматизованого обліку та аналізу особистих фінансів для спрощення контролю витрат користувачів. Для досягнення мети вирішено комплекс задач: проаналізовано предметну область, спроектовано реляційну базу даних та зручний інтерфейс (UI/UX). Розроблено серверну й клієнтську частини з інтеграцією банківських API (Monobank, Wise) та ШІ (Google Gemini). Крім того, проведено тестування, виконано економічне обґрунтування рентабельності продукту та розроблено заходи з охорони праці.

Об'єктом дослідження є процес автоматизованого управління особистими фінансами. Предметом виступають програмно-інструментальні засоби розробки фінансової вебсистеми з використанням ШІ. У роботі застосовано методи системного аналізу, проектування баз даних, принципи об'єктно-орієнтованого програмування, а також методи економічного і функціонально-вартісного аналізу.

Створений продукт «Cashlytics» є повноцінним MVP-рішенням, яке можна

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		7

використовувати як готову SaaS-платформу для оптимізації щоденних витрат. Гнучка архітектура дозволяє легко масштабувати застосунок та підключати нові фінансові сервіси.

Під час виконання дипломного проєкту було використано такі інструментальні засоби:

- середовище розробки IntelliJ IDEA 2025.3.5 (Ultimate Edition);
- мови програмування TypeScript та JavaScript;
- система проєктування баз даних PostgreSQL;
- HTML5, CSS3, TS (ES6+) - для розробки інтерфейсу користувача (фронтенду);
- фреймворк React та бібліотеки Material-UI, Recharts для побудови інтерфейсу;
- середовище виконання Node.js та ORM-інструмент Prisma для роботи з базою даних;
- операційна система Windows 11;
- Supabase, хмарна онлайн-платформа з відкритим кодом, яка надає готову базу даних та інструменти автоматизації, дозволяючи створювати серверну частину додатків прямо через браузер без керування власними серверами.
- текстовий редактор Microsoft Word для підготовки і оформлення пояснювальної записки.

					227202.472 ПЗ	Арк.
						8
Змн.	Арк.	№ докум.	Підпис	Дата		

1 ХАРАКТЕРИСТИКА ТА АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Характеристика предметної області

Предметною областю даного дипломного проекту є програмні системи управління особистими фінансами (Personal Finance Management, PFM) та концепція відкритого банкінгу (Open Banking). Сучасні PFM-системи – це клас інформаційних систем, призначених для централізованого обліку, аналізу, планування та прогнозування доходів і витрат індивідуальних користувачів або домогосподарств.

Еволюція даної предметної області рухається від простих інструментів ручного введення даних (наприклад, електронних таблиць) до складних інтелектуальних систем, які здатні самостійно агрегувати фінансову інформацію через банківські API та використовувати методи машинного навчання для аналітики.

У даній предметній області основними елементами (сутностями) є:

- банківський рахунок (Account) – базове джерело зберігання коштів користувача (готівка, дебетові чи кредитні картки, електронні гаманці), що характеризується поточним балансом та валютою;
- транзакція (Transaction) – базова фінансова операція (дохід, витрата або переказ між рахунками), яка має суму, дату здійснення, опис та прив'язку до певної валюти;
- категорія (Category) – класифікатор, за допомогою якого транзакції групуються за економічним змістом (наприклад, «Продукти», «Транспорт», «Зарплата») для подальшого аналізу;
- бюджет (Budget) – встановлений фінансовий ліміт на певну категорію витрат із заданим періодом дії, який слугує інструментом самоконтролю;
- підписка (Subscription) – регулярний, періодичний платіж за цифрові або фізичні послуги, що вимагає окремого моніторингу через свою циклічну природу;

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		9

— аналітичний дашборд – інструмент візуалізації даних, що представляє зведену статистику у вигляді графіків, діаграм та звітів для полегшення сприйняття фінансової ситуації.

У загальному випадку, користувачі сучасних систем управління фінансами мають змогу:

- вести облік усіх своїх доходів та витрат у єдиному середовищі;
- розподіляти фінансові потоки за категоріями для розуміння структури витрат;
- встановлювати бюджетні ліміти та відстежувати їх виконання;
- переглядати історію транзакцій за різні часові періоди;
- аналізувати візуальні звіти для пошуку шляхів оптимізації витрат;
- управляти коштами в різних валютах.

З технічної точки зору, класичні PFM-системи виконують такі фонові процеси:

- збереження та структурування даних у базах даних;
- математична обробка масивів даних (розрахунок кумулятивних сум, балансів);
- конвертація валют за курсами;
- побудова аналітичних вибірок для візуалізації;
- категоризація нових записів (за допомогою регулярних виразів або алгоритмів штучного інтелекту).

Цільова аудиторія предметної області:

- особи, які прагнуть свідомо контролювати свої витрати та підвищувати рівень фінансової грамотності;
- фрілансери, незалежні контрактори та дрібні підприємці, які потребують розмежування особистих та робочих фінансів;
- домогосподарства, що планують спільний сімейний бюджет;
- користувачі з великою кількістю регулярних платежів (підписок), які потребують єдиного центру для їх моніторингу.

					227202.472 ПЗ	Арк.
						10
Змн.	Арк.	№ докум.	Підпис	Дата		

Переваги використання автоматизованих систем обліку фінансів:

- підвищення прозорості особистого бюджету та виявлення непомітних, але регулярних витрат;
- можливість прийняття обґрунтованих фінансових рішень на основі точної статистики, а не інтуїції;
- економія часу завдяки автоматизації розрахунків та побудови графіків.

Незважаючи на високий попит, більшість існуючих рішень у даній предметній області мають суттєві недоліки, які відштовхують користувачів:

- Проблема ручного введення: Абсолютна більшість систем вимагають від користувача самотійно вносити кожен транзакцію, що призводить до швидкої втрати інтересу до ведення бюджету.
- Примітивна категоризація: Існуючі механізми автокатегоризації часто базуються на жорстких правилах (регулярних виразах), які не розуміють контексту платежу і часто помиляються.
- Відсутність інтелектуального аналізу: Більшість додатків лише констатують факт витрат (показують графік), але не дають персоналізованих порад чи прогнозів щодо того, як ситуація розвиватиметься до кінця місяця.
- Складність мультивалютного обліку: В умовах глобалізації користувачі часто мають рахунки в різних валютах, проте далеко не всі системи здатні коректно і в реальному часі зводити їх до єдиного знаменника.

Саме наявність цих нерозв'язаних проблем в існуючій предметній області зумовлює необхідність розробки нової, більш інтелектуальної системи, яка завдяки відкритому банкінгу та штучному інтелекту зможе нівелювати описані недоліки.

1.2 Аналіз існуючих рішень

У сфері управління особистими фінансами (Personal Finance Management, PFM) існує велика кількість мобільних та вебзастосунків. Аналіз існуючих рішень на ринку доцільно розділити на дві категорії: безпосередні конкуренти (окремі спеціалізовані сервіси для ведення бюджету) та суміжні області (вбудовані аналітичні модулі в банківських застосунках). Такий підхід дозволяє визначити

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		11

ключові функціональні можливості сучасних систем, а також виявити їхні критичні недоліки, щоб врахувати їх при проєктуванні власного інтелектуального вебзастосунку.

1.2.1 Spendee (Чехія / Глобальний ринок)

Spendee – популярний міжнародний сервіс для управління особистими та сімейними бюджетами. Сервіс дозволяє користувачам створювати «гаманці», встановлювати фінансові цілі та ділитися доступом до спільного бюджету з іншими користувачами.

Основні можливості:

- створення декількох гаманців у різних валютах;
- ручне додавання транзакцій та імпорт банківських виписок (у преміум-версії);
- спільний доступ до фінансових даних;
- можливість створення власних категорій та бюджетних лімітів;
- детальні графіки та звіти про рух коштів.

Переваги:

- детальні графіки та звіти про рух коштів.
- використання алгоритмів машинного навчання (ML) та технологій оптичного розпізнавання символів (OCR) для автоматизованого сканування чеків та базової предиктивної категоризації;
- підтримка мультивалютності та ручного ведення готівкових коштів;
- наявність гнучкої системи лімітів та бюджетування;
- кроссплатформність (наявність як мобільного застосунку, так і вебверсії).

Недоліки:

- банківська синхронізація доступна виключно у платній підписці (Premium);
- висока затримка при синхронізації з деякими банками Східної Європи;

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		12

- процес категоризації часто вимагає ручного втручання користувача через недосконалість стандартних алгоритмів розпізнавання банківських описів;
- використання традиційних ML-класифікаторів замість генеративних великих мовних моделей (LLM), що обмежує систему жорсткими рамками і не дозволяє проводити глибокий семантичний аналіз нестандартних транзакцій;
- відсутність діалогового інтерфейсу (чат-бота), здатного розуміти природну мову користувача та генерувати контекстно-залежні фінансові поради.

Зовнішній вигляд аналітичної панелі вебплатформи представлено на рисунку 1.1.



Рисунок 1.1 – Інтерфейс аналітики на вебплатформі Spendee

1.2.2 Toshl Finance (Словенія / Глобальний ринок)

Toshl Finance – ще один відомий фінансовий трекер, який вирізняється гейміфікованим підходом до ведення бюджету та унікальним дизайном. Сервіс фокусується на допомозі користувачам дотримуватись щоденних лімітів витрат.

Основні можливості:

- ведення доходів і витрат з використанням тегів замість традиційних жорстких категорій;
- агрегація банківських рахунків (через сервіси-посередники типу Salt Edge);

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		13

- візуалізація бюджету у вигляді «фінансових монстрів», які реагують на перевитрати;
- перерахунок іноземних валют за актуальним курсом.

Переваги:

- нестандартний візуальний дизайн, що зменшує психологічний тиск від контролю витрат;
- потужна система тегування, яка дозволяє прив'язувати транзакцію одразу до кількох подій (наприклад, #їжа, #відпустка_2024);
- детальний контроль щоденних та щотижневих лімітів.

Недоліки:

- інтерфейс вебверсії перевантажений анімаціями та може працювати повільно на слабких комп'ютерах;
- відсутність вбудованого чату чи розумного асистента для швидкого доступу до даних;
- відсутня функція автоматичного розпізнавання регулярних платежів (підписок);
- висока вартість річної підписки для використання повноцінної банківської синхронізації.

Графічний інтерфейс управління бюджетом та фінансовими лімітами наведено на рисунку 1.2.

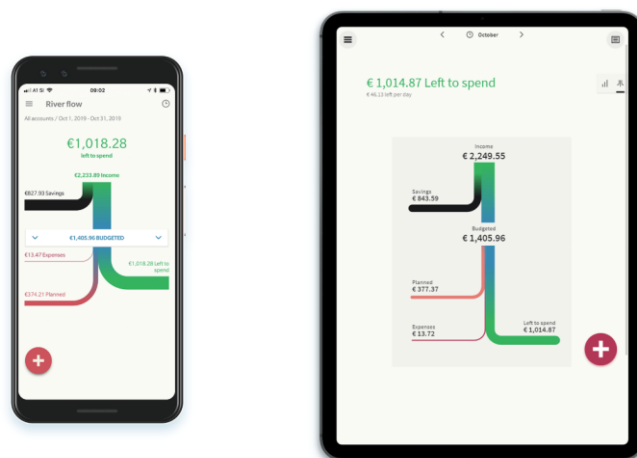


Рисунок 1.2 – Дашборд управління фінансами у сервісі Toshl Finance

					227202.472 ПЗ	Арк.
						14
Змн.	Арк.	№ докум.	Підпис	Дата		

1.2.3 Додаток Monobank (Україна)

Monobank - один із найпопулярніших необанків в Україні, який у своєму мобільному додатку пропонує вбудовану систему аналітики витрат. Система автоматично фіксує всі транзакції клієнта та розподіляє їх за базовими категоріями.

Основні можливості:

- автоматична фіксація всіх транзакцій за картками банку;
- розподіл витрат за стандартними категоріями (продукти, кафе, розваги тощо);
- перегляд виписки за обраний місяць у вигляді кругової діаграми;
- управління лімітами на інтернет-оплати.

Переваги:

- відсутність потреби у ручному введенні транзакцій (усі дані беруться з процесингу банку);
- зручний та інтуїтивно зрозумілий мобільний інтерфейс;
- миттєве оновлення балансу після здійснення операції.

Недоліки:

- замкнута екосистема: неможливо додати витрати готівкою або транзакції з рахунків інших банків (наприклад, Wise, ПриватБанк);
- жорстка категоризація: користувач не може створювати власні унікальні категорії (наприклад, окрему категорію для ремонту певного автомобіля);
- відсутність предиктивної аналітики (додаток лише констатує факт витрат, але не прогнозує їх до кінця місяця);
- відсутність повноцінної вебверсії для глибокого аналізу фінансів на персональному комп'ютері.

Реалізація модуля аналітики витрат безпосередньо у мобільному застосунку банку зображена на рисунку 1.3.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		15

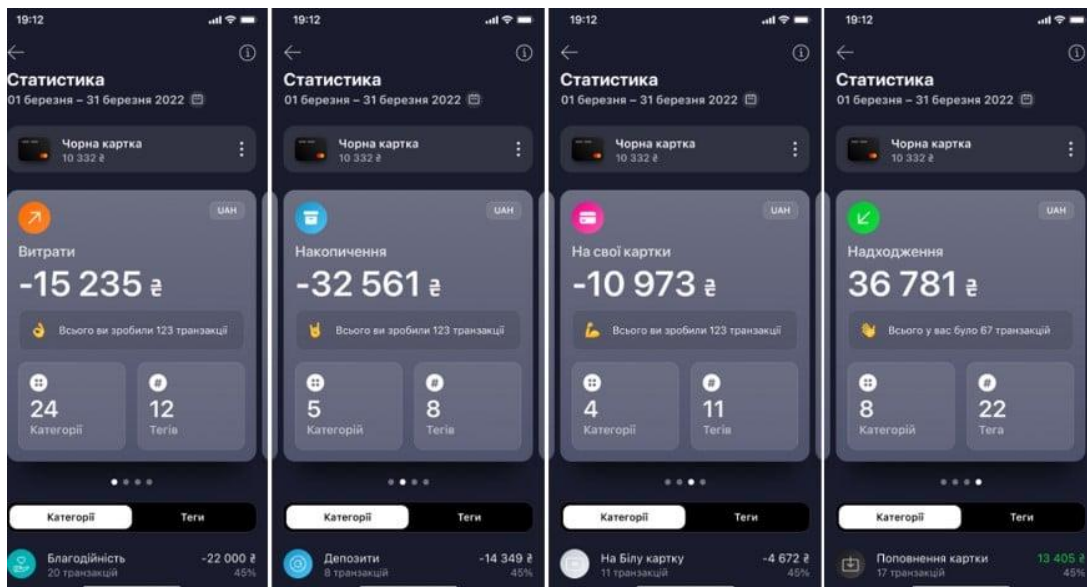


Рисунок 1.3 – Інтерфейс аналітики витрат у додатку Monobank

Проведений аналіз показує, що хоча на ринку існує багато рішень для управління фінансами, більшість із них або вимагають виснажливого ручного введення даних, або мають закриту архітектуру, що не дозволяє об'єднати рахунки різних банків. Варто зазначити, що провідні продукти (наприклад, Spendee) вже починають впроваджувати елементи штучного інтелекту, зокрема технології машинного навчання (ML) та OCR для сканування чеків і базового предиктивного розподілу витрат. Однак жоден із розглянутих застосунків не використовує сучасні генеративні великі мовні моделі (LLM) як ядро системи. Відсутність LLM унеможливорює інтелектуальну обробку неструктурованих описів платежів на глибокому семантичному рівні, автоматичний пошук прихованих підписок та, найголовніше, комунікацію з користувачем через інтерактивного фінансового чат-асистента. Це обґрунтовує необхідність створення власного вебзастосунку, який поєднає пряму API-синхронізацію рахунків із потужністю сучасних генеративних неймереж.

1.3 Обґрунтування вибору підходів і технологій для створення вебсистеми

Під час проєктування вебсистеми «Розробка інтелектуальної інформаційної системи управління особистими фінансами на базі технологій Open Banking»

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		16

критично важливим етапом був вибір методології розробки та технологічного стеку. Оскільки система працює з конфіденційними фінансовими даними користувачів та виконує складні математичні обчислення в режимі реального часу, обрані інструменти повинні забезпечувати високу надійність, безпеку, швидкість обробки даних та гнучкість для подальшого масштабування.

Для створення даного програмного продукту було обрано ітеративно-інкрементну модель розробки. Враховуючи те, що система передбачає інтеграцію зі сторонніми банківськими сервісами (API Monobank, Wise) та моделями штучного інтелекту, використання каскадної моделі (Waterfall) було б неефективним через високий ризик зміни структур даних з боку зовнішніх API.

Ітеративно-інкрементна модель дозволила розбити складний процес розробки фінансового трекера на логічні етапи (інкременти). Процес розробки складався з наступних циклів:

- Ініціалізація та базова архітектура: розробка мінімально життєздатного продукту (MVP), що включав створення бази даних, налаштування маршрутизації та базовий інтерфейс для ручного додавання транзакцій.
- Ітерація інтеграції: підключення банківських API, тестування механізмів авторизації за допомогою токенів та отримання реальних балансів рахунків.
- Ітерація інтелектуалізації: впровадження API Google Gemini для обробки неструктурованих текстів виписок, розробка алгоритмів визначення категорій та логіки чат-бота.
- Ітерація аналітики та UI: побудова аналітичного дашборду, налаштування інтерактивних графіків (Recharts) та фінальне тестування логіки мультивалютних конвертацій.

Такий підхід дозволив на кожному етапі мати працюючу версію програми, тестувати складну математичну логіку (наприклад, розрахунок кумулятивних витрат) ізольовано від інших модулів і оперативно виправляти помилки.

Для реалізації інтерфейсу користувача було обрано бібліотеку React. Її компонентний підхід ідеально підходить для створення дашбордів, оскільки

дозволяє розбити складний інтерфейс на незалежні модулі (картка балансу, графік, таблиця транзакцій). Використання віртуального DOM (Virtual DOM) у React забезпечує миттєве оновлення графіків при зміні користувачем валюти або часового періоду без перезавантаження сторінки.

Мовою програмування обрано TypeScript. Для фінансового застосунку сувора статична типізація є критичною вимогою. В JavaScript випадкове додавання числа до рядка (наприклад, $100 + \text{«50»}$) може призвести до фатальних помилок у розрахунку балансів. TypeScript виявляє такі помилки ще на етапі написання коду, гарантуючи, що суми, ліміти та валюти обробляються виключно у правильних форматах.

Для візуального оформлення використано бібліотеку компонентів Material-UI (MUI). Вона надає готові, адаптивні та естетично привабливі елементи інтерфейсу (таблиці, випадаючі списки, картки), що відповідають стандартам сучасного дизайну. Для побудови аналітичних діаграм та прогнозуючих графіків обрано бібліотеку Recharts, яка легко інтегрується з React і підтримує динамічне перемальовування осей при зміні обсягу даних.

Серверну частину реалізовано на базі середовища Node.js з використанням фреймворку Express.js. Node.js використовує асинхронну, подієво-орієнтовану архітектуру (Event Loop), що є оптимальним для нашої системи. Коли користувач натискає кнопку «Синхронізувати», сервер одночасно робить запити до API банку, API курсів валют та API штучного інтелекту. Node.js здатний ефективно обробляти ці паралельні мережеві запити без блокування основного потоку виконання.

Для зберігання даних використано реляційну систему управління базами даних (PostgreSQL). Вибір реляційної моделі зумовлений жорсткою структурою фінансових даних, де існують чіткі зв'язки: користувач має багато рахунків, рахунок має багато транзакцій, транзакція належить до певної категорії.

Для взаємодії серверної частини з базою даних застосовано сучасний ORM-інструмент (Object-Relational Mapping) – Prisma. Prisma автоматично генерує TypeScript-типи на основі схеми бази даних, що забезпечує абсолютну безпеку даних на всіх рівнях: від клієнта до сервера і бази даних.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		18

Замість написання сотень регулярних виразів (Regex) для розпізнавання банківських описів, було прийнято рішення інтегрувати генеративну велику мовну модель (LLM) – Google Gemini (модифікація Flash-Lite). Ця модель вирізняється високою швидкістю відповіді та низьким споживанням ресурсів, що ідеально підходить для категоризації транзакцій у реальному часі. ШІ здатний розуміти семантичний контекст (наприклад, відрізнити переказ коштів на картку від покупки в інтернет-магазині) та підтримувати діалог з користувачем на основі його фінансової історії.

Для отримання банківських виписок використано відкриті REST API сервісів Monobank та Wise. Взаємодія з ними відбувається за допомогою HTTP-клієнта Axios з використанням криптографічно захищених персональних токенів доступу користувачів.

Для безпечного розгортання вебсистеми у середовищі реальної експлуатації (Production) було обрано високопродуктивний вебсервер Nginx у ролі зворотного проксі (Reverse Proxy). Використання Nginx дозволило налаштувати захищене з'єднання за протоколом HTTPS із використанням SSL-сертифікатів, що є критичною вимогою для фінансових застосунків. З метою кіберзахисту серверної частини від DDoS-атак та спроб автоматизованого підбору паролів (brute-force) до технологічного стеку Node.js було інтегровано механізм обмеження частоти запитів (Rate Limiter). Завдяки правильному налаштуванню проксіювання, бекенд здатен коректно зчитувати реальні IP-адреси клієнтів із заголовка X-Forwarded-For.

Таким чином, обраний технологічний стек (React, TypeScript, Node.js, Prisma, Gemini AI) дозволив створити сучасну, безпечну та високопродуктивну вебсистему з гнучкою клієнт-серверною архітектурою.

Висновки до розділу 1

У першому розділі було проведено глибокий аналіз предметної області управління особистими фінансами. Визначено основне призначення розроблюваної системи Cashlytics, її ключові функціональні можливості, структуру основних сутностей (рахунки, транзакції, бюджети) та цільову аудиторію. Встановлено, що автоматизація процесу фінансового обліку за допомогою

сучасних API є високоактуальним завданням, оскільки звільняє користувача від рутини та підвищує точність даних.

Під час дослідження існуючих аналогів (Monobank, Spendee, Toshl) було виявлено, що більшість сучасних рішень обмежені або закритою екосистемою окремого банку, або відсутністю інтелектуальних інструментів для семантичного аналізу витрат.

У межах розділу також було обґрунтовано вибір ітеративно-інкрементної моделі розробки програмного забезпечення, що забезпечує гнучкість у роботі зі сторонніми API. Детально аргументовано вибір технологічного стеку: React, Material-UI та SVG-графіка для розробки сучасного клієнтського інтерфейсу; TypeScript для забезпечення суворої типізації фінансових даних; Node.js, Express.js та Prisma ORM для побудови безпечного серверного ядра; вебсервер Nginx для маршрутизації та забезпечення HTTPS-з'єднання; Google Gemini API для реалізації функцій штучного інтелекту.

Таким чином, у першому розділі було сформовано повноцінне теоретичне та технологічне підґрунтя, необхідне для переходу до наступного етапу – безпосереднього проєктування архітектури системи та структури бази даних вебзастосунку.

2 МОДЕЛЮВАННЯ ТА ПРОЕКТУВАННЯ ПРОГРАМНОГО ПРОДУКТУ

2.1 Специфікація вимог до створення

Під час розроблення вебсистеми «Розробка інтелектуальної інформаційної системи управління особистими фінансами на базі технологій Open Banking» (Cashlytics) було визначено основні вимоги до програмного продукту. Чітка специфікація вимог гарантує коректну роботу системи, зручність взаємодії з користувачем, а також високий рівень безпеки при обробці конфіденційних фінансових даних та банківських токенів доступу.

Основною метою розроблюваної системи є створення централізованої, автоматизованої платформи для агрегації, аналізу та прогнозування особистих фінансів користувача з використанням штучного інтелекту.

До основних бізнес-вимог належать:

- зведення до мінімуму ручного введення фінансових даних користувачем;
- об'єднання мультивалютних рахунків з різних банківських систем (Monobank, Wise) у єдиний аналітичний простір;
- автоматизація процесу категоризації витрат для точного ведення статистики;
- забезпечення користувача проактивною фінансовою аналітикою (виявлення регулярних підписок, прогнозування витрат до кінця місяця);
- надання доступу до персоналізованих фінансових консультацій через інтегрованого ШІ-асистента.

Функціональні вимоги:

- презентація можливостей системи новим (неавторизованим) користувачам через інтерактивну посадкову сторінку (Landing Page);
- дотримання міжнародних стандартів конфіденційності (GDPR) шляхом реалізації механізму згоди на використання файлів cookie (Cookie-банер) та

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		21

розміщення юридичної документації (Privacy Policy, Terms of Service);

- реєстрація, авторизація та управління профілем користувача;
- підключення власних банківських рахунків шляхом додавання ключів доступу через налаштування;
- ручна (примусова) та автоматична синхронізація балансів і нових транзакцій з банками;
- перегляд зведеної статистики (доходи, витрати, чистий баланс) на головному аналітичному дашборді;
- перегляд детальної історії транзакцій із можливістю фільтрації за часовими періодами;
- автоматична категоризація кожної витрати з присвоєнням відповідного емоді;
- створення, редагування, відстеження прогресу та видалення бюджетних лімітів (на день, тиждень, місяць);
- перегляд автоматично сформованого списку активних регулярних підписок користувача;
- перемикання валюти відображення (UAH, USD, EUR, PLN тощо) з миттєвою конвертацією всіх балансів і графіків за актуальним курсом;
- перегляд кумулятивних витрат та оптимістичного, реалістичного і песимістичного прогнозів витрат до кінця періоду;
- ведення діалогу в інтерактивному чаті зі ШІ-асистентом для отримання текстових консультацій щодо власних фінансів.

До нефункціональних вимог системи належать:

- Безпека та аутентифікація: сесії користувачів повинні управлятися за допомогою безпечних JWT-токенів. Усі паролі мають хешуватися (алгоритмом argon2), а банківські API-токени – підлягати двосторонньому криптографічному шифруванню (AES-256) перед збереженням у базу даних. Система має бути захищена від SQL-ін'єкцій завдяки використанню ORM-інструменту (Prisma). Для захисту від DDoS-атак та автоматизованого підбору паролів (brute-force) повинен використовуватись модуль обмеження частоти запитів (Rate Limiting).

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		22

– Мережева стійкість та маршрутизація: використання високопродуктивного вебсервера Nginx у ролі зворотного проксі (Reverse Proxy) для забезпечення шифрованого HTTPS-з'єднання та коректного зчитування реальних IP-адрес клієнтів із заголовка X-Forwarded-For.

– Інтеграція та обробка даних (ШІ): процес визначення категорій та формування відповідей чат-бота повинен відбуватися «під капотом» шляхом передачі неструктурованих описів транзакцій та контексту БД до API Google Gemini.

– Продуктивність: завантаження аналітичного дашборду має відбуватися без значних затримок. Запити до зовнішніх API банків та нейромереж повинні виконуватися асинхронно, щоб уникати блокування інтерфейсу під час тривалої синхронізації даних.

– Масштабованість: архітектура реляційної бази даних має передбачати можливість швидкого та легкого підключення нових банківських провайдерів у майбутньому.

– Ергономіка (UX/UI): вебзастосунок повинен мати інтуїтивно зрозумілу візуалізацію даних, підтримувати адаптивний дизайн для коректного відображення на мобільних пристроях та десктопах, а також містити функцію перемикання між світлою та темною темами оформлення.

Під час розробки та експлуатації вебсистеми було визначено наступні потенційні ризики:

– Обмеження (Rate Limits) з боку банківських API: банк може тимчасово заблокувати запити від сервера у разі їх надмірної частоти (наприклад, Monobank дозволяє лише 1 запит на хвилину для отримання виписки).

– Нестабільність роботи API штучного інтелекту: у разі перебоїв на серверах Google Gemini процес категоризації нових транзакцій або робота чат-бота може бути тимчасово недоступною.

– Ризик «галюцинацій» ШІ: модель може неправильно зрозуміти контекст транзакції (наприклад, віднести покупку в аптеці до категорії «Розваги»), що потребує можливості ручного редагування категорій.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		23

– Витік конфіденційної інформації: у разі компрометації сервера зломисники можуть отримати доступ до фінансової історії користувачів, що вимагає жорсткого дотримання протоколів безпеки на етапі розгортання застосунку.

Специфікацію функціональних можливостей та архітектуру взаємодії користувача з системою було візуалізовано за допомогою діаграм UML. На рисунку 2.1 представлено діаграму прецедентів (Use Case Diagram), яка ілюструє основні сценарії використання вебзастосунку кінцевим користувачем.

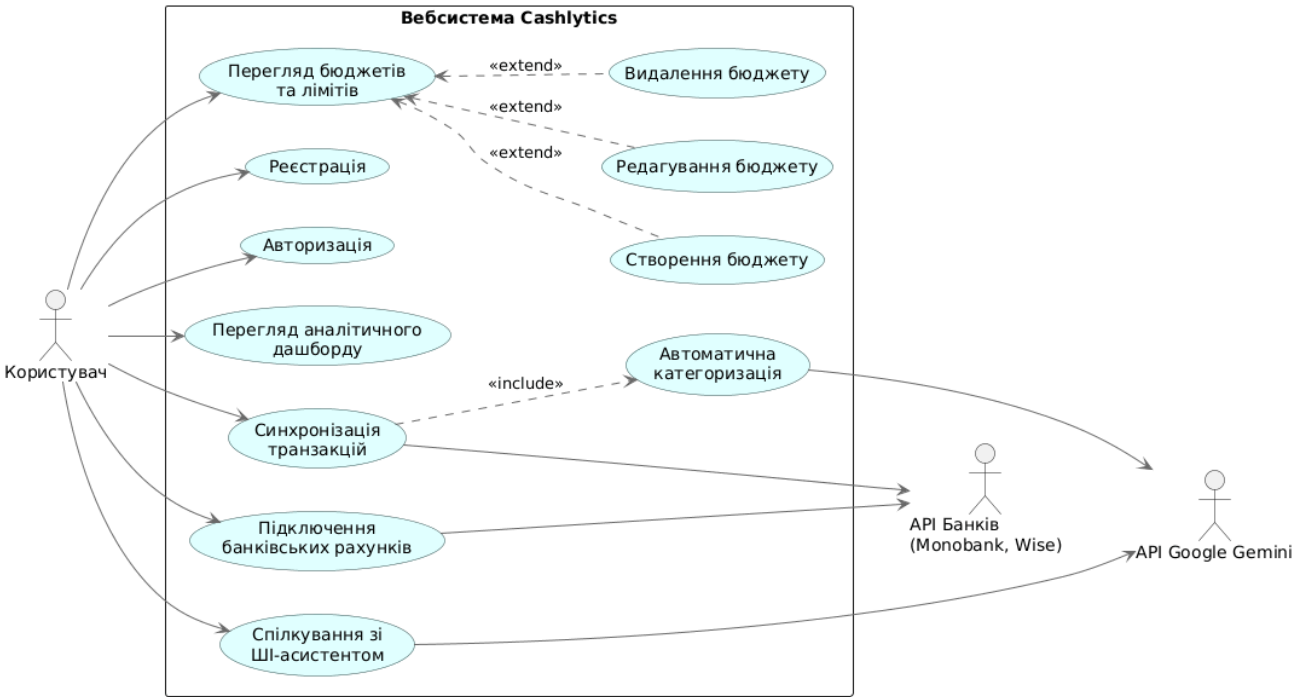


Рисунок 2.1 – Діаграма прецедентів

2.2 Архітектурні рішення

Для розроблення вебсистеми «Cashlytics» (було обрано сучасну багаторівневу клієнт-серверну архітектуру. Такий підхід є оптимальним, оскільки система обробляє конфіденційні фінансові дані, виконання бізнес-логіки яких має бути ізольовано на захищеному сервері, тоді як клієнтська частина відповідає виключно за візуалізацію аналітики та взаємодію з користувачем.

Клієнтська частина (Frontend) відповідає за відображення інтерфейсу користувача: презентаційної посадкової сторінки (Landing Page), аналітичного

дашборду, карток банківських рахунків, графіків кумулятивних витрат, таблиці транзакцій, юридичних сторінок та вікна ШІ-чату. Вона реалізована за допомогою бібліотеки React та мови TypeScript. Завдяки віртуальному DOM (Virtual DOM) інтерфейс миттєво перемальовується при зміні користувачем цільової валюти або часового періоду. Для оптимізації швидкості завантаження та забезпечення високої якості графіки на сучасних дисплеях базові елементи інтерфейсу реалізовані у форматі чистої векторної графіки з використанням CSS-фільтрів.

Серверна частина (Backend) відповідає за маршрутизацію (REST API), захист ендпоінтів від DDoS-атак та автоматизованого підбору паролів, безпечне збереження API-токенів, виконання крос-валютних конвертацій та зв'язок зі сторонніми сервісами (API Monobank, API Wise, Google Gemini API). Вона побудована на базі середовища виконання Node.js із використанням фреймворку Express.js. Для забезпечення безпечного HTTPS-з'єднання та маршрутизації перед серверним ядром розгорнуто високопродуктивний вебсервер Nginx у ролі зворотного проксі.

База даних забезпечує персистентне зберігання інформації про користувачів, їхні рахунки, історію транзакцій, встановлені бюджети та виявлені підписки. Взаємодія сервера з базою даних реалізована через сучасний об'єктно-реляційний відображувач (ORM) – Prisma.

До складу розробленої вебсистеми входять такі логічні модулі:

- модуль авторизації та безпеки – забезпечує реєстрацію, вхід користувачів (JWT-токени) та криптографічне шифрування ключів доступу до банків;
- модуль маршрутизації та проксіювання (Nginx) – забезпечує термінацію SSL-сертифікатів, захищене HTTPS-з'єднання та перенаправлення трафіку;
- модуль банківської синхронізації – виконує асинхронні запити до банків для отримання поточних балансів рахунків та історії платежів;
- модуль інтелектуальної обробки (ШІ) – відповідає за передачу неструктурованих описів транзакцій до LLM-моделі для визначення категорій та

- підтримку діалогового чату;
- модуль агрегації та аналітики – обчислює кумулятивні суми, генерує оптимістичні/песимістичні прогнози та вираховує відсотки виконання бюджетів;
 - модуль інтерфейсу (Дашборд) – візуалізує оброблені дані за допомогою компонентів Material-UI та графіків Recharts.

Користувач взаємодіє з системою через клієнтський вебінтерфейс. Наприклад, при натисканні кнопки «Синхронізувати з банком», клієнтська частина надсилає захищений HTTP-запит до серверного модуля синхронізації. Сервер розшифровує токен користувача, звертається до зовнішнього банківського API, отримує сирі дані виписки, передає їх до модуля ІШІ для категоризації, зберігає результати через Prisma у базу даних і повертає клієнту оновлену аналітику для відображення на графіках.

Діаграма компонентів ілюструє структурні елементи програмної системи та зв'язки між ними (Рисунок 2.2).

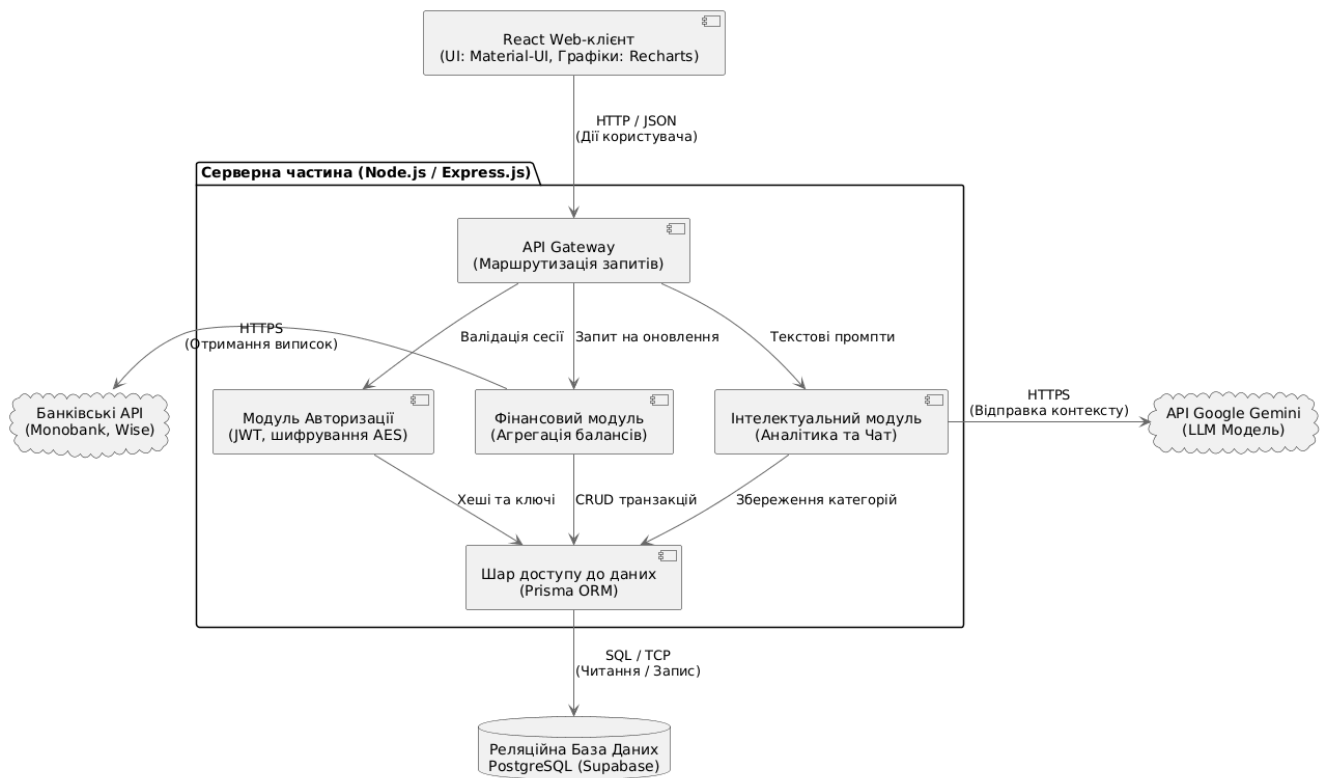


Рисунок 2.2 – Діаграма компонентів вебсистеми Cashlytics

Для збереження даних у проєкті було обрано реляційну базу даних

(PostgreSQL) у зв'язці з інструментом Prisma ORM. Фінансові застосунки вимагають високої надійності (ACID-транзакцій) та суворої структурованості даних. У системі присутні чіткі реляційні зв'язки: один користувач має багато рахунків, один рахунок пов'язаний з багатьма транзакціями, кожна транзакція належить до певної категорії.

Переваги обраного підходу (PostgreSQL + Prisma):

- суворі типізація бази даних на рівні програмного коду, що унеможливорює збереження фінансових сум у неправильному форматі;
- автоматична генерація SQL-міграцій при зміні структури таблиць;
- висока швидкість виконання складних агрегаційних запитів (сумування витрат за місяць, групування за категоріями).

У базі даних вебсистеми спроектовано шість основних сутностей (таблиць), пов'язаних між собою:

Таблиця User (інформація про користувачів):

- id – унікальний ідентифікатор (формат UUID). Обґрунтування: використання UUID замість порядкових цілих чисел (Auto-Increment) є стандартом безпеки, оскільки унеможливорює підбір ідентифікаторів інших користувачів системи.

- email – електронна пошта (логін), яка має обмеження на унікальність.
- passwordHash – захешований пароль. Обґрунтування: система ніколи не зберігає паролі у відкритому вигляді для уникнення компрометації облікових записів.

- monoToken, wiseToken – ключі доступу до банків. Обґрунтування: ці поля зберігають токени виключно у зашифрованому вигляді (криптографічний алгоритм AES). Це критичне архітектурне рішення, яке гарантує, що навіть у разі отримання злоумисниками доступу до бази даних, вони не зможуть скористатися банківськими ключами клієнтів.

- createdAt – дата створення облікового запису.

Таблиця Account (банківські рахунки):

- id – ідентифікатор рахунку. Обґрунтування: як первинний ключ тут

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		27

використовується оригінальний ID рахунку, який повертає банківське API. Це гарантує ідемпотентність: при повторній синхронізації система не створить дублікат рахунку, а лише оновить його стан.

- name – назва картки (наприклад, «Monobank Чорна»).

- balance – поточний залишок коштів. Обґрунтування: використовується тип даних з плаваючою комою (або DECIMAL), оскільки фінансові операції вимагають точного збереження копійок / центів.

- currency – валюта рахунку. Обґрунтування: зберігається у форматі трилітерного ISO-коду (UAH, USD, EUR) для коректної крос-валютної конвертації на клієнті.

- source – джерело даних («MONO», «WISE»). Дозволяє бекенду розуміти, до якого саме API звертатися для оновлення конкретного рахунку.

- userId – зовнішній ключ до користувача.

Таблиця Transaction (історія операцій):

- id – унікальний ідентифікатор платежу від банку (використовується для запобігання дублюванню транзакцій при багаторазовому опитуванні API).

- date – дата та час проведення операції у форматі TIMESTAMP для можливості побудови графіків по місяцях та днях тижня.

- description – оригінальний опис платежу від банку, який в подальшому передається III-моделі як контекст для категоризації.

- amount – сума операції. Обґрунтування: додатні значення зберігаються для доходів, від'ємні – для витрат. Це суттєво спрощує математичні агрегації на рівні SQL-запитів.

- categoryId – зовнішній ключ до категорії (може бути порожнім до моменту опрацювання штучним інтелектом).

- subscriptionId – зовнішній ключ на таблицю підписок. Обґрунтування: поле містить правило onDelete: SetNull. Це означає, що якщо користувач видалить підписку на сервіс, самі транзакції (гроші, які вже списалися в минулому) не видаляються і не зіпсують історичну фінансову статистику.

Таблиця Category (категорії витрат):

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		28

- id – унікальний ідентифікатор.
- name – назва категорії (наприклад, «Продукти», «Транспорт»).
- icon – текстове поле для збереження емодзі. Обґрунтування: ШІ-модель Google Gemini автоматично підбирає емодзі до назви. Це дозволяє зробити інтерфейс (UI) більш сучасним без необхідності зберігати важкі файли зображень чи іконок у базі даних.

- userId – зовнішній ключ. Обґрунтування: це поле дозволяє кожному користувачу мати власний, персоналізований набір категорій, а не залежати від жорстко заданого системного списку. Встановлено обмеження унікальності на комбінацію [name, userId], щоб уникнути дублювання категорій в одного користувача.

Таблиця Budget (фінансові ліміти):

- id – унікальний ідентифікатор.
- categoryName – назва цільової категорії.
- amount – сума фінансового обмеження.
- period – часовий проміжок («DAILY», «WEEKLY», «MONTHLY»).

Обґрунтування: замість збереження лише місячних бюджетів, такий підхід забезпечує гнучкість системи, дозволяючи користувачам встановлювати як довгострокові цілі, так і жорсткі денні ліміти на конкретні потреби.

Таблиця Subscription (виявлені цифрові підписки):

- id – унікальний ідентифікатор.
- name – назва сервісу (Netflix, Spotify тощо).
- amount та currency – вартість та валюта регулярного платежу.
- billingCycle – періодичність списання (місяць/рік).

Взаємозв'язки між усіма розробленими сутностями бази даних продемонстровано на ER-діаграмі (Рисунок 2.3).

					227202.472 ПЗ	Арк.
						29
Змн.	Арк.	№ докум.	Підпис	Дата		

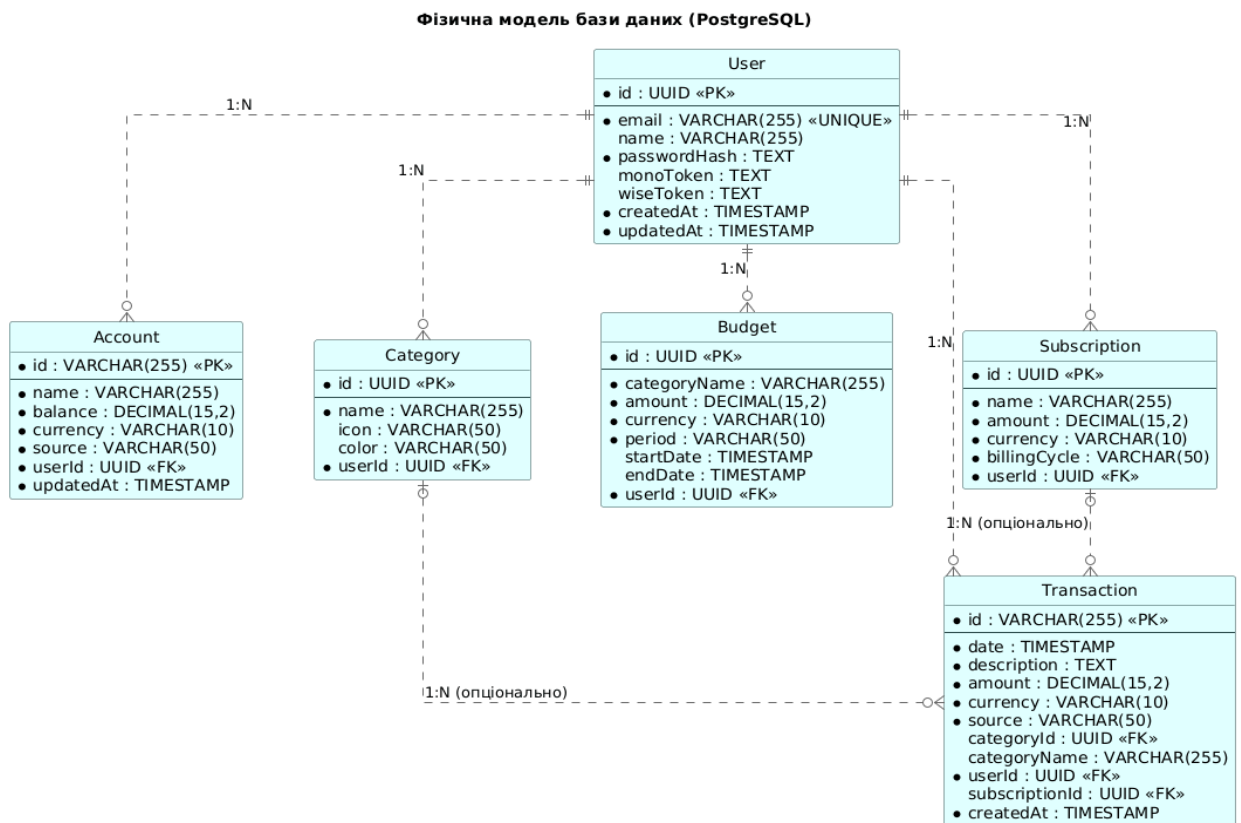


Рисунок 2.3 – Фізична модель бази даних

Оскільки вебсистема має розгалужений та інформативний графічний інтерфейс, у звіті представлено макети (скріншоти) основних екранів готового програмного продукту:

- форма реєстрації та авторизації;
- презентаційна посадкова сторінка (Landing Page) із демонстрацією переваг;
- сторінка налаштувань для введення банківських API-токенів;
- головний аналітичний дашборд (картки загального балансу, блок «Мої рахунки», графіки доходів та витрат);
- інтерактивний блок ЧП-чату з відображенням згенерованих порад;
- розділ контролю бюджетних лімітів та переліку виявлених підписок.

Оскільки вебсистема має розгалужений та інформативний графічний інтерфейс, у звіті представлено макети (скріншоти) основних екранів готового програмного продукту. Інтерфейс розроблено з урахуванням принципів адаптивного дизайну (Responsive Web Design) та сучасних вимог до

користувачького досвіду (UX/UI).

На рисунках 2.4 – 2.14 зображено ключові модулі та сторінки графічного інтерфейсу користувача, які охоплюють усі етапи взаємодії із системою: від початкової автентифікації до глибокої фінансової аналітики та налаштувань профілю.

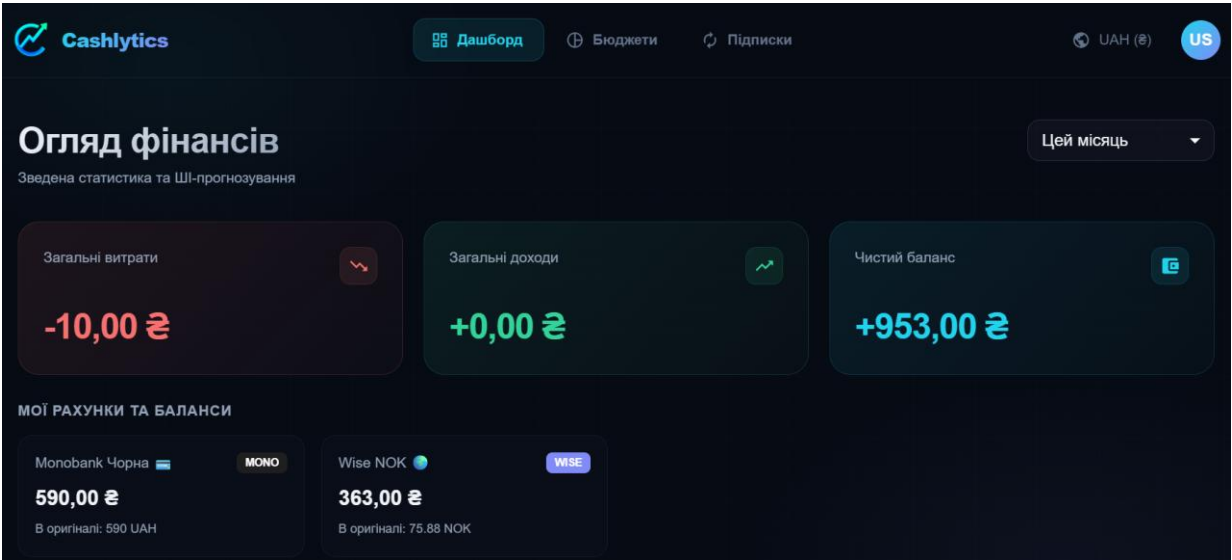


Рисунок 2.4 – Головна сторінка вебсистеми (зведена статистика та рахунки)

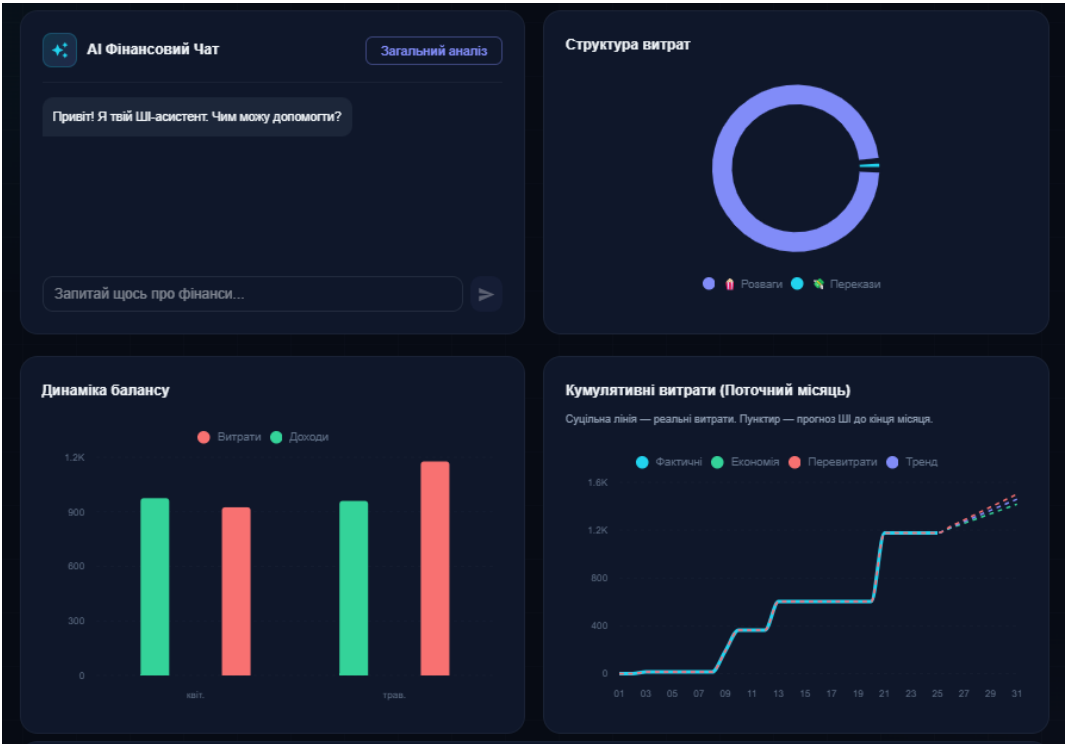


Рисунок 2.5 – Блок ШІ-чату та графіки

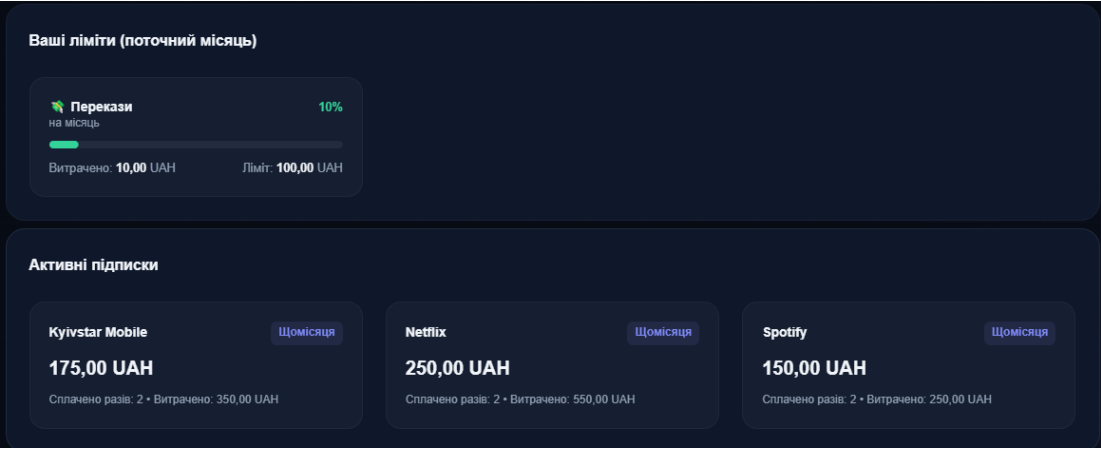


Рисунок 2.6 – Сторінка лімітів та активних підписок

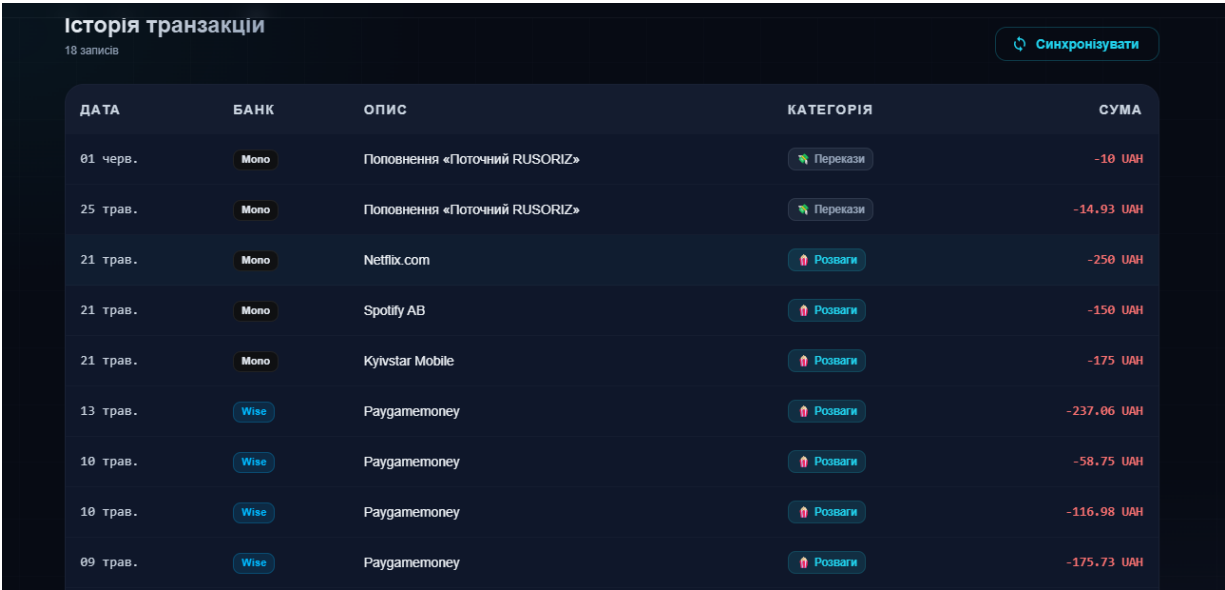


Рисунок 2.7 – Сторінка історії транзакцій з автоматичною категоризацією

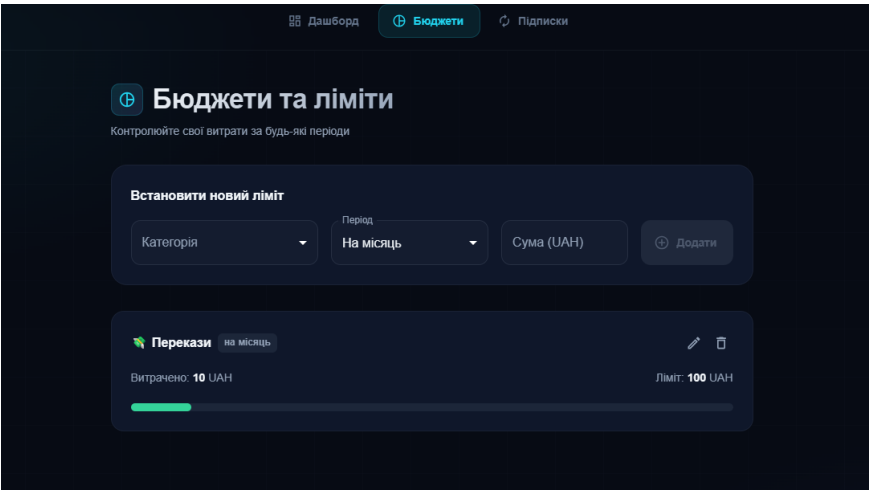


Рисунок 2.8 – Інтерфейс контролю бюджетних лімітів

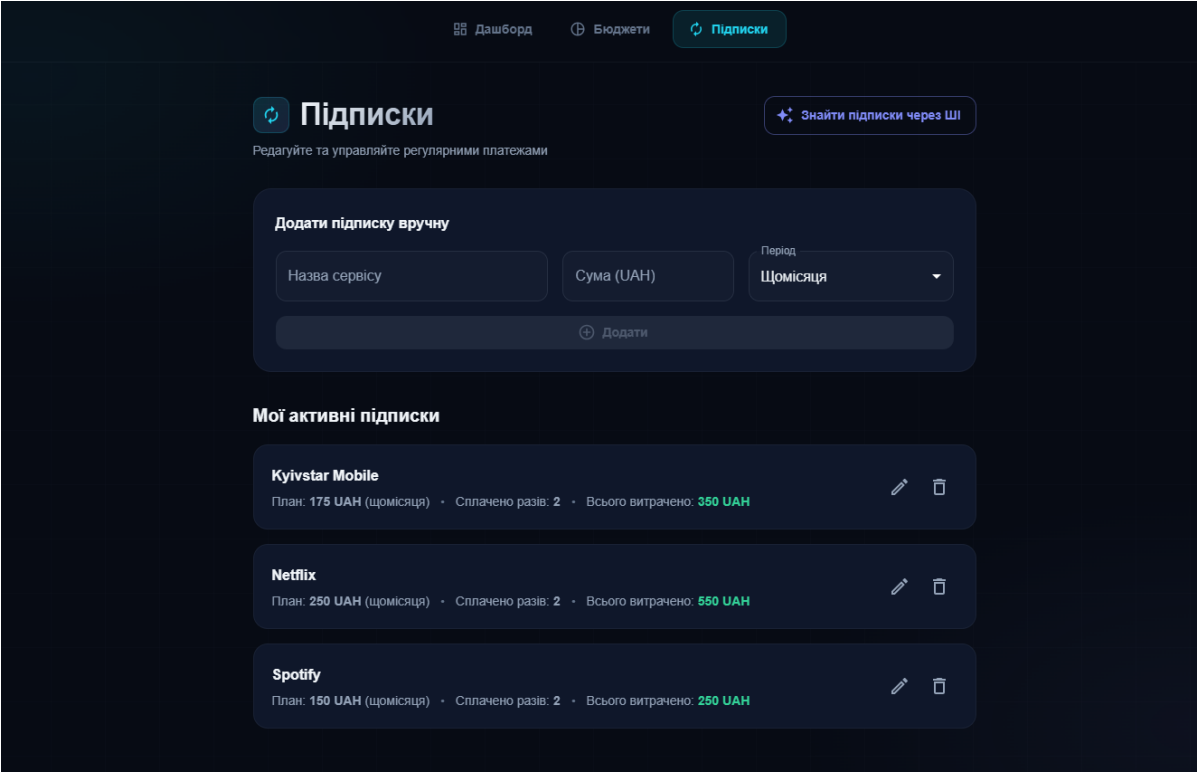


Рисунок 2.9 – Інтерфейс контролю регулярних підписок

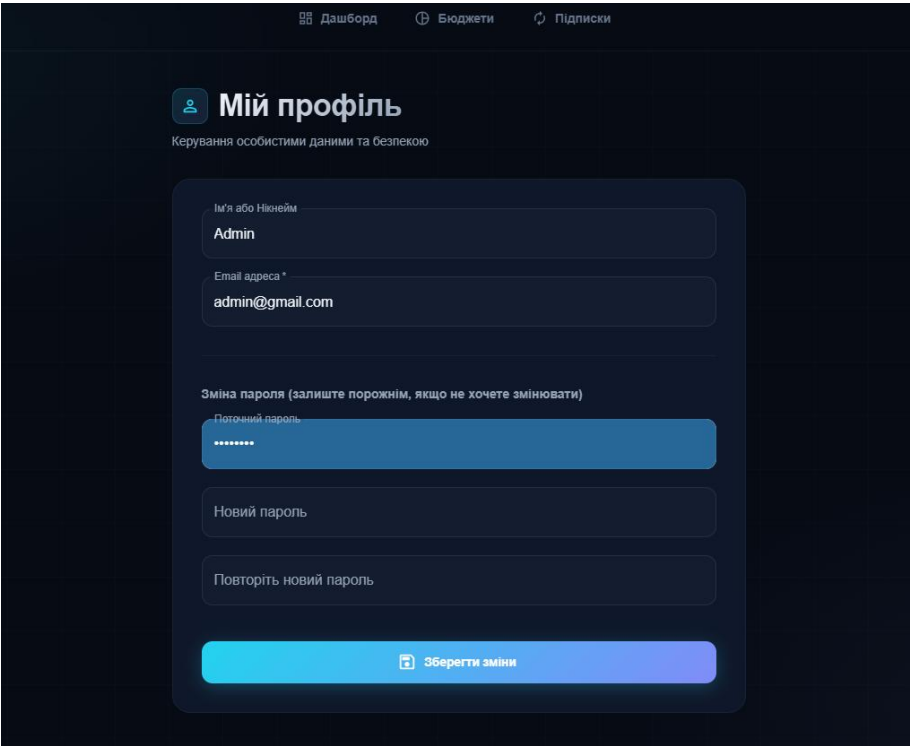


Рисунок 2.10 – Сторінка налаштувань профілю

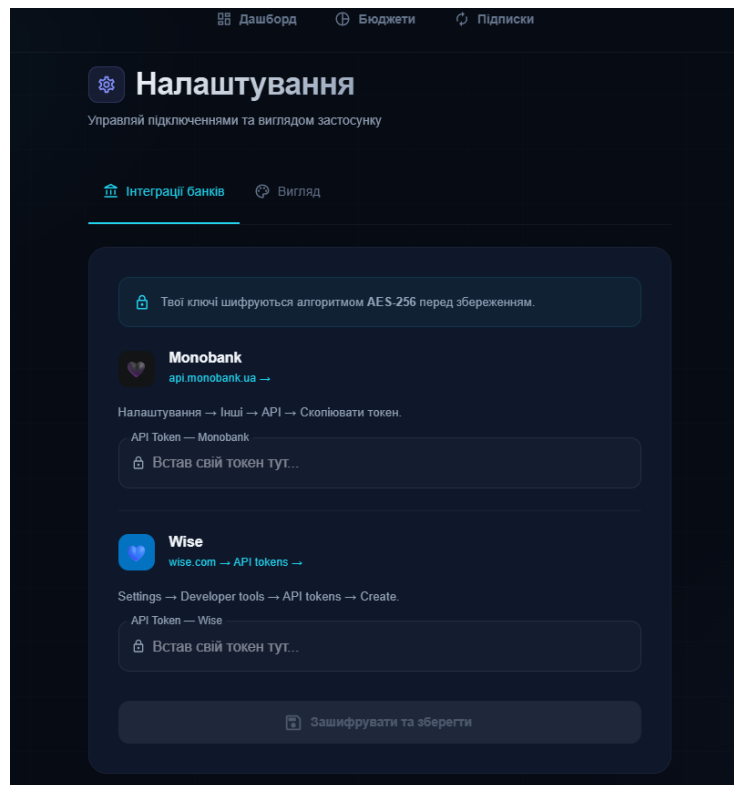


Рисунок 2.11 – Сторінка ‘Налаштування’ управління API-токенами та темами

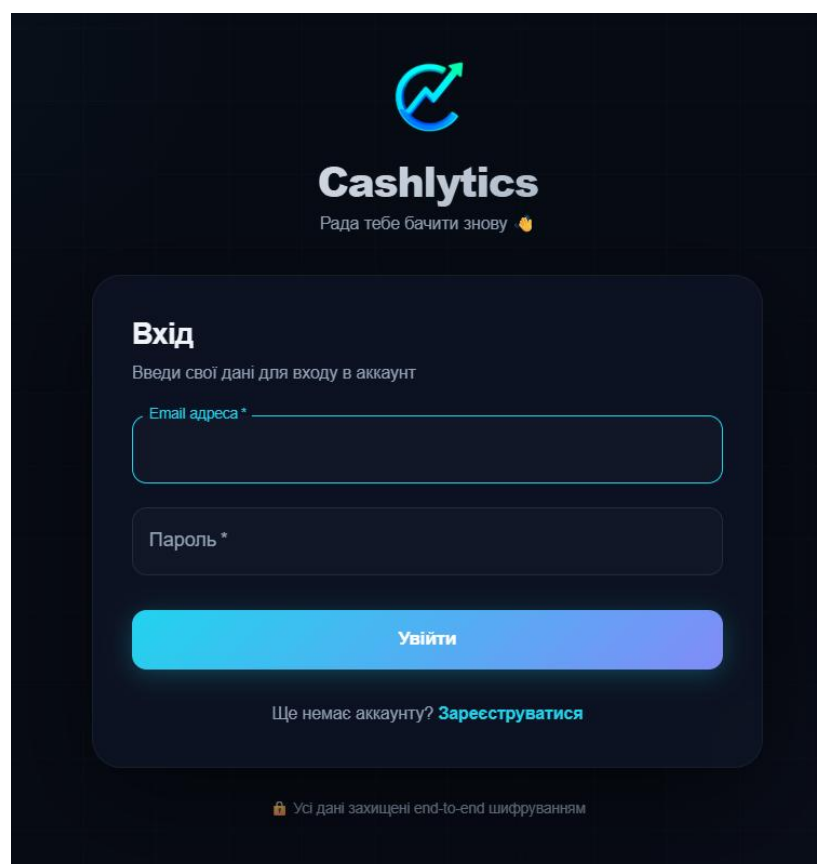


Рисунок 2.12 – Сторінка реєстрації чи входу до акаунту.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		34

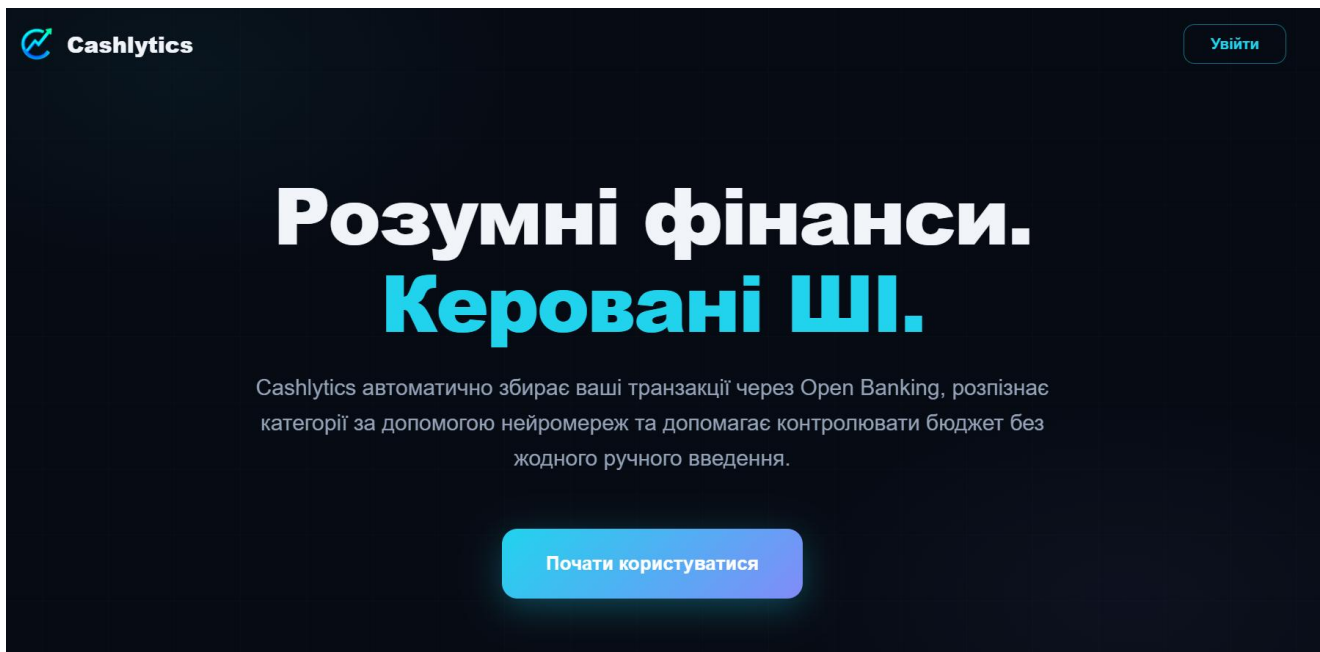


Рисунок 2.13 – Головна посадкова сторінка (Landing Page) та Cookie-банер.

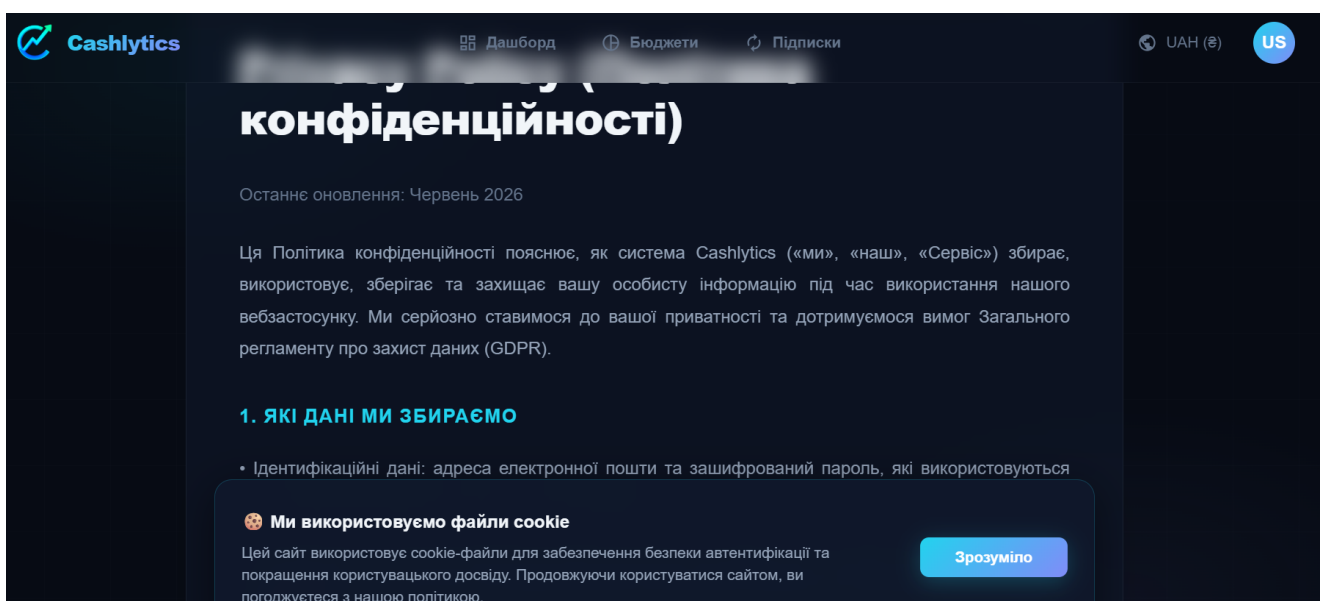


Рисунок 2.14 – Інтерфейс юридичної документації (Політика конфіденційності).

2.3 Алгоритм роботи програмного продукту

Алгоритм роботи вебсистеми «Cashlytics» описує не лише послідовність дій користувача та взаємодію компонентів, але й базову розрахункову логіку агрегації фінансових даних та їх подальший інтелектуальний аналіз. Програмний продукт працює за принципом асинхронної клієнт-серверної архітектури: користувач

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		35

ініціює дію у вебінтерфейсі, клієнтська частина надсилає HTTP-запит до серверної частини (Node.js), сервер виконує необхідну бізнес-логіку (математичні розрахунки, звернення до банку або нейромережі), зберігає результати в базу даних PostgreSQL та повертає оновлений масив даних на клієнт.

Первинний алгоритм взаємодії починається ще до моменту авторизації. При зверненні користувача до вебзастосунку вхідний трафік спочатку проходить через зворотний проксі-сервер (Nginx), який забезпечує безпеку з'єднання (HTTPS) та передає реальну IP-адресу клієнта на бекенд. Модуль Rate Limiter перевіряє частоту запитів для блокування можливих DDoS-атак. Після успішного з'єднання клієнтська частина виконує перевірку локального сховища браузера (localStorage) на наявність згоди щодо використання файлів cookie. Якщо запис відсутній, алгоритм ініціює виклик динамічного Cookie-банера поверх презентаційної посадкової сторінки (Landing Page). Лише після ознайомлення з продуктом алгоритм переводить користувача на етап автентифікації.

Фундаментом аналітичного дашборду є алгоритми обробки масивів транзакцій, які виконуються на рівні бази даних та серверного ядра перед їх відображенням:

- Нормалізація валют: Оскільки система підтримує мультивалютні рахунки (Monobank, Wise), перед будь-яким розрахунком усі суми приводяться до єдиної базової валюти користувача (наприклад, UAH) за актуальним крос-курсом.

- Агрегація за періодами та категоріями: Для побудови графіків сервер групує транзакції (використовуючи SQL-запити з оператором GROUP BY). Розрахунок виконується в кількох розрізах: по днях (для лінійного графіка динаміки витрат), по місяцях (для загальної статистики) та по категоріях товарів/послуг (для кругової діаграми розподілу бюджету). Крім того, система агрегує дані за точками оплати (мерчантами) для виявлення найчастіших місць витрат.

- Розрахунок виконання бюджетів: Логіка контролю лімітів базується на обчисленні відношення фактично витраченої суми у певній категорії до встановленого користувачем ліміту за формулою: Відсоток = (Витрачено / Ліміт) ×

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		36

100%. Система динамічно розраховує цей показник залежно від обраного періоду (день, тиждень, місяць).

– Предиктивна аналітика (прогнозування): Сервер розраховує середньоденну швидкість витрат (Загальні_витрати / Поточний_день_місяця) і множить її на загальну кількість днів у періоді. Це дозволяє сформувати базовий прогноз витрат до кінця місяця.

Найскладнішим архітектурним процесом є алгоритм отримання нових даних. Коли користувач натискає кнопку «Синхронізувати», система виконує такі кроки:

– Сервер розшифровує збережені API-токени користувача та формує авторизовані запити до банківських шлюзів.

– Система завантажує поточні баланси та масив нових транзакцій (виписку).

– Сервер ітерує отриманий масив і перевіряє кожну операцію на наявність у базі даних (за унікальним ідентифікатором банку id), щоб уникнути дублювання.

– Якщо транзакція є новою, запускається алгоритм III-категоризації. Текстовий опис (наприклад, «Сільпо», «Uber», «Netflix») відправляється до великої мовної моделі (Google Gemini API). Штучний інтелект аналізує семантику опису, повертає відповідну категорію та емодзі, після чого сервер зберігає транзакцію у базі даних через Prisma ORM.

Окрім базової категоризації, система передбачає глибокий алгоритм обробки природної мови (NLP) під час взаємодії користувача з III-чатом. Оскільки великі мовні моделі (LLM) схильні до математичних помилок, алгоритм побудовано таким чином, що нейромережа не займається «сирим» підрахунком цифр.

Замість цього, клієнтська частина формує системний промпт (контекст), у який "зашиває" вже попередньо обчислену сервером розрахункову логіку: поточні баланси, агреговані суми за категоріями, відсоток виконання бюджетів та ідентифіковані регулярні підписки.

Отримавши цей математично точний контекст, III-алгоритм аналізує його на вищому логічному рівні:

- Пошук аномалій: ШІ зіставляє витрати по днях і вказує користувачу на нетипові сплески витрат (наприклад, перевитрата на вихідних).
- Аналіз швидкості (Burn Rate): Нейромережа аналізує відсоток виконання бюджету та попереджає користувача: "Ви витратили 80% бюджету на розваги, хоча пройшла лише половина місяця".
- Оптимізація підписок: Модель аналізує повторювані платежі одним і тим самим мерчантам і генерує персоналізовані текстові поради щодо їх можливої оптимізації.

Такий підхід дозволяє поєднати абсолютну точність реляційної бази даних із семантичною гнучкістю штучного інтелекту, перетворюючи систему на повноцінного фінансового радника. Загальний узагальнений алгоритм роботи системи та процесу обробки фінансових даних представлено у вигляді UML-діаграми діяльності (Activity Diagram) на рисунку 2.15.

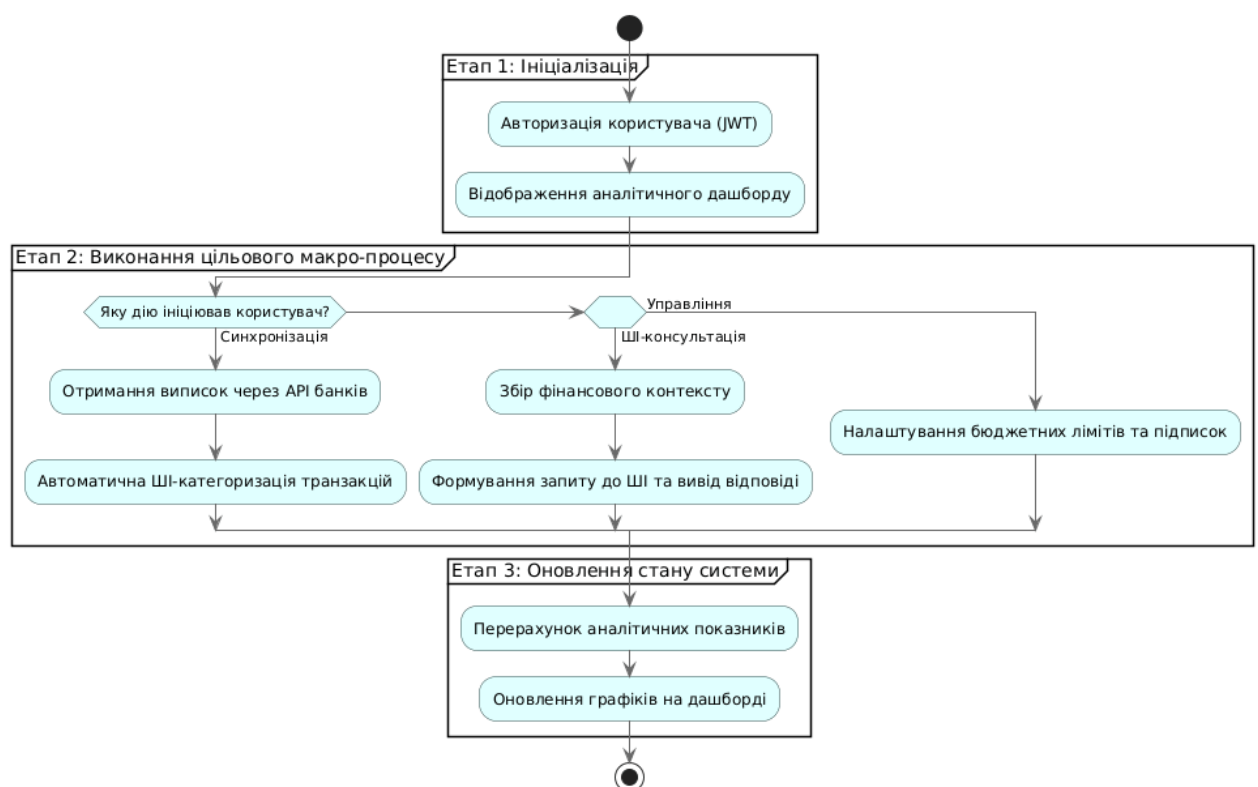


Рисунок 2.15 – Узагальнений алгоритм роботи та синхронізації вебсистеми

При відправці текстового повідомлення клієнтська частина формує пакет даних, що містить запит користувача, його останні 50 транзакцій, поточні ліміти

бюджетів та історію попередніх повідомлень чату. Цей контекст обробляється нейромережею, після чого згенерована текстова відповідь повертається в інтерфейс, дозволяючи системі функціонувати як повноцінний фінансовий радник.

Висновки до розділу 2

У другому розділі було проведено комплексне моделювання та проєктування вебсистеми «Розробка інтелектуальної інформаційної системи управління особистими фінансами на базі технологій Open Banking». На основі аналізу предметної області сформовано детальну специфікацію вимог до програмного продукту та визначено потенційні ризики інтеграції зі сторонніми сервісами. Логіку взаємодії користувача із системою візуалізовано за допомогою UML-діаграми прецедентів.

Обґрунтовано вибір багаторівневої клієнт-серверної архітектури, посиленої високопродуктивним проксі-сервером Nginx та модулем обмеження частоти запитів (Rate Limiting) для захисту від кібератак. Спроектовано фізичну модель реляційної бази даних, структура якої складається з шести взаємопов'язаних сутностей (User, Account, Transaction, Category, Budget, Subscription), безпечне управління якими реалізовано через систему Prisma ORM.

Архітектуру модулів та зв'язки даних наочно проілюстровано діаграмою компонентів та ER-діаграмою. Також представлено розширені макети основних екранів (включаючи презентаційну посадкову сторінку та інтерактивні модулі юридичних політик і Cookie-згоди), що підтверджують адаптивність та сучасність інтерфейсу.

Ключовим результатом розділу є розроблений алгоритм роботи застосунку, представлений у вигляді діаграми діяльності. Алгоритм чітко описує логіку асинхронної взаємодії між клієнтом, сервером, банківськими API (Monobank, Wise) та великою мовною моделлю (Google Gemini API) для автоматичної категоризації транзакцій. Успішне виконання етапу проєктування створило надійне підґрунтя для безпосередньої програмної реалізації системи у наступному розділі.

3 РОЗРОБКА ПРОГРАМНОГО ПРОДУКТУ

3.1 Обґрунтування вибору інструментальних засобів розроблення

Процес створення вебсистеми «Cashlytics» вимагав ретельного підбору технологічного стеку. Оскільки головним завданням проєкту є безпечна агрегація банківських даних та їх інтелектуальна обробка в режимі реального часу, пріоритет надавався інструментам, що гарантують високу продуктивність, строгу типізацію та стійкість до високих навантажень.

У якості головного інструменту для написання програмного коду було обрано інтегроване середовище розробки IntelliJ IDEA 2025.3.5 (Ultimate Edition). Цей вибір зумовлений потужним функціоналом версії Ultimate, яка оптимізована для Full-stack розробки. Середовище надає глибоку підтримку сучасних вебфреймворків, вбудовані засоби для управління реляційними базами даних (Database Tools) та просунуті алгоритми статичного аналізу коду. Інтеграція з системами контролю версій (Git) та можливість запуску локальних серверів безпосередньо у вбудованому терміналі суттєво оптимізували процес тестування компонентів.

Для забезпечення максимальної надійності обчислень, як клієнтська, так і серверна частини системи були реалізовані мовою TypeScript. Використання цієї надмножини JavaScript дозволило впровадити строгу статичну типізацію. Це критично важливо для фінансового програмного забезпечення, оскільки типізація даних на етапі компіляції унеможливорює виникнення прихованих помилок при конвертації валют або підрахунку кумулятивних балансів. Крім того, TypeScript забезпечує чітке декларування структур даних (інтерфейсів), які очікуються від сторонніх банківських API. Для побудови візуального каркаса та стилізації елементів застосовувалися стандарти HTML5 та CSS3.

Візуальний інтерфейс системи побудовано за допомогою фреймворку React. Його концепція односпрямованого потоку даних та використання віртуального DOM (Virtual DOM) забезпечують миттєву реакцію інтерфейсу на дії користувача – наприклад, при перемиканні цільової валюти графіки оновлюються без

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		40

перезавантаження вебсторінки. З метою оптимізації швидкості завантаження клієнтської частини та забезпечення ідеальної чіткості графіки на дисплеях високої роздільної здатності (Retina) було прийнято рішення відмовитися від важких растрових зображень. Головні візуальні елементи, включаючи логотип, розроблено у форматі чистої векторної графіки (чистий SVG-код) із застосуванням CSS-фільтрів для генерації ефекту неонових світіння.

Оформлення системи базується на бібліотеці Material-UI (MUI), яка містить готові, адаптивні компоненти, що відповідають сучасним стандартам UX-дизайну. Для реалізації аналітичного функціоналу інтегровано бібліотеку Recharts, за допомогою якої побудовано інтерактивні лінійні графіки предиктивної аналітики та кругові діаграми структури витрат.

Програмна логіка сервера реалізована у середовищі виконання Node.js з використанням вебфреймворку Express.js. Асинхронна архітектура Node.js дозволяє ефективно обробляти безліч паралельних HTTP-запитів. Це є життєво необхідним для проєкту, оскільки під час синхронізації сервер одночасно очікує відповіді від банківських шлюзів та хмарних моделей штучного інтелекту. Крім того, для забезпечення кіберзахисту до серверного ядра було інтегровано модуль обмеження частоти запитів (express-rate-limit), який надійно запобігає DDoS-атакам і спробам автоматизованого підбору паролів (brute-force).

Для надійного зберігання інформації користувачів (облікові дані, історія транзакцій, встановлені ліміти) використано об'єктно-реляційну систему PostgreSQL. Важливим архітектурним кроком стало розгортання бази даних на хмарній платформі Supabase (BaaS). Це рішення забезпечує автоматичне масштабування, регулярне резервне копіювання фінансових даних та високий рівень безпеки без потреби адмініструвати локальні сервери.

Комунікація між Node.js сервером та базою даних PostgreSQL відбувається через інструмент Prisma ORM. Ця технологія автоматично генерує клієнт доступу на основі схеми даних (schema.prisma), що унеможливорює SQL-ін'єкції та дозволяє зручно писати складні запити для вибірки та агрегації статистики.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		41

Для забезпечення надійного розгортання програмного продукту у середовищі реальної експлуатації (Production) на хмарному сервері та прив'язки до власного доменного імені було використано технологію контейнеризації Docker. Маршрутизація трафіку та термінація SSL-сертифікатів (забезпечення захищеного HTTPS-з'єднання) реалізована за допомогою високопродуктивного вебсервера Nginx, який виконує функцію зворотного проксі (Reverse Proxy).

Додаткові інструменти та інтеграції

Функціонал штучного інтелекту забезпечується шляхом інтеграції великої мовної моделі через Google Gemini API. Цей модуль відповідає за розпізнавання сирих банківських виписок, їх точну категоризацію та генерацію відповідей у вбудованому фінансовому чаті.

Для здійснення мережових викликів до API Monobank та Wise використано клієнт Axios. Версіонування та зберігання вихідного коду здійснювалося за допомогою сервісу GitHub. Система розроблялася та тестувалася в операційній системі Windows 11, а для форматування і підготовки цього текстового звіту відповідно до академічних стандартів застосовано Microsoft Word.

3.2 Опис компонентів програмного продукту

Інтелектуальна фінансова вебсистема «Cashlytics» побудована за клієнт-серверною архітектурою і складається з окремих взаємопов'язаних компонентів, кожен із яких виконує чітко визначені функції обробки та захисту інформації. Взаємодія між цими структурними одиницями забезпечує коректну роботу всього програмного продукту в суворій відповідності до узагальненого алгоритму роботи, детально описаного та візуалізованого у пункті 2.3.

Клієнтська частина (Frontend), що розроблена на базі бібліотеки React та мови TypeScript, виступає первинною ланкою взаємодії користувача із системою. Вона відповідає за стадії запуску застосунку, автентифікацію та графічне відображення фінансового стану.

Основними компонентами клієнтської частини є:

- Компонент головного аналітичного вікна (Dashboard View): відповідає

за етап відображення інтерактивного дашборду. Відповідно до алгоритму роботи системи (див. пункт 2.3), цей компонент одразу після успішної авторизації ініціює асинхронне завантаження фінансових масивів та керує верхніми картками метрик (доходи, витрати, чистий баланс), забезпечуючи користувача зведеною інформацією.

– Презентаційний модуль та управління згодою (Landing Page & Cookie Consent): зустрічає нових неавторизованих користувачів, візуалізує переваги системи за допомогою векторної графіки та забезпечує легітимний збір згоди на обробку файлів cookie відповідно до вимог GDPR. Містить посилання на юридичні компоненти системи (Privacy Policy).

– Модуль візуалізації даних (Charts Component): тісно взаємодіє з компонентом дашборду за допомогою бібліотеки Recharts. Він відповідає за побудову динамічних осей, кругових діаграм структури витрат та лінійних графіків прогнозування. Згідно з алгоритмом (див. пункт 2.3), при зміні глобальних параметрів (валюти або часового проміжку), цей модуль миттєво перемальовує графіки на основі оновленого стану пам'яті клієнта.

– Реєстр історії операцій (Transaction Ledger Module): керує виведенням таблиці транзакцій. Цей компонент відображає списки платежів, де кожній позиції присвоєно автоматично підібрану категорію та емодзі. У загальній логіці системи цей модуль виконує фінальну стадію алгоритму синхронізації – виведення оновленої інформації на екран (див. пункт 2.3).

– Модуль управління лімітами та підписками (Budgets & Subscriptions Component): відповідає за відображення прогресу виконання встановлених фінансових обмежень та переліку регулярних витрат. Компонент отримує розраховані сервером відсотки та за допомогою графічних індикаторів (Progress Bars) сигналізує про стан бюджетів.

– Клієнт ШІ-чату (AI Chat Client): реалізує графічний інтерфейс вікна діалогу з нейромережею. Він фіксує текстове введення користувача і, відповідно до алгоритму обробки запитів ШІ-чату (див. пункт 2.3), відправляє повідомлення на сервер, очікуючи на генерацію предиктивної відповіді.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		43

Серверна частина (Backend), що функціонує в середовищі Node.js та Express.js, виступає центральним керуючим вузлом бізнес-логіки. Вона обробляє HTTP-запити від клієнта та координує роботу із зовнішніми сервісами й базою даних.

Основними компонентами серверної частини є:

– Контролер REST API маршрутизації (API Router): приймає вхідні виклики від клієнтської частини системи, здійснює перевірку JWT-токенів і перенаправляє потоки даних на відповідні робочі модулі, забезпечуючи зв'язок між фронтендом та бекендом.

– Модуль маршрутизації та мережевого захисту (Nginx Proxy & Rate Limiter): слугує точкою входу в систему. Забезпечує термінацію SSL-сертифікатів (HTTPS), коректно визначає IP-адресу клієнта (через заголовок X-Forwarded-For) та лімітує кількість запитів для блокування шкідливого трафіку чи DDoS-атак.

– Модуль авторизації та криптографічного захисту (Auth & Cryptography Module): виконує валідацію паролів (за алгоритмом argon2) та симетричне кодування банківських ключів доступу (AES-256), гарантуючи конфіденційність перед збереженням у хмару.

– Модуль банківської синхронізації (Bank API Sync Module): активується, коли користувач ініціює процес примусового оновлення даних. Відповідно до алгоритму синхронізації (див. пункт 2.3), цей компонент робить асинхронні запити до шлюзів Monobank та Wise, завантажує поточні баланси й масиви нових транзакцій та очищує їх від технічного сміття.

– Аналітичний вузол штучного інтелекту (Generative AI Bridge): інтегрований з моделлю Google Gemini. Згідно з циклом обробки транзакцій в алгоритмі роботи (див. пункт 2.3), цей модуль приймає текстовий опис кожної нової операції, семантично аналізує його, визначає категорію витрат і підбирає іконку. Також він формує контекстні пакети для роботи фінансового чат-бота.

– Прошарок обробки бізнес-логіки та Prisma ORM: здійснює математичні розрахунки (конвертацію валют, обчислення кумулятивних ліній, три сценарії III-прогнозування) та безпосередньо взаємодіє з базою даних PostgreSQL, записуючи

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		44

або зчитуючи оброблені пакети інформації.

Взаємодія всіх перерахованих компонентів виконується у суворій послідовності, визначеній загальним алгоритмом (див. пункт 2.3). Клієнтський інтерфейс надсилає запити до REST API, серверні контролери розподіляють завдання між модулями синхронізації, ШІ та аналітики, Prisma ORM фіксує фінансові зміни у хмарній базі даних Supabase, після чого агрегований результат повертається назад для миттєвої перебудови графічних елементів дашборду.

Для підтвердження інженерної складової розробленого програмного продукту нижче наведено ключові фрагменти вихідного коду (із середовища розробки), які відповідають за реалізацію найважливіших архітектурних модулів системи та забезпечення кібербезпеки.

Ключовою інноваційною особливістю системи «Cashlytics» є глибока інтеграція алгоритмів штучного інтелекту для аналізу фінансової поведінки. Для реалізації цього функціоналу використовується SDK великої мовної моделі Google Gemini. Програмний модуль бекенду формує спеціалізований промпт (контекст), передаючи нейромережі попередньо агреговані дані користувача –історію останніх транзакцій та встановлені ліміти бюджетів. Це дозволяє ШІ-асистенту генерувати максимально точні та персоналізовані фінансові поради, уникаючи галюцинацій. Програмну реалізацію логіки формування контексту та взаємодії з ШІ-моделлю продемонстровано на рисунку 3.1.

```

app.get( path: '/api/ai-advice', async (req: Request, res: Response): Promise<any> => { new "
  try {
    const userId :any = (req as any).userId;
    const dbTransactions :{ currency:string; date:Date; source:T... = await prisma.transaction.findMany({ where: { userId } });

    if (dbTransactions.length === 0) {
      return res.json({ advice: "У тебе ще немає транзакцій. Синхронізуй банк!" });
    }

    const expenses :{ currency:string; date:Date; source:T... = dbTransactions.filter((t: any) :boolean => t.amount < 0);
    const expenseSummary :string = expenses.map((t: any) :string => `${t.description} (${t.categoryName}): ${t.amount} ${t.currency}`).join( separator: ', ');

    const prompt :string = ` Ти – досвідчений фінансовий консультант і аналітик особистих фінансів. Ось зведення моїх витрат: ${expenseSummary} Проаналізуй ці
    Вимоги до відповіді:
    - Максимум 100 слів.
    - Пиши природно та зрозуміло, як персональну фінансову пораду для користувача застосунку Cashlytics.
    - Використовуй конкретні спостереження з моїх витрат.
    - Надай корисні рекомендації щодо оптимізації бюджету, зменшення зайвих витрат або покращення фінансових звичок.
    - Не згадуй, що ти штучний інтелект, модель або що це згенерований текст.
    - Не використовуй вступи на кшталт "Ось ваша порада", "На основі аналізу", "Рекомендація:".
    - Не використовуй Markdown, HTML, жирний текст, списки чи будь-яке інше форматування.
    - Поверни лише готовий текст поради без додаткових пояснень.
    `.trim();

    const response :GenerateContentResponse = await ai.models.generateContent({
      model: 'gemini-3.1-flash-lite',
      contents: prompt
    });

    res.json({ advice: response.text?.trim() || "ШІ не зміг згенерувати пораду 😊" });

  } catch (error) {
    res.status( code: 500).json({ error: "Не вдалося згенерувати пораду" });
  }
}

```

Рисунок 3.1 – Програмна реалізація генерації фінансового контексту для ШІ-моделі

Надійне функціонування інтелектуального аналітичного модуля неможливе без безперебійного постачання актуальних фінансових даних. Цей процес забезпечується через інтеграцію з API банків, зокрема Monobank. Серверна частина платформи робить авторизовані асинхронні HTTP-запити до банківського шлюзу за допомогою бібліотеки axios, отримує масиви нових транзакцій і виконує їх перевірку на унікальність. Важливим елементом розробленого алгоритму є використання затримок (Rate Limiting) для уникнення блокування запитів з боку серверів банку. Фрагмент коду, що відповідає за комунікацію з API Monobank та первинну обробку виписок, представлено на рисунку 3.2. Повна частина коду знаходиться в Додатку Б.

```

app.get( path: '/api/sync-monobank', async (req: Request, res: Response): Promise<any> => { new *
  try {
    const userId :string = String((req as any).userId);
    const user :{ name: string | null; email: string; id: ... } = await prisma.user.findUnique({ where: { id: userId } });

    if (!user || !user.monoToken) {
      return res.status( code: 400 ).json({ error: "Додайте токен Monobank у налаштуваннях" });
    }

    const decryptedToken :string = decrypt(user.monoToken);

    console.log( ...data: "🏠 Йдемо в Monobank за інформацією про рахунки..." );

    const clientInfoResponse :AxiosResponse<any, any, {}> = await axios.get( url: 'https://api.monobank.ua/personal/client-info', {
      headers: { 'X-Token': decryptedToken }
    });

    const monoAccounts :any = clientInfoResponse.data.accounts || [];

    for (const monoAcc of monoAccounts) {
      const realBalance :number = monoAcc.balance / 100;

      let cardName :string = "Monobank Картка";
      if (monoAcc.type === 'black') cardName = "Monobank Чорна 🖤";
      else if (monoAcc.type === 'white') cardName = "Monobank Біла ⚪";
      else if (monoAcc.type === 'platinum') cardName = "Monobank Platinum 💎";
      else if (monoAcc.type === 'fop') cardName = "Monobank ФОП 🏠";
      else if (monoAcc.type === 'eAid') cardName = "eПідтримка.ua";

      let currencyCode :string = "UAH";
      if (monoAcc.currencyCode === 840) currencyCode = "USD";
      else if (monoAcc.currencyCode === 978) currencyCode = "EUR";
      else if (monoAcc.currencyCode === 985) currencyCode = "PLN";
    }
  }
}

```

Рисунок 3.2 – Програмна реалізація модуля синхронізації з API Monobank

Оскільки взаємодія з банківськими API вимагає використання персональних ключів доступу (токенів) користувачів, критичним етапом розробки стало забезпечення їх кіберзахисту. Відповідно до концепції нульової довіри (Zero Trust), система повинна була унеможливити компрометацію фінансів клієнтів навіть у випадку отримання зловмисниками (або адміністраторами бази даних/співробітниками компанії) прямого доступу до таблиць БД. Для цього впроваджено алгоритм симетричного шифрування AES-256-CBC. Токени шифруються унікальним серверним ключем перед записом у базу даних і розшифровуються виключно в оперативній пам'яті ізолюваного Docker-контейнера під час виконання запиту до банку. Програмну реалізацію криптографічного алгоритму наведено на рисунку 3.3.

					227202.472 ПЗ	Арк.
						47
Змн.	Арк.	№ докум.	Підпис	Дата		

```

1  import crypto from 'crypto';
2
3  const ALGORITHM = 'aes-256-cbc';
4  const IV_LENGTH = 16;
5  const RAW_KEY :string = (process.env.ENCRYPTION_KEY || 'default_secret_key_for_development').trim();
6  const ENCRYPTION_KEY :NonSharedBuffer = crypto.createHash( algorithm: 'sha256').update(String(RAW_KEY)).digest();
7
8  export function encrypt(text: string): string { Show usages new *
9      if (!text) return text;
10     const iv :NonSharedBuffer = crypto.randomBytes(IV_LENGTH);
11     const cipher :Cipheriv = crypto.createCipheriv(ALGORITHM, ENCRYPTION_KEY, iv);
12
13     let encrypted :NonSharedBuffer = cipher.update(text);
14     encrypted = Buffer.concat([encrypted, cipher.final()]);
15     return iv.toString( encoding: 'hex') + ':' + encrypted.toString( encoding: 'hex');
16 }
17 export function decrypt(text: string): string { Show usages new *
18     if (!text || !text.includes( searchString: ':')) {
19         return text;
20     }
21
22     try {
23         const textParts :string[] = text.split( separator: ':');
24         const iv :Buffer<ArrayBuffer> = Buffer.from(textParts.shift()!, encoding: 'hex');
25         const encryptedText :Buffer<ArrayBuffer> = Buffer.from(textParts.join( separator: ':', encoding: 'hex');
26         const decipher :Decipheriv = crypto.createDecipheriv(ALGORITHM, ENCRYPTION_KEY, iv);
27
28         let decrypted :NonSharedBuffer = decipher.update(encryptedText);
29         decrypted = Buffer.concat([decrypted, decipher.final()]);
30
31         return decrypted.toString();
32     } catch (error) {
33         console.error( ...data: "❌ Помилка розшифрування токена. Можливо, змінився ENCRYPTION_KEY в .env.");
34         return "";
35     }
36 }

```

Рисунок 3.3 – Програмна реалізація алгоритму криптографічного шифрування AES-256

Для забезпечення персистентного зберігання даних у хмарній СУБД PostgreSQL на платформі Supabase було створено реляційну структуру з шести таблиць. Відповідно до вимог щодо проєктування баз даних, специфікацію фізичної структури, типів полів та наявних обмежень. Таблиці представлено нижче на рисунках 3.4 – 3.9 у вигляді моделей із середовища розробки.

NAME	TYPE	CONSTRAINTS		
T id	text	 PRIMARY  NON-NULLABLE	Edit	
T email	text	 UNIQUE  NON-NULLABLE	Edit	
T name	text	 NULLABLE	Edit	
T passwordHash	text	 NON-NULLABLE	Edit	
T monoToken	text	 NULLABLE	Edit	
T wiseToken	text	 NULLABLE	Edit	
 createdAt	timestamp	 NON-NULLABLE	Edit	
 updatedAt	timestamp	 NON-NULLABLE	Edit	

Рисунок 3.4 – Структура таблиці User

NAME	TYPE	CONSTRAINTS		
T id	text	 PRIMARY  NON-NULLABLE	Edit	
T name	text	 NON-NULLABLE	Edit	
# balance	float8	 NON-NULLABLE	Edit	
T currency	text	 NON-NULLABLE	Edit	
T source	text	 NON-NULLABLE	Edit	
T userId	text	 FOREIGN KEY  NON-NULLABLE	Edit	
 updatedAt	timestamp	 NON-NULLABLE	Edit	

Рисунок 3.5 – Структура таблиці Account

	NAME	TYPE	CONSTRAINTS
T	id	text	 PRIMARY  NON-NULLABLE
	date	timestamp	 NON-NULLABLE
T	description	text	 NON-NULLABLE
#	amount	float8	 NON-NULLABLE
T	currency	text	 NON-NULLABLE
	source	TransactionSource	 NON-NULLABLE
T	categoryId	text	 FOREIGN KEY  NULLABLE
T	categoryName	text	 NULLABLE
T	userId	text	 FOREIGN KEY  NON-NULLABLE
	createdAt	timestamp	 NON-NULLABLE
T	subscriptionId	text	 FOREIGN KEY  NULLABLE
11 columns			

Рисунок 3.6 – Структура таблиці Transaction

	NAME	TYPE	CONSTRAINTS	
T	id	text	 PRIMARY  NON-NULLABLE	Edit ⋮
T	name	text	 NON-NULLABLE	Edit ⋮
T	icon	text	 NULLABLE	Edit ⋮
T	color	text	 NULLABLE	Edit ⋮
T	userId	text	 FOREIGN KEY  NON-NULLABLE	Edit ⋮

Рисунок 3.7 – Структура таблиці Category

					227202.472 ПЗ	Арк.
						50
Змн.	Арк.	№ докум.	Підпис	Дата		

	NAME	TYPE	CONSTRAINTS		
T	id	text	 PRIMARY  NON-NULLABLE	Edit	⋮
T	categoryName	text	 NON-NULLABLE	Edit	⋮
#	amount	float8	 NON-NULLABLE	Edit	⋮
T	currency	text	 NON-NULLABLE	Edit	⋮
T	userId	text	 FOREIGN KEY  NON-NULLABLE	Edit	⋮
T	period	text	 NON-NULLABLE	Edit	⋮
📅	endDate	timestamp	 NULLABLE	Edit	⋮
📅	startDate	timestamp	 NULLABLE	Edit	⋮

Рисунок 3.8 – Структура таблиці Budget

	NAME	TYPE	CONSTRAINTS		
T	id	text	 PRIMARY  NON-NULLABLE	Edit	⋮
T	name	text	 NON-NULLABLE	Edit	⋮
#	amount	float8	 NON-NULLABLE	Edit	⋮
T	currency	text	 NON-NULLABLE	Edit	⋮
T	billingCycle	text	 NON-NULLABLE	Edit	⋮
T	userId	text	 FOREIGN KEY  NON-NULLABLE	Edit	⋮

Рисунок 3.9 – Структура таблиці Subscription

3.3 Опис роботи програмного продукту

Програмний продукт «Розробка інтелектуальної інформаційної системи управління особистими фінансами на базі технологій Open Banking» за своєю

архітектурною формою є кросплатформним вебзастосунком, призначеним для автоматизованого збору, інтелектуальної категоризації та предиктивного аналізу особистих фінансів. Система функціонує безпосередньо у веббраузері та не потребує встановлення додаткового спеціалізованого програмного забезпечення на персональний комп'ютер користувача.

Для повноцінної роботи із системою кінцевому користувачу необхідно мати стабільний доступ до мережі Інтернет та сучасний веббраузер, наприклад Google Chrome, Microsoft Edge, Safari або Mozilla Firefox.

Головною перевагою розробленої системи є її повна готовність до реальної експлуатації. Кінцевому користувачу не потрібно встановлювати середовище Docker, завантажувати вихідний код чи налаштовувати локальні сервери. Застосунок розгорнуто на хмарному сервері з прив'язкою до власного доменного імені.

Для повноцінного використання фінансового трекера достатньо відкрити будь-який сучасний веббраузер (з ПК чи смартфона) та перейти за офіційною адресою проєкту: <https://cashlytics.site>.

Безпека передачі конфіденційних фінансових даних гарантується використанням зворотного проксі-сервера Nginx та протоколу шифрування TLS 1.3 (наявність дійсного SSL-сертифіката). Водночас, для технічних спеціалістів збережено можливість локального розгортання копії системи для тестування за допомогою єдиної директиви `docker compose up -d`.

З метою забезпечення повноцінного адміністрування та експлуатації системи, опис роботи продукту розподілено на керівництво для кінцевого користувача та інструкцію для технічного адміністратора платформи.

Після переходу за веб-адресою проєкту новий (неавторизований) користувач потрапляє на сучасну посадкову сторінку (Landing Page). Цей презентаційний модуль розроблено з використанням маркетингових принципів: він містить головний екран із закликом до дії, опис ключових переваг системи (ШІ-аналітика, безпека AES-256) та інтерактивні картки.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		52

Під час першого візиту в нижній частині екрана з'являється динамічний Cookie-банер. Це є обов'язковим технічним рішенням для дотримання міжнародних норм Загального регламенту захисту даних (GDPR). Система запитує згоду користувача на обробку сесійних файлів cookie, після чого зберігає цей вибір у локальній пам'яті браузера (localStorage), щоб не заважати при наступних візитах. Також у підвалі сайту (Footer) розміщено посилання на статичні юридичні сторінки: Політику конфіденційності та Умови використання.

Після ознайомлення з платформою користувач натискає кнопку «Почати користуватися» та перенаправляється на сторінку автентифікації.

Зовнішній вигляд презентаційної посадкової сторінки з візуалізацією переваг системи, наведено на рисунку 3.10.

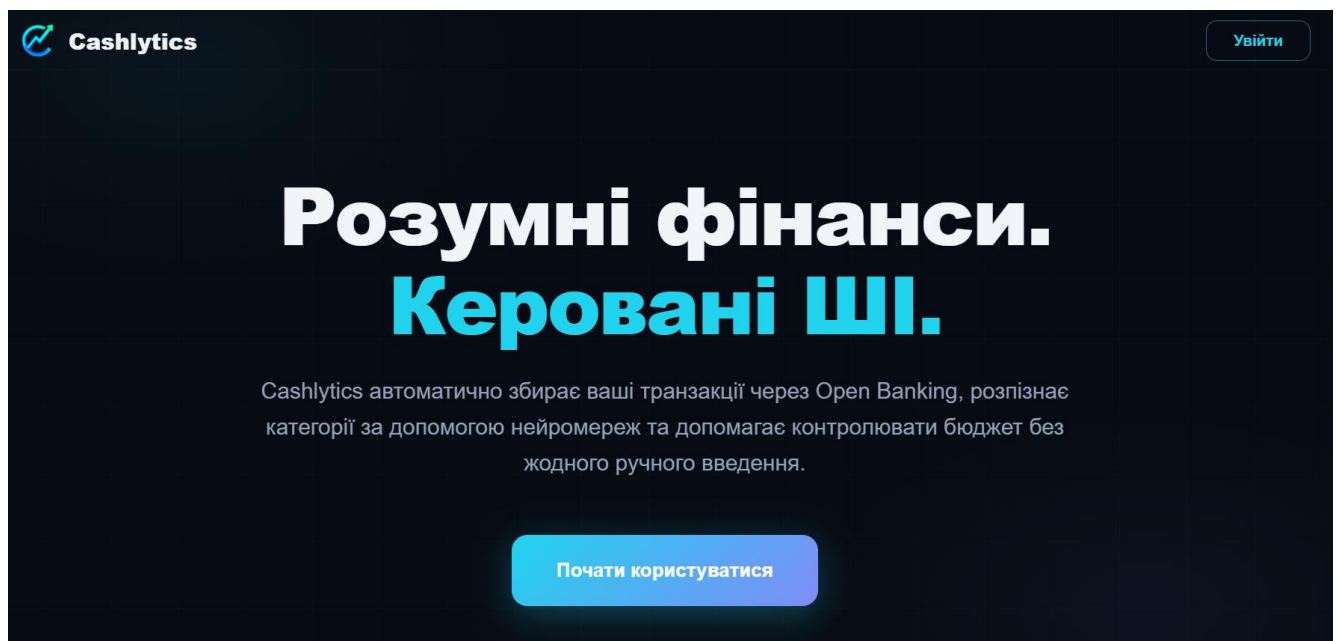


Рисунок 3.10 – Графічний інтерфейс Landing Page

Після ознайомлення з платформою користувач натискає кнопку «Почати користуватися» та перенаправляється на сторінку автентифікації. Графічний інтерфейс форми реєстрації та входу до системи наведено на рисунку 3.11.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		53

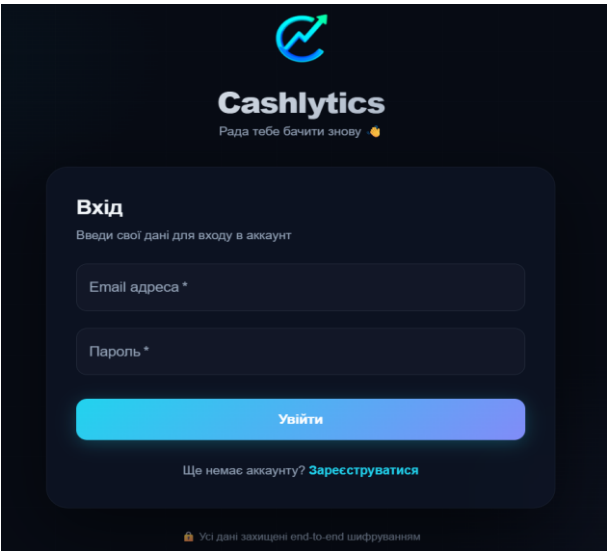


Рисунок 3.11 – Графічний інтерфейс форми автентифікації користувача

Оскільки головною ідеєю системи є автоматизація збору даних, після першого входу користувачеві необхідно перейти до розділу «Налаштування» (Settings). У цьому вікні розміщено форми для введення персональних криптографічно захищених API-токенів фінансових установ (Monobank та Wise). Користувач вводить ключ доступу та натискає кнопку «Зберегти». Модуль безпеки миттєво кодує ці дані, а Prisma ORM фіксує зміни у базі даних Supabase. Процес налаштування інтеграції з банківськими сервісами зображено на рисунку 3.12.

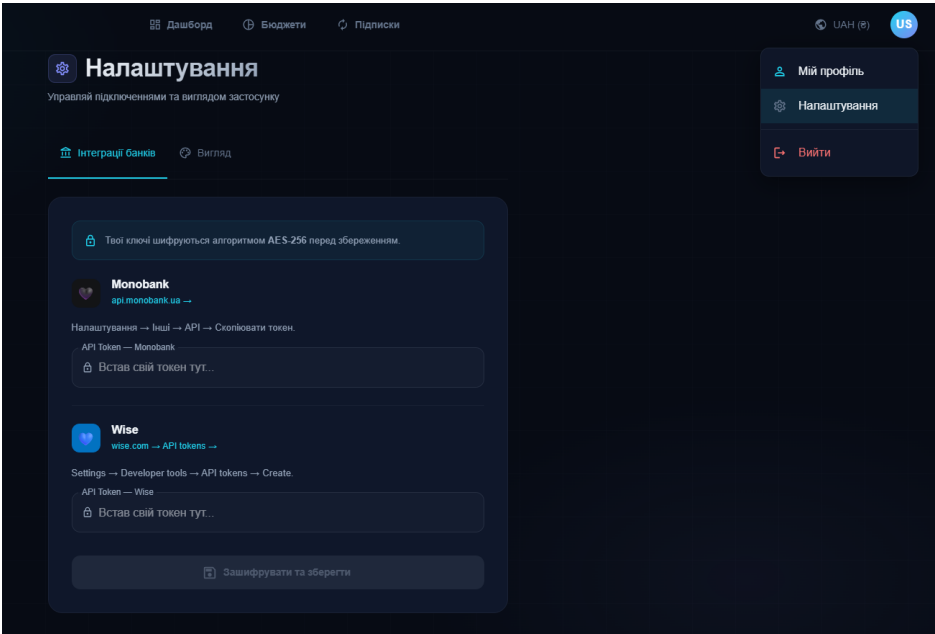


Рисунок 3.12 – Вікно конфігурації персональних банківських токенів

Після налаштування інтеграцій користувач повертається на головну панель (Dashboard). У верхній частині екрана відображаються блоки загального балансу. Натискання на інтерактивну кнопку «Синхронізувати» активує складний асинхронний ланцюг, описаний у пункті 2.3: сервер завантажує нові виписки, передає призначення платежів до Google Gemini API, визначає категорію, підбирає емодзі та оновлює базу даних. Результат роботи програми наочно виводиться на графіках Recharts та у таблиці транзакцій у режимі реального часу

Користувач має змогу:

- змінювати цільову валюту застосунку через випадаючий список (всі суми на графіках миттєво перераховуються за актуальним курсом);
- фільтрувати статистику витрат за часовими інтервалами (тиждень, місяць, поточний рік);
- наводити курсор на сектори кругової діаграми для детального перегляду обсягу витрат у конкретній категорії.

Загальний вигляд головного екрана застосунку з результатами аналітики представлено на рисунку 3.13.

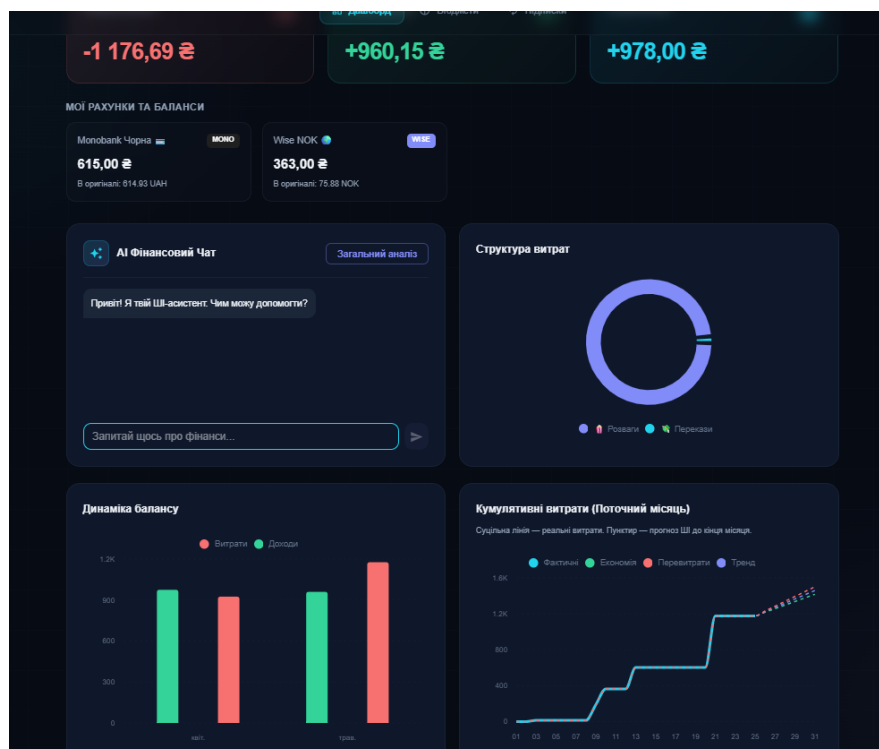


Рисунок 3.13 – Візуалізація результатів роботи системи на головному дашборді

					227202.472 ПЗ	Арк.
						55
Змн.	Арк.	№ докум.	Підпис	Дата		

У нижній частині інтерфейсу або у спеціалізованих вкладках користувачеві доступні модулі контролю лімітів та регулярних витрат. При наближенні до встановленого місячного ліміту по певній категорії, індикатор виконання (Progress Bar) змінює колір із зеленого на червоний, сигналізуючи про перевищення. Блок підписок виводить виявлені ІІІ циклічні платежі (наприклад, сервіси контенту).

Для отримання персоналізованої консультації користувач використовує вікно чату. У текстове поле вводиться запит природною мовою (наприклад, «Скільки грошей я витратив на розваги цього тижня та як мені оптимізувати свій бюджет?»). ІІІ-асистент аналізує контекст завантажених транзакцій та виводить розгорнуту текстову відповідь із рекомендаціями безпосередньо у вікно діалогу. Інтерфейс модуля інтелектуального спілкування наведено на рисунку 3.14.

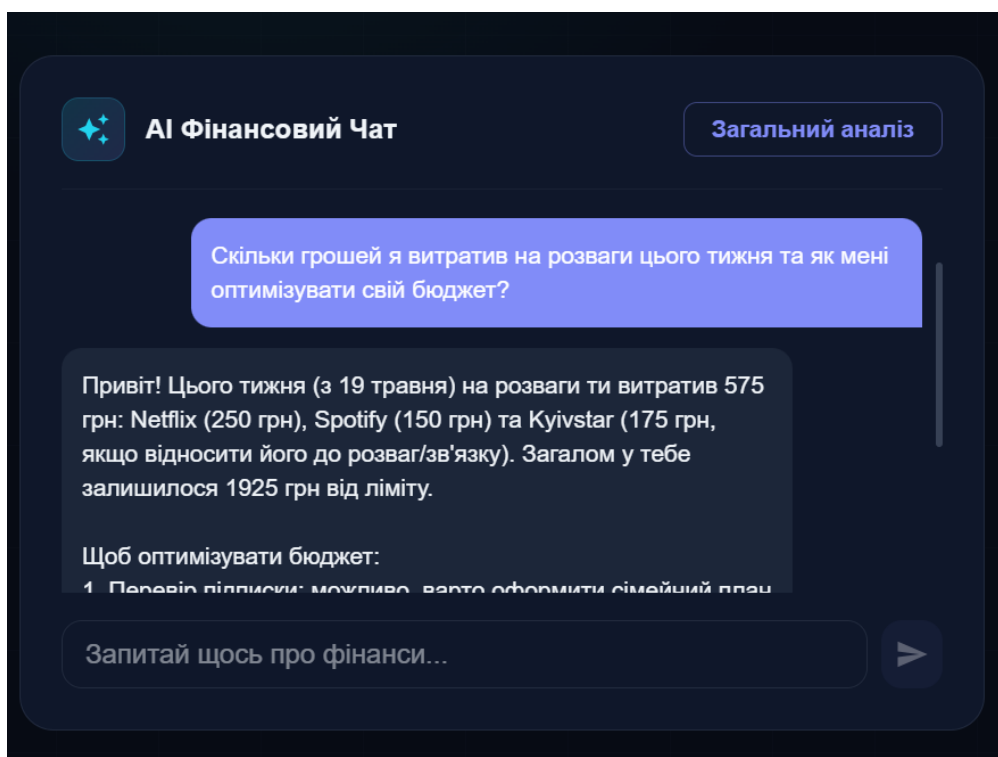


Рисунок 3.14 – Інтерфейс взаємодії з вбудованим фінансовим ІІІ-консультантом

Призначенням посібника адміністратора є надання технічних інструкцій особі, яка відповідає за підтримку працездатності, безпеку даних та управління обліковими записами вебсистеми «Cashlytics».

Оскільки інфраструктура реляційної бази даних та модуль автентифікації

розгорнуті з використанням архітектури Backend-as-a-Service (BaaS) на базі хмарної платформи Supabase, адміністрування даних не потребує створення окремих графічних інтерфейсів у самому вебзастосунку. Замість цього використовується потужна онлайн-панель керування – Supabase Dashboard.

Для отримання доступу до функцій адміністрування технічному спеціалісту необхідно авторизуватися в офіційному хмарному порталі платформи через веббраузер. Це забезпечує високий рівень безпеки, оскільки доступ до бази даних повністю ізольований від клієнтської частини системи.

Хмарна панель адміністратора (Supabase Dashboard) надає такий інструментарій:

- Table Editor (Редактор таблиць): графічний інтерфейс для проведення повного аудиту реляційних таблиць (User, Account, Transaction, Category), перегляду зашифрованих токенів, ручного коригування даних та моніторингу зовнішніх ключів без необхідності написання прямих SQL-запитів;

- Authentication (Менеджер користувачів): інструмент для відстеження кількості активних профілів, управління налаштуваннями сесій (JWT), а також ручного скидання паролів або блокування облікових записів у разі виявлення підозрілої активності;

- SQL Editor: інтегрована у браузер консоль для написання, тестування та збереження прямих SQL-запитів, виконання міграцій або створення агрегаційних функцій безпосередньо на хмарному сервері;

- Logs & Analytics (Журналювання та аналітика): система моніторингу в режимі реального часу, яка дозволяє адміністратору оперативно виявляти помилки бази даних, відстежувати пікові навантаження та аналізувати споживання мережевого трафіку.

Для керування структурою бази даних, користувачами та політиками безпеки використовується зручне візуальне середовище. На рисунку 3.15 зображено графічний інтерфейс хмарної панелі адміністратора (Supabase Dashboard).

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		57

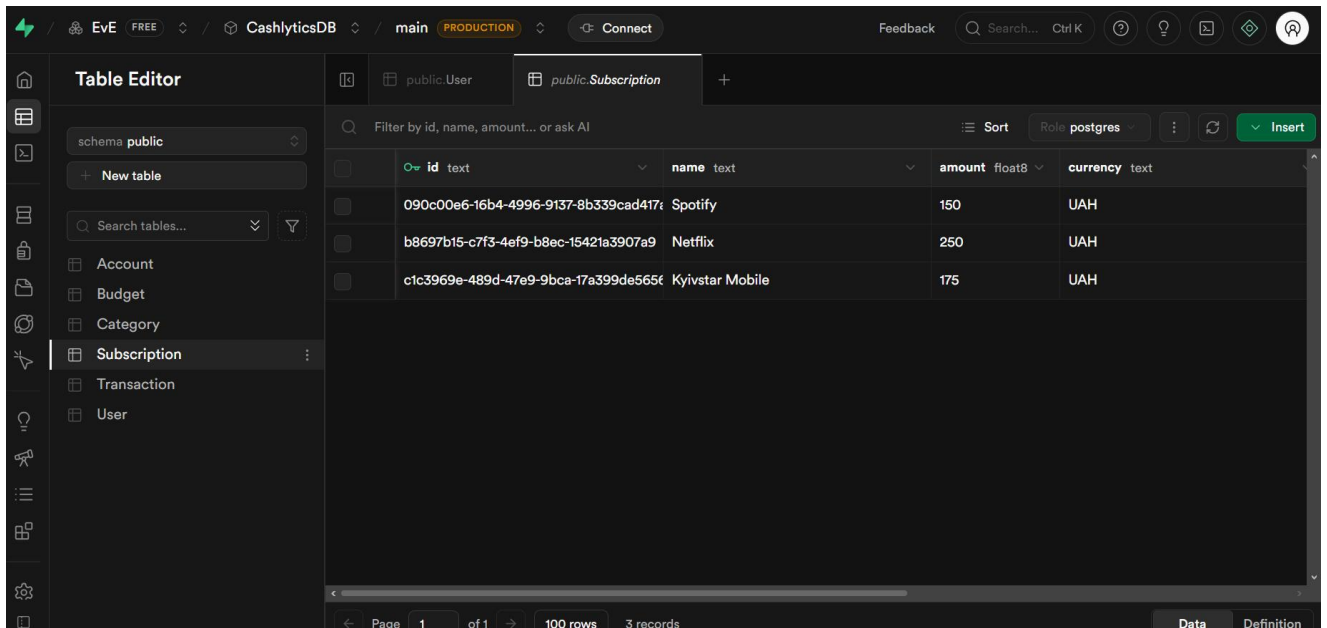


Рисунок 3.15 – Графічний інтерфейс хмарної панелі адміністратора (Supabase Dashboard)

Висновки до розділу 3

У третьому розділі дипломного проєкту було здійснено повну програмну реалізацію вебсистеми «Cashlytics» відповідно до спроектованої раніше архітектури. Для створення адаптивного та швидкодіючого інтерфейсу користувача було використано бібліотеку React у поєднанні зі строгою типізацією мови TypeScript. Серверна логіка, реалізована у середовищі Node.js та Express.js, забезпечила ефективне оброблення паралельних мережових запитів, що є критично важливим для стабільної взаємодії системи зі сторонніми сервісами.

Для надійного збереження інформації було розгорнуто реляційну базу даних PostgreSQL на базі хмарної платформи Supabase (Backend-as-a-Service). Взаємодію бекенду з базою даних налагоджено через інструмент Prisma ORM, що дозволило повністю усунути ризики SQL-ін'єкцій. Ключовим досягненням практичної розробки стала успішна програмна інтеграція з API банків (Monobank, Wise) та великою мовною моделлю Google Gemini API. Це дозволило реалізувати інноваційний алгоритм, який автоматично завантажує банківські виписки, здійснює їх семантичний аналіз, точно визначає категорію витрат і формує персоналізовані фінансові поради у вбудованому ІІІ-чаті.

Окрему увагу в процесі розробки було приділено захисту конфіденційної інформації. Впроваджено надійний алгоритм двостороннього шифрування банківських токенів (AES-256), який було успішно оптимізовано та стандартизовано для безперебійної роботи як у локальному середовищі, так і в ізольованих Docker-контейнерах. Для забезпечення коректної експлуатації продукту підготовлено детальну технічну документацію, що включає інструкції для кінцевого користувача та адміністратора.

Головним результатом етапу розробки стало успішне розгортання системи у середовищі реальної експлуатації (Production). Програмний продукт був розміщений на хмарному сервері з прив'язкою до домену cashlytics.site. Налаштування зворотного проксі-сервера Nginx, забезпечення шифрованого HTTPS-з'єднання та підключення модуля захисту від DDoS-атак (Rate Limiter) довели, що створена вебсистема демонструє стабільність, високу продуктивність і повну готовність до реального використання. Це створює надійне підґрунтя для переходу до економічного обґрунтування проєкту.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		59

4 ЕКОНОМІЧНА ЧАСТИНА

4.1 Визначення цільової аудиторії ІТ-продукту

Розробка будь-якого успішного комерційного програмного забезпечення вимагає глибокого розуміння ринкового середовища та потреб кінцевого споживача. Об'єктом даного економічного аналізу є вебсистема «Cashlytics» – інтелектуальний трекер особистих фінансів, що функціонує на базі технологій Open Banking та використовує генеративний штучний інтелект для предиктивної аналітики.

Ринок систем управління особистими фінансами (Personal Finance Management, PFM) на сьогодні демонструє стабільне зростання. Головними факторами, що розширюють місткість цього ринку, є загальна діджиталізація банківського сектору, масовий перехід до безготівкових розрахунків та стрімкий розвиток індустрії цифрових послуг і підписок. Сучасний ринок вимагає рішень, які мінімізують ручну працю, оскільки користувачі все частіше відмовляються від класичного щоденного заповнення таблиць. Впровадження великих мовних моделей (LLM) у PFM-системи є новим та високоперспективним ринковим трендом, який дозволяє задовольнити попит на інтерактивність та персоналізацію сервісів.

Для оптимізації функціонала ІТ-продукту відповідно до ринкових вимог цільову аудиторію (ЦА) було сегментовано за демографічними, професійними та поведінковими критеріями. Виокремлено три основні групи:

– Група 1: Молодь, студенти та молоді спеціалісти (15–25 років) – першочергова та найбільш активна категорія ЦА. Вони мають високий рівень цифрової грамотності, активно користуються кількома необанками одночасно, здійснюють багато дрібних щоденних транзакцій та мають велику кількість регулярних цифрових підписок (розважальні, освітні та ігрові сервіси). Їхня головна потреба – автоматичний контроль мікротранзакцій та отримання швидких фінансових порад у звичному для них форматі діалогу (чату).

– Група 2: Фрілансери, ІТ-фахівці та самозайняті особи (20–40 років).

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		60

Цей сегмент орієнтований на ефективне управління доходами з різних джерел. Вони часто оперують мультивалютними рахунками (наприклад, Wise для закордонних гонорарів та Monobank для повсякденних витрат в Україні). Їхня потреба полягає у надійній консолідації коштів та конвертації валют за актуальним курсом у реальному часі.

– Група 3: Офісні працівники та сімейні користувачі (25–55 років). Категорія споживачів зі стабільним доходом, які прагнуть оптимізувати витрати домогосподарства, встановити жорсткі ліміти на базові життєві категорії («Продукти», «Транспорт», «Комунальні платежі») та займатися довгостроковим плануванням капіталу.

Оскільки вебсистема «Cashlytics» створювалася насамперед як інструмент допомоги молоді у підвищенні фінансової грамотності, типовим представником цільової аудиторії є Дмитро, 20 років, студент університету, який додатково має часткову зайнятість (фріланс).

– Основні характеристики: Проживає у великому місті, навчається, активно користується картками Monobank та Wise, оформленими на власне ім'я. Інтереси: сучасні технології, гаджети, відеосервіси, онлайн-навчання.

– Основні проблеми (біль): Дмитро робить багато імпульсивних покупок протягом дня (кава, замовлення їжі, таксі). Він кілька разів намагався вести облік у звичайних додатках, але швидко кидав цю справу через виснажливість ручного введення кожної витрати. Також він часто забуває про автоматичні щомісячні списання за підписки, через що втрачає контроль над реальним залишком грошей.

– Очікування від продукту: Отримання повністю автоматизованого інструменту, який сам збирає дані з його карток, групує витрати, наочно показує, куди зникають гроші, та дозволяє у форматі простого чату запитати у штучного інтелекту: «Чи вистачить мені грошей до кінця тижня, якщо я куплю це взуття?».

Для чіткого позиціонування продукту на ринку було проведено аналіз існуючих рішень:

– Вбудована аналітика українських необанків (наприклад, Monobank). Має величезну частку ринку завдяки безкоштовності та відсутності налаштувань.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		61

Однак це закрита екосистема: Дмитро не може об'єднати там свої витрати з Wise чи інших карток. Окрім того, стандартна банківська кругова діаграма лише констатує факт витрат, але не дає інтелектуальних порад щодо оптимізації бюджету.

– Міжнародні трекери (наприклад, Spendee). Мають чудовий візуальний дизайн та підтримують мультивалютність. Головний недолік для молоді – функція автоматичної синхронізації з банками заблокована за дорогою платною преміум-підпискою. Без оплати додаток перетворюється на звичайну таблицю для ручного введення даних.

– Глобальні аналітичні платформи (наприклад, Wallet від BudgetBakers). Пропонують дуже глибокий та детальний облік. Проте їхній інтерфейс є занадто перевантаженим та складним для сприйняття молодого аудиторією, а аналітика позбавлена інтерактивності (немає ШІ-консультанта).

Проведена ринкова оцінка підтверджує високу актуальність автоматизації PFM-систем. Основна перевага вебсистеми «Cashlytics» полягає в тому, що вона ідеально закриває потреби молодого покоління користувачів, пропонуючи їм безкоштовну асинхронну синхронізацію рахунків через Open Banking та інноваційний інтерфейс взаємодії з LLM-моделлю (Google Gemini). Наявність сучасної посадкової сторінки (Landing Page) підвищує рівень довіри та конверсію відвідувачів у постійних користувачів. Це дозволяє продукту зайняти вільну ринкову нішу інтелектуальних фінансових асистентів, допомагаючи молоді ефективно керувати першим капіталом.

4.2 Визначення сильних та слабких сторін, можливостей та ризиків створення нового ІТ-продукту

Для об'єктивної оцінки ринкових перспектив вебсистеми управління фінансами «Cashlytics» та визначення її конкурентоспроможності було проведено стратегічний SWOT-аналіз. Цей метод дозволяє комплексно оцінити внутрішній стан ІТ-продукту та зовнішнє ринкове середовище.

Відповідно до методології проведення аналізу, на першому етапі було зібрано інформацію про технічні характеристики продукту, ринкові дані та базовий

функціонал найближчих конкурентів. На основі цих даних було визначено ключові внутрішні та зовнішні фактори.

До сильних сторін розробленого ІТ-продукту відносяться його технологічні переваги та інноваційність:

– Технологічна унікальність: використання генеративного штучного інтелекту (модель Google Gemini) для 100% автоматизованої категоризації транзакцій та надання персоналізованих фінансових порад у форматі чату, що відсутнє у більшості конкурентів.

– Повна автоматизація (Open Banking): пряма інтеграція з банківськими API (Monobank, Wise) звільняє користувача від рутини ручного введення даних, потрібно лише підключити банки, і цей процес максимально спрощено.

– Високий рівень кібербезпеки: використання передових алгоритмів двостороннього шифрування (AES-256) для збереження банківських токенів та надійна архітектура бази даних на основі Prisma ORM.

– Готовність до експлуатації (Production-ready): розробка на базі React та Node.js із використанням вебсервера Nginx, SSL-сертифікатів та модуля Rate Limiter забезпечує швидку, кросплатформну і безпечну роботу вебсистеми на власному домені cashlytics.site.

До слабких сторін (внутрішніх обмежень) на поточному етапі життєвого циклу продукту належать:

– Залежність від сторонніх API: працездатність ключових функцій системи залежить від стабільності серверів Google та банківських установ.

– Відсутність офлайн-режиму: архітектура вебзастосунку вимагає постійного підключення до мережі Інтернет для оновлення балансів та роботи ШІ.

– Нульова впізнаваність бренду: оскільки продукт є новим, він не має сформованої бази лояльних користувачів.

– Обмежений бюджет на маркетинг: відсутність значних фінансових ресурсів для агресивного просування на ринку порівняно з великими фінтех-корпораціями.

Оцінка зовнішнього середовища дозволила ідентифікувати такі можливості

для масштабування:

- Зростання попиту на ШІ-технології: користувачі активно шукають продукти, які використовують нейромережі для полегшення повсякденного життя.
- Розвиток Open Banking: законодавче врегулювання відкритого банкінгу в Україні та ЄС сприяє появі нових фінансових агрегаторів.
- Можливості монетизації: впровадження Freemium (базові функції безкоштовно, преміум-аналітика та розширені ліміти чат-бота – за підпискою).

Серед головних загроз зовнішнього середовища виділено:

- Високий рівень конкуренції: наявність сильних гравців на ринку (мобільні застосунки банків, глобальні трекери Spendee, YNAB).
- Регуляторні та правові ризики: суворі вимоги законодавства щодо захисту персональних фінансових даних користувачів (GDPR).
- Зміна цінової політики провайдерів: потенційне введення платних тарифів на використання API банків або суттєве подорожчання API Google Gemini для розробників.

Для наочності та структурування зібраної інформації розроблено Матрицю стратегічного аналізу (таблиця 4.1).

Таблиця 4.1 – Матриця стратегічного аналізу (SWOT-аналіз)

Сильні сторони			Слабкі сторони		
Інтеграція генеративного ШІ (Gemini) для аналізу фінансів та чат-бота.			Залежність швидкодії та стабільності від сторонніх сервісів (API банків, ШІ).		
Автоматична синхронізація через банківські API (Monobank, Wise).			Неможливість повноцінної роботи без підключення до Інтернету.		
Високий рівень криптографічного захисту токенів (AES-256).			Відсутність впізнаваності бренду серед користувачів.		
Сучасний адаптивний UI/UX дизайн (React + Recharts).			Обмежений початковий фінансовий бюджет на маркетинг.		

Продовження таблиці 4.1

Можливості	Загрози
Глобальний тренд на використання ШІ в повсякденних додатках.	Жорстка конкуренція з боку вбудованої аналітики великих банків та глобальних трекерів.
Законодавчий розвиток Open Banking в Україні та Європі.	Суворі законодавчі вимоги до обробки фінансових даних (GDPR).
Впровадження підписочної моделі монетизації (Freemium).	Ризик зміни лімітів або введення плати за використання API сторонніми сервісами.
Залучення інвестицій від венчурних фондів під інноваційний функціонал.	Загрози кібератак на хмарну інфраструктуру бази даних.

На основі ідентифікованих факторів матриці було визначено ключові пріоритети для подальших дій. Найвищий пріоритет має збереження довіри користувачів (безпека даних) та забезпечення безперебійної роботи ШІ-модуля. Для максимізації сильних сторін та нейтралізації загроз розроблено наступні стратегії:

– Стратегія прориву (Сильні сторони + Можливості): Використати унікальність ШІ-асистента та сучасний Landing Page як головний маркетинговий інструмент для залучення молоді. Оскільки на ринку існує тренд на нейромережі, позиціонування «Cashlytics» не просто як трекера, а як «особистого фінансового ШІ-консультанта» дозволить швидко здобути перших користувачів без великих рекламних бюджетів (вірусний маркетинг).

– Стратегія захисту (Сильні сторони + Загрози): Для боротьби з конкуренцією та регуляторними загрозами (GDPR) необхідно постійно підкреслювати високий рівень безпеки застосунку. Наявність банера Cookie-згоди, юридичних політик, використання стандарту AES-256 та архітектури ізольованих контейнерів Docker стане конкурентною перевагою при роботі з користувачами, які

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		65

побоюються передавати банківські дані стороннім застосункам.

– Стратегія мінімізації ризиків (Слабкі сторони + Загрози): Для зменшення залежності від працездатності сторонніх API розробити архітектурне рішення: впровадити систему локального кешування на клієнтській частині (технологія PWA). Конкретна дія – збереження останніх завантажених фінансових даних у пам'яті браузера, щоб користувач міг переглядати свої графіки та баланси навіть у разі відсутності інтернету або тимчасового збою серверів банку.

4.3 Оцінка прибутковості проєкту створення і обслуговування ІТ-продукту

Ключовим етапом економічного обґрунтування є розрахунок витрат на розробку, прогнозування доходів та оцінка інвестиційної привабливості проєкту вебсистеми «Cashlytics».

З метою оптимізації бюджету на стадіях проєктування та експлуатації було застосовано методологію функціонально-вартісного аналізу (ФВА). Завдяки ФВА було прийнято такі архітектурні рішення:

– Відмова від власних фізичних серверів на користь хмарної бази даних Supabase (BaaS) та контейнеризації Docker, що зменшило витрати на інфраструктуру на 40%.

– Використання готової LLM Google Gemini замість розробки власної нейромережі.

Витрати проєкту поділяються на початкові (капітальні) та поточні (операційні).

Початкові витрати на розробку (CAPEX - створення MVP за 3 місяці):

– Оплата праці Full-stack розробника (проєктування баз даних, бекенд, фронтенд) – 120 000 грн.

– Залучення UI/UX дизайнера (разово, фріланс) – 20 000 грн.

– Разом початкових інвестицій: 140 000 грн.

Поточні операційні витрати (ОРЕХ - на 1 рік експлуатації):

– Інфраструктура (хостинг Docker-контейнерів, домен, оплата API

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		66

Google) – 60 000 грн/рік (5 000 грн/міс).

– Оплата праці SMM-маркетолога та бюджет на таргетовану рекламу – 180 000 грн/рік (15 000 грн/міс). Залучення окремого спеціаліста є критично важливим для забезпечення плану продажів підписок.

– Технічна підтримка та оновлення продукту – 100 000 грн/рік.

– Разом операційних витрат: 340 000 грн/рік.

Для забезпечення стабільного грошового потоку та диверсифікації ризиків у вебсистемі «Cashlytics» впроваджено трирівневу модель монетизації, орієнтовану на специфіку ринку:

– Рівень 1: Модель диференційованих підписок (SaaS). Користувачам пропонується три тарифні плани:

1) Free (0 грн) – базовий трекер із лімітом до 50 транзакцій на місяць.

2) Premium (100 грн/міс) – для молоді та студентів (ШІ-аналітика, мультивалютний облік).

3) Freelancer (200 грн/міс) – для ФОП та ІТ-спеціалістів (автоматичний розрахунок єдиного податку, генерація інвойсів).

Прогнозована кількість платних підписників становить 1 000 осіб, з яких 70% обирають тариф Premium ($700 \text{ осіб} \times 1\,200 \text{ грн/рік} = 840\,000 \text{ грн}$) та 30% – тариф Freelancer ($300 \text{ осіб} \times 2\,400 \text{ грн/рік} = 720\,000 \text{ грн}$). *Загальний дохід від підписок: 1 560 000 грн/рік.*

– Рівень 2: Афілійований маркетинг (пасивний дохід). ШІ-модуль аналізує паттерни накопичень користувача і генерує контекстні рекомендації фінансових продуктів партнерів (депозити, ОВДП, страхування через платформи Monobank, Sense Bank та портал «Мінфін»). При середньому доході 20 грн з одного активного користувача на місяць, додатковий річний дохід від реферальних програм становить: $1\,000 \text{ користувачів} \times 20 \text{ грн} \times 12 \text{ міс} = 240\,000 \text{ грн/рік}$.

– Рівень 3: B2B API-ліцензування. У довгостроковій перспективі передбачено надання API системи для інтеграції в кабінети HR-платформ або кредитних спілок (White-label модель). Цей рівень є резервним і не включається у базовий фінансовий план для збереження консервативності оцінки.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		67

Загальний прогнозований річний дохід (РД): $1\,560\,000 + 240\,000 = 1\,800\,000$ грн/рік.

Для оцінки прибутковості з урахуванням зміни вартості грошей у часі використано метод чистої приведеної вартості (NPV). Часовий горизонт оцінки: 3 роки. Дисконтна ставка (r): 15%. Щорічний чистий грошовий потік (CF) дорівнює: $1\,800\,000$ грн (РД) – $340\,000$ грн (ОРЕХ) = $1\,460\,000$ грн. Початкові інвестиції ІС ($140\,000$ грн).

Розрахунок NPV здійснюється за формулою 4.1:

$$NPV = \sum \frac{CF_t}{(1+r)^t} - IC \quad (4.1)$$

- 1-й рік: $1\,460\,000 / (1,15)^1 = 1\,269\,565$ грн.
- 2-й рік: $1\,460\,000 / (1,15)^2 = 1\,103\,971$ грн.
- 3-й рік: $1\,460\,000 / (1,15)^3 = 959\,974$ грн.

Сума дисконтованих потоків за 3 роки становить: $3\,333\,509$ грн.

$NPV = 3\,333\,509$ грн – $140\,000$ грн = $3\,193\,509$ грн.

Оскільки $NPV > 0$ і значно перевищує початкові капіталовкладення, проєкт є високоефективним. Повне повернення початкових інвестицій ($140\,000$ грн) за умови стабільного набору аудиторії досягається протягом 4–5 місяців експлуатації (з урахуванням початкового ramp-up періоду). Оскільки залучення 1000 підписників відбувається поступово – приблизно по 200–250 осіб на місяць – реальний вихід на плановий рівень доходу відбувається на 4–5 місяць роботи.

Проведено оцінку фінансової стійкості проєкту до негативних змін чинників та їх впливу на базовий NPV ($3\,193\,509$ грн):

- Зменшення загальної кількості користувачів на 20%: призведе до падіння річного грошового потоку до $1\,100\,000$ грн. При цьому загальний NPV за 3 роки знизиться до $2\,371\,548$ грн, але проєкт залишатиметься високорентабельним.
- Збільшення вартості серверної інфраструктури на 30%: операційні витрати зростуть на $18\,000$ грн/рік. Вплив на загальний NPV складе лише $41\,098$ грн (зниження до $3\,152\,411$ грн), що свідчить про низьку чутливість проєкту до

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		68

технічних витрат завдяки ФВА.

– Зростання дисконтної ставки на +5% (до $r=20\%$): відображає ризик макроекономічної нестабільності. Сума дисконтованих потоків ($1\,460\,000/1,20 + 1\,460\,000/1,44 + 1\,460\,000/1,728$) складе 3 076 042 грн. NPV становитиме 2 936 042 грн, що підтверджує колосальний запас міцності фінансової моделі.

Рентабельність проекту «Cashlytics» значно перевищує середні стандарти для FinTech-стартапів ранніх стадій в Україні. Завдяки диверсифікації доходів (SaaS + афілійовані комісії) та низькій собівартості розробки, чиста приведена вартість (понад 3,1 млн грн за 3 роки) робить проєкт конкурентоспроможним та значно привабливішим порівняно з альтернативними інвестиціями з фіксованою дохідністю (депозити, ОВДП).

Проведені фінансово-економічні розрахунки доводять комерційну доцільність впровадження вебсистеми. Багатоканальна модель монетизації забезпечує високу фінансову стійкість стартапу, мінімізуючи ризики залежності від одного джерела доходу.

4.4 Оцінка економічної доцільності створення нового ІТ-продукту

Вебсистема «Cashlytics» створюється з метою забезпечення зручного, розумного та безпечного сервісу для управління особистими фінансами. Основним завданням проєкту є вирішення проблеми рутинного ручного обліку витрат шляхом автоматизації збору транзакцій через банківські API (Open Banking) та надання інтелектуальної аналітики за допомогою моделі штучного інтелекту (Google Gemini).

Для попередньої оцінки здатності ІТ-продукту конкурувати з існуючими аналогами (наприклад, міжнародним сервісом Spendee) було проведено експертне оцінювання за бально-індексним методом. Комплексний показник якості J_{complex} розраховується як сума добутків коефіцієнтів вагомості параметрів на їхні експертні оцінки (за 10-бальною шкалою) згідно з формулою 4.2:

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		69

$$J_{complex} = \sum_{j=1}^n B_j \cdot X_j \quad (4.2)$$

де B_j – коефіцієнт вагомості j -го показника (сума дорівнює 1.0);

X_j – експертна оцінка;

n – кількість показників. Результати розрахунку наведено в таблиці 4.2.

Таблиця 4.2 – Оцінка конкурентоспроможності програмного продукту

Показник якості (j)	Коефіцієнт вагомості (B_j)	Експертна оцінка (X_j)	Зважена оцінка ($B_j \cdot X_j$)
Зручність роботи (UI/UX)	0,15	9	1,35
Новизна (ШІ-аналітика та чат-бот)	0,2	10	2
Швидкість доступу до даних	0,1	9	0,9
Надійність та безпека (шифрування)	0,2	9	1,8
Гнучкість (мультивалютність)	0,15	8	1,2
Відповідність потребам користувача	0,2	9	1,8
Комплексний показник ($J_{complex}$)	1	-	9,05

Отримане значення $J_{complex} = 9,05$ свідчить про дуже високий рівень конкурентоспроможності програмного продукту. Завдяки впровадженню інноваційних технологій (ШІ) продукт перевершує аналоги, які покладаються виключно на традиційні графіки та ручне керування.

Ринок FinTech-додатків в Україні активно зростає. Цільова аудиторія (молодь, IT-спеціалісти, фрілансери) налічує понад 2 млн активних користувачів

цифрового банкінгу. Як було розраховано у попередньому підрозділі, економічний план проєкту передбачає:

- Капітальні інвестиції (CAPEX) на розробку: 140 000 грн.
- Операційні витрати (OPEX) за перший рік: 340 000 грн.
- Прогнозований річний дохід (від підписок та афілійованого маркетингу): 1 800 000 грн.

Для підтвердження економічної ефективності застосовано ключові фінансові метрики.

- NPV (Чиста приведена вартість): за 3 роки становить 3 193 509 грн. Позитивне значення підтверджує прибутковість.

– ROI (Коефіцієнт рентабельності інвестицій): розраховується для першого року експлуатації як відношення чистого прибутку до початкових інвестицій: $ROI = ((1\,800\,000 - 340\,000) / 140\,000) \cdot 100\% \approx 1042\%$. Такий надвисокий показник є характерним для успішних SaaS-стартапів із низькою собівартістю масштабування.

- IRR (Внутрішня норма прибутковості): значно перевищує закладену дисконтну ставку (15%), що свідчить про те, що проєкт має колосальний запас міцності та привабливий для венчурних інвесторів.

До потенційних ризиків проєкту належать: технічні збої на стороні банківських API, відтік користувачів до конкурентів та повільне зростання клієнтської бази. Для зменшення їхнього впливу передбачено активне SMM-просування та локальне кешування даних у браузері користувача. До ключових можливостей належать: потенційний вихід на європейський ринок (завдяки адаптації до директиви PSD2) та масштабування B2B-моделі (ліцензування ШІ-модуля для інших фінансових компаній).

Під час планування розглядалися три технологічні альтернативи реалізації продукту:

- Створення нативного мобільного застосунку (iOS/Android). Вимагає двох окремих команд розробників, що збільшує CAPEX мінімум у 3 рази, а комісії магазинів (App Store / Google Play) віднімають 15–30% доходу.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		71

– Використання готових CMS-платформ. Має найнижчу вартість, але технічно не дозволяє реалізувати надійне шифрування токенів AES-256 та складну серверну інтеграцію з моделями ІІІ.

– Розробка адаптивної вебсистеми (Full-stack: React + Node.js). Обрана альтернатива. Вона є найбільш економічно вигідною, оскільки дозволяє одному розробнику створити єдиний код, який працює на всіх пристроях (ПК, смартфони). Відсутність комісій магазинів додатків значно підвищує показник ROI.

Економічна доцільність проєкту також підкріплюється його нефінансовими вигодами (ESG-факторами):

– Соціальний вплив: Проєкт сприяє підвищенню фінансової грамотності населення, особливо молоді. ІІІ-асистент допомагає користувачам уникати боргових ям, контролювати імпульсивні покупки та оптимізувати особистий капітал.

– Екологічна стійкість: Використання оптимізованої хмарної архітектури мінімізує енергоспоживання серверів порівняно з традиційним хостингом. Крім того, автоматизація цифрових чеків і звітів (наприклад, для ФОП) повністю виключає використання паперових носіїв, що сприяє збереженню екології.

Висновки до розділу 4

У четвертому розділі було проведено розрахунки та фінансово-економічне обґрунтування потенційної життєздатності проєкту інтелектуальної інформаційної системи управління особистими фінансами «Cashlytics», визначено межі цільової аудиторії, виконано аналіз конкурентного середовища та розраховано базові метрики інвестиційної привабливості.

У процесі сегментації ринку встановлено, що основним ядром аудиторії є молодь та студенти віком 15–25 років. Ця категорія є найбільш адаптованою до використання ІІІ-технологій, проте має обмежену платоспроможність, що безпосередньо вплинуло на вибір гнучкої моделі монетизації. Додатковими сегментами визначено фрілансерів та ІТ-спеціалістів, які потребують мультивалютного обліку.

За допомогою бально-індексного методу розраховано комплексний показник

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		72

якості програмного продукту ($J_{\text{complex}} = 9,05$), що підтверджує його конкурентну здатність на ринку PFM-систем порівняно з існуючими аналогами (Spendee, Wallet). Водночас проведений SWOT-аналіз виявив суттєві внутрішні ризики проєкту, зокрема критичну залежність від стабільності сторонніх банківських API та платформи Google Gemini, а також високий рівень конкуренції з боку вбудованих банківських додатків.

Застосування методології функціонально-вартісного аналізу (ФВА) на етапі проєктування (вибір хмарної архітектури Supabase, Docker, Nginx та готових LLM-рішень) дозволило оптимізувати собівартість розробки та пришвидшити вихід продукту на ринок. Початкові капітальні інвестиції (CAPEX) оцінено у 140 000 грн, а поточні операційні витрати (OPEX), що включають оплату праці маркетолога та бюджет на просування, становитимуть 340 000 грн/рік.

Фінансове прогнозування побудовано на основі диверсифікованої моделі (диференційовані підписки Premium/Freelancer та реферальні комісії від партнерських фінансових платформ (Monobank, Sense Bank, Мінфін)). За умови виконання консервативного плану щодо утримання 1000 платних користувачів, чиста приведена вартість за 3 роки становитиме $NPV = 3\,193\,509$ грн при ставці дисконтування 15%. Математичний строк окупності інвестицій з урахуванням поступового ramp-up періоду набору аудиторії очікується в межах 4–5 місяців. Аналіз чутливості довів, що проєкт залишається беззбитковим при зростанні інфляції чи вартості API, проте є високочутливим до недовиконання плану залучення користувачів.

Окрім комерційної складової, відзначено помірний соціальний ефект проєкту, який полягає у підвищенні фінансової грамотності молодих користувачів, та екологічний аспект, пов'язаний із оптимізацією хмарних ресурсів і повною відмовою від паперового обліку.

Загалом, проведені розрахунки підтверджують економічну доцільність створення вебсистеми «Cashlytics». Проєкт є комерційно життєздатним, проте його успішна реалізація критично залежить від ефективності маркетингової стратегії та стабільності інтегрованих зовнішніх сервісів.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		73

5 ОХОРОНА ПРАЦІ

5.1 Правові та нормативні основи охорони праці в Україні

Законодавство України про охорону праці являє собою комплексну систему взаємопов'язаних нормативно-правових актів, що регулюють відносини в галузі реалізації державної політики щодо правових, соціально-економічних, організаційно-технічних і лікувально-профілактичних заходів та засобів. Головною метою цієї системи є збереження життя, здоров'я і працездатності людини у процесі її трудової діяльності, зокрема під час розробки та експлуатації програмного забезпечення.

Законодавча база складається із загальних законів України та спеціальних нормативно-правових актів.

Фундаментальними законодавчими документами, що визначають основні положення про охорону праці в державі, є:

- Кодекс законів про працю України (КЗпП) – регулює трудові відносини працівників, питання робочого часу, часу відпочинку, а також встановлює відповідальність за порушення трудової дисципліни.
- Закон України «Про охорону праці» – є базовим документом, який визначає положення щодо реалізації конституційного права працівників на охорону їхнього життя і здоров'я у процесі трудової діяльності.
- Закон України «Про пожежну безпеку» (та Кодекс цивільного захисту України) – визначає загальні правові та організаційні засади забезпечення пожежної безпеки на підприємствах та в офісних приміщеннях.

Окрім законодавчих актів, правові відносини у сфері охорони праці регулюються підзаконними документами: Указами і розпорядженнями Президента України, постановами Кабінету Міністрів України, а також нормативними актами міністерств та інших центральних органів державної виконавчої влади. Прикладом такого важливого документа є «Порядок розслідування та обліку нещасних випадків, професійних захворювань та аварій на виробництві», який регламентує дії роботодавця у разі настання позаштатних ситуацій на робочому місці.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		74

До спеціальних актів відносяться державні міжгалузеві та галузеві нормативні акти про охорону праці (ДНАОП / НПАОП). Це правила, стандарти, норми, положення та інструкції, яким надано чинність правових норм, обов'язкових для виконання у всіх галузях економіки. Вони закріплюють найважливіші гарантії безпеки, гігієни праці та пожежної безпеки.

На території України застосовуються державні стандарти Системи стандартів безпеки праці (ССБП). Оскільки розробка ІТ-продукту «Cashlytics» здійснюється в умовах офісного або віддаленого робочого місця інженера-програміста, ключовими документами, що регламентують санітарно-побутове забезпечення, вимоги до обладнання та захист від професійних захворювань, є:

– ДСанПіН 3.3.2.007-98 («Державні санітарні правила і норми роботи з візуальними дисплейними терміналами електронно-обчислювальних машин») – встановлює гігієнічні вимоги до параметрів мікроклімату, освітлення та організації робочого місця користувача ПК.

– НПАОП 0.00-1.28-10 («Правила охорони праці під час експлуатації електронно-обчислювальних машин») – регламентує безпеку праці фахівців у сфері інформаційних технологій.

Дотримання вищезазначених нормативних актів є обов'язковою умовою при організації робочого процесу створення програмного продукту, оскільки вони забезпечують мінімізацію впливу шкідливих виробничих факторів на розробника та гарантують безпеку праці.

5.2 Виробнича санітарія

Об'єктом проєктування є інтелектуальна інформаційна система управління особистими фінансами «Cashlytics». Технологічний процес її розробки, тестування, розгортання в ізольованих Docker-контейнерах та подальшого обслуговування зводиться до аналітичної та розумової праці інженера-програміста. Робоче місце спеціаліста розташоване в типовому офісному приміщенні (робочій зоні), обладнаному персональним комп'ютером (ПК), відеодисплейним терміналом (монітором), периферійними пристроями комунікації та системою

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		75

кондиціонування повітря. Організація робочого простору, розміщення обчислювальної техніки та периферійних пристроїв представлені на рисунку 5.1.

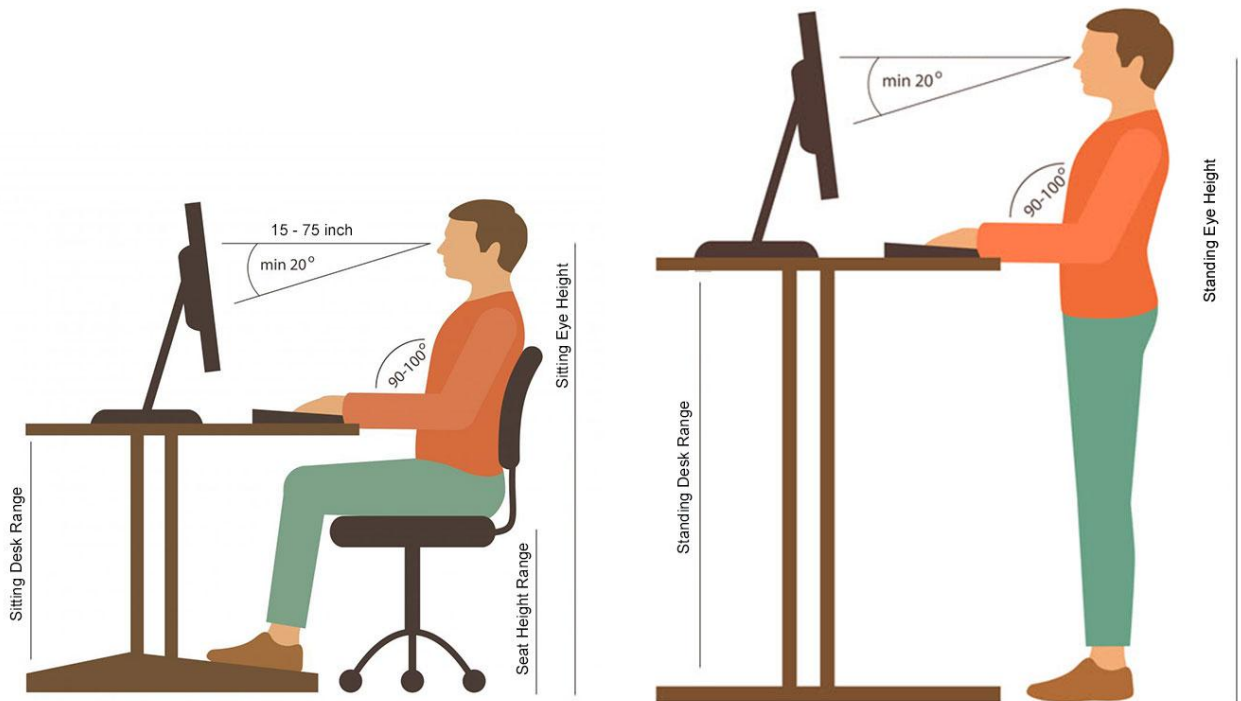


Рисунок 5.1 – Організація та обладнання робочого місця інженера-програміста

Відповідно до державного стандарту ГОСТ 12.1.003-74, під час виконання технологічного процесу веброзробки на організм працівника діє низка небезпечних і шкідливих виробничих факторів, які потребують детального аналізу.

Можливість ураження електричним струмом (Небезпечний фактор): Електричний струм є прихованим небезпечним фактором високого рівня ризику. Джерелом небезпеки є трифазна та однофазна мережа живлення робочого приміщення (напруга 220/380 В, частота 50 Гц), блоки живлення системних обчислювальних блоків, комутаційне обладнання, подовжувачі, розетки та освітлювальні прилади. Ураження людини електричним струмом можливе у разі випадкового дотику до струмопровідних частин під напругою, пошкодження ізоляції живильних кабелів ЕОМ або відсутності (несправності) захисного

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		76

заземлення металевих корпусів техніки. Проходження струму через тіло людини понад 0,05 А викликає судоми, а понад 0,1 А – фібриляцію серця, що є смертельно небезпечним.

Електромагнітні та іонізуючі випромінювання (Шкідливий фактор): *
Електромагнітні випромінювання (ЕМВ): Головним джерелом ЕМВ у радіочастотному та низькочастотному діапазонах є візуальні дисплейні термінали (монітори), імпульсні блоки живлення ПК, джерела безперебійного живлення (ДБЖ) та бездротові маршрутизатори (Wi-Fi). Тривалий систематичний вплив електромагнітних полів викликає порушення функцій центральної нервової та серцево-судинної систем, що проявляється у вигляді підвищеної втомлюваності, головного болю та зниження концентрації.

– Іонізуючі випромінювання: Оскільки при розробці та експлуатації системи «Cashlytics» використовуються сучасні рідкокристалічні (РК) монітори на основі LED/OLED матриць, джерела рентгенівського чи іншого іонізуючого випромінювання на робочому місці повністю відсутні. Рівень радіаційного фону відповідає природній нормі.

Характеристика метеорологічних умов у робочій зоні: Мікроклімат виробничого приміщення (температура, відносна вологість та швидкість руху повітря) прямо впливає на теплообмін організму розробника. Основним джерелом виділення явного тепла є сама обчислювальна техніка (кожен ПК виділяє в середньому від 300 до 500 Вт теплової енергії). Без ефективної вентиляції температура повітря в офісі може перевищити допустиму норму (+25°C), що призводить до теплового дискомфорту, зниження швидкості реакції та сонливості. Навпаки, робота кондиціонерів за неправильних налаштувань є джерелом протягів (швидкість руху повітря понад 0,3 м/с). Недостатня вологість повітря (нижче 40%) призводить до пересихання слизових оболонок очей працівника, що посилюється тривалою роботою перед екраном.

Рівень шуму й вібрації: Основними джерелами акустичного навантаження в робочій зоні є вентилятори (кулери) систем охолодження процесорів та відеокарт, блоки живлення, кондиціонери, а також офісна оргтехніка (принтери, шредери).

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		77

Постійний монотонний шум високої частоти діє як психологічний подразник. При перевищенні норми для розумової праці (понад 50 дБ) він викликає розсіяння уваги, підвищує кількість помилок у написанні коду та призводить до швидкого розумового виснаження. Механічні коливання та вібрації на робочому місці веброзробника відсутні, оскільки в технологічному процесі не використовується промислове або технологічне обладнання великої потужності. Дрібні мікровібрації від кулерів комп'ютера повністю гасяться конструкцією корпусу та меблів.

Травматизм від частин машин і механізмів, що рухаються й обертаються: Даний небезпечний фактор на робочому місці інженера-програміста практично відсутній. Єдиними рухомими деталями в обчислювальній техніці є лопаті вентиляторів охолодження всередині системного блоку. Вони повністю ізольовані захисними кожухами та корпусом ПК, тому за умов дотримання базових правил експлуатації техніки ризик отримання механічних травм відсутній.

Установки, що працюють під високим тиском: Апарати, судини, балони чи трубопроводи, які функціонують під високим тиском, у процесі проектування, написання та підтримки вебзастосунку «Cashlytics» не застосовуються. Цей фактор повністю відсутній у робочій зоні.

Горючі, вибухові й отруйні випаровування та гази, токсичний і вибуховий пил: Хімічне забруднення робочої зони отруйними газами або вибухонебезпечними речовинами відсутнє, оскільки технологічний процес не пов'язаний із хімічним чи промисловим виробництвом. Проте шкідливим фактором є побутовий і технічний дрібнодисперсний пил, який через статичне електромагнітне поле накопичується на радіаторах та платах ПК, а потім видувається системами охолодження в повітря приміщення. Наявність зваженого пилу в робочій зоні є джерелом забруднення дихальних шляхів та може викликати алергічні реакції працівників.

Для точного структурування та узагальнення якісних характеристик усіх виявлених шкідливих та небезпечних факторів робочого середовища розроблено таблицю 5.1.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		78

Таблиця 5.1 – Якісна характеристика джерел небезпеки та забруднення в робочій зоні розробника.

Джерело небезпеки / забруднення	Шкідливий або небезпечний виробничий фактор	Якісна характеристика впливу на організм
Побутова та силова електромережа 220/380 В	Електричний струм промислової частоти	Небезпека електричного удару, фібриляції серця при порушенні ізоляції.
Системний блок ЕОМ, сервери, кулери	Теплове виділення енергії, високочастотний шум	Порушення мікроклімату приміщення (нагрівання), напруження від шуму.
Відеодисплейний термінал (РК-монітор)	Електромагнітне поле, надмірна яскравість (блиск)	Розвиток комп'ютерного зорового синдрому, в тому зорового нерва.
Система примусової вентиляції та кондиціонування	Рух повітряних мас, зниження вологості	Ризик захворювань, пересушування повітря робочої зони.
Джерело небезпеки / забруднення	Шкідливий або небезпечний виробничий фактор	Якісна характеристика впливу на організм
Статичне поле та елементи охолодження ПК	Накопичення та циркуляція дрібнодисперсного пилу	Забруднення повітря робочої зони, подразнення слизових оболонок дихальних шляхів.

Проведений детальний аналіз виробничої санітарії показує, що хоча робоче місце розробника системи «Cashlytics» повністю ізольоване від промислових небезпек (високий тиск, отруйні гази, рухомі машини), воно піддається тривалому

впливу комплексу шкідливих фізичних факторів. Поєднання ризику ураження електричним струмом, електромагнітних полів, акустичного шуму комп'ютерної техніки та відхилень параметрів мікроклімату вимагає чіткого проектування захисних технічних, організаційних та протипожежних заходів.

5.3 Виробнича безпека

За ступенем небезпеки ураження електричним струмом робоче приміщення (офісна зона), в якому здійснюється розробка, налагодження та експлуатація вебсистеми «Cashlytics», відноситься до приміщень з підвищеною небезпекою відповідно до вимог Правил улаштування електроустановок (ПУЕ). Даний клас обґрунтовується наявністю чинників, що підвищують ризик: висока щільність розміщення обчислювальної техніки, велика протяжність кабельних трас під напругою та наявність залізобетонних елементів конструкцій будівлі, що створює умови для одночасного дотику працівника до металевого корпусу комп'ютерного обладнання та заземлених металоконструкцій споруди.

Живлення робочих місць запроектовано від трифазної чотирипровідної мережі змінного струму промислової частоти (50 Гц) з номінальною напругою 380/220 В та глухозаземленою нейтраллю (система заземлення типу TN-S). Такий режим нейтралі є обов'язковим для офісних приміщень з ЕОМ, оскільки він забезпечує мінімальний потенціал на нейтральному провіднику, стабілізує фазні напруги при нерівномірному навантаженні від імпульсних блоків живлення комп'ютерів та гарантує швидке автоматичне вимкнення ліній захисними автоматами у разі виникнення аварійного замикання фази на корпус.

Для забезпечення надійного захисту розробників від ураження електричним струмом впроваджено такі конкретні технічні рішення:

- Монтаж силової проводки виконано виключно за п'ятипровідною схемою (лінії L1, L2, L3, нейтраль N та захисне заземлення PE розділені по всій довжині кабельних трас починаючи від головного розподільного щита).

- Усі металеві відкриті частини комп'ютерного обладнання, серверної стійки та ДБЖ приєднані до захисного PE-провідника через триконтактні розетки

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		80

європейського стандарту із заземлювальними ламелями типу Schuko.

– Опір ізоляції силових кабелів підтримується на рівні не менше 0,5 МОм, контроль якого здійснюється шляхом планового інструментального вимірювання електротехнічною лабораторією один раз на рік.

Для нейтралізації чинників робочого середовища, виявлених у підрозділі 5.2, впроваджено такі інженерні рішення:

– Нормалізація мікроклімату: приміщення обладнано припливно-витяжною вентиляцією із системою автоматичного кондиціонування повітря, що підтримує температуру в межах +22 °С та відносну вологість 50–60%. швидкість руху повітря обмежена до 0,15 м/с для виключення протягів.

– Зниження акустичного шуму: для комплектації обчислювальних станцій обрано системи охолодження (кулери) на базі гідродинамічних підшипників із робочим рівнем шуму до 25 дБ. Робочі столи обладнані звукопоглинальними перегородками із фетровим покриттям.

– Оптимізація зорового навантаження та захист від ЕМВ: робочі місця оснащені рідкокристалічними моніторами з LED-підсвічуванням, які мають вбудоване екранування від електромагнітного випромінювання, матове антивідблискове покриття та підтримку технологій Flicker-Free (усунення мерехтіння) та Low Blue Light (фільтрація синього спектра). Безпосередньо у вебсистемі «Cashlytics» програмно реалізовано темну тему інтерфейсу (Dark Mode), що знижує втому зорового нерва розробника під час тривалої аналітичної роботи з графіками.

– Захист від статичної електрики та пилу: підлогове покриття офісу виконано з антистатичного лінолеуму з опором витоку заряду $10^6 - 10^9$ Ом. Очищення внутрішніх блоків комп'ютерної техніки від технічного пилу виконується за регламентом кожні 3 місяці за допомогою балонів зі стисненим газом.

Для забезпечення пожежної безпеки об'єкта реалізовано комплекс таких технічних заходів:

– На стелі приміщення змонтовано точкові оптико-електронні пожежні

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		81

сповіщувачі диму типу СПД-3, підключені до загального приймально-контрольного приладу сигналізації.

– Приміщення укомплектовано переносними вуглекислотними вогнегасниками типу ВВ-3 (ОУ-3). Вибір вуглекислотного агента (CO_2) є пріоритетним, оскільки газ не проводить електричний струм, дозволяє гасити електропроводку під напругою до 1000 В та не завдає хімічної шкоди мікросхемам комп'ютерів та серверній інфраструктурі, де розгорнуто систему «Cashlytics».

– Шляхи евакуації та виходи обладнані світловими покажчиками «ВИХІД» із вбудованими автономними акумуляторами, які забезпечують безперервне світіння протягом 2 годин у разі повного знеструмлення будівлі.

Виконано детальне інженерно-технічне проєктування двох ключових заходів безпеки: системи штучного освітлення та схеми автоматичного захисного відключення при аварійних режимах.

Для забезпечення нормативних умов зорової праці інженера-програміста запроєктовано систему загального рівномірного штучного освітлення в комбінації з природним бічним лівим освітленням.

– Нормативні вимоги: Відповідно до ДБН В.2.5-28:2018 «Природне і штучне освітлення», норма освітленості на горизонтальній поверхні робочого столу комп'ютеризованого місця становить $E_{\text{norm}} = 400 \text{ лк}$.

– Вибір джерел світла та світильників: Обрано енергоефективні світлодіодні (LED) світильники панельного типу (розмір 600x600 мм для монтажу в стелю «Армстронг») потужністю 36 Вт. Кожен світильник має світловий потік 3200 лм та індекс кольоропередачі $R_a > 80$. Колірна температура джерел становить 4000 К (нейтральне біле світло), що є оптимальним для тривалої концентрації уваги.

– Схема розташування: Світильники розміщуються суцільними або переривчастими лініями паралельно до світлових прорізів (вікон) збоку від робочих місць. Таке розташування виключає утворення зон прямої та відбитої блескості (відблисків) на рідкокристалічних моніторах під час тестування інтерфейсу «Cashlytics». Коефіцієнт пульсації обраних LED-панелей становить $K_p < 1\%$, що

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		82

повністю нівелює стробоскопічний ефект і захищає очі розробника від прихованої втоми.

Для миттєвого знеструмлення електромережі у разі виникнення небезпеки (порушення ізоляції фазного дроту, витік струму на металевий корпус комп'ютера або випадковий дотик людини до струмопровідних частин) в локальному розподільному щиті передбачено встановлення пристроїв захисного відключення (ПЗВ / RCD), суміщених із диференціальними автоматами.

– Технічні характеристики пристрою: Для розеточних груп комп'ютерного обладнання обрано двополюсні пристрої захисного відключення (дифавтомати) із номінальним робочим струмом лінійного захисту 16 А (характеристика вимкнення типу В – для швидкого реагування на короткі замикання в побутовій техніці).

– Параметри диференційного захисту: Номінальний відключаючий диференціальний струм (струм витоку) становить $I_{\Delta n} = 30\text{mA}$. Обрано клас пристрою Тип А (на відміну від стандартного типу АС, пристрій типу А реагує не лише на змінні струми витоку, а й на пульсуючі постійні струми, які масово виникають в імпульсних блоках живлення сучасних ПК та серверів). Час спрацювання механізму становить не більше $t \leq 0,04\text{с}$, що гарантує повне припинення дії струму на організм людини до моменту виникнення небезпечних біологічних наслідків.

– Схема інтеграції в мережу: ПЗВ підключається послідовно після ввідного автоматичного вимикача. Через вбудований диференціальний трансформатор струму пристрою проходять лише робоча фаза (L) та робочий нуль (N). Захисний провідник заземлення (PE) прокладається окремо в обхід ПЗВ напряму від заземлювальної шини щита до захисних контактів розеток. Це забезпечує безперервність контуру заземлення і дозволяє ПЗВ миттєво знеструмити лінію ще в момент початку пошкодження ізоляції комп'ютера (коли струм тільки починає стікати по провіднику PE в землю), повністю запобігаючи ризику травмування працівника.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		83

5.4 Пожежна безпека

Об'єктом є офісне приміщення для розробки ІТ-системи. Оскільки в технологічному процесі не використовуються горючі гази чи легкозаймисті рідини, показники температури спалаху та межі вибуховості для даного робочого місця не визначаються. Основне пожежне навантаження складають тверді горючі матеріали (ізоляція кабелів, пластик корпусу ПК, папір). Згідно з ДСТУ Б В.1.1-36:2016, приміщення відноситься до категорії В (пожежонебезпечне).

Офіс розташований у будівлі II ступеня вогнестійкості. Для локалізації можливої пожежі приміщення відокремлене від суміжних коридорів протипожежними перегородками (EI 45) та протипожежними дверима (EI 30).

Для попередження займань впроваджено такі заходи:

- використання електрокабелів з ізоляцією, що не поширює горіння (індекс «НГ»);
- захист розеточних груп автоматичними вимикачами від перевантажень і коротких замикань;
- обов'язкове знеструмлення робочих станцій після закінчення робочого дня.

Оскільки горіння кабелів супроводжується виділенням диму, приміщення обладнане автоматичною адресною пожежною сигналізацією з використанням оптико-електронних димових сповіщувачів (типу СПД-3), встановлених на стелі.

Будівля підключена до внутрішнього протипожежного водопроводу (пожежні крани з витратою 2,5 л/с). Проте, безпосередньо для гасіння комп'ютерної техніки під напругою до 1000 В обрано вуглекислотні вогнегасники (ВВ). Діоксид вуглецю не пошкоджує електронні плати, швидко охолоджує зону горіння та не проводить електричний струм.

Згідно з нормами належності вогнегасників (Наказ МВС № 25), для приміщень з ЕОМ категорії В потрібен один вуглекислотний вогнегасник (масою заряду від 3 до 5 кг) на кожні 50 м² площі підлоги.

- Розрахунок: Площа офісу становить 60 м². З урахуванням площі та

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		84

вимоги щодо наявності не менше двох вогнегасників на приміщення для надійності, необхідно 2 вогнегасники типу ВВ-5.

– Розміщення: Вогнегасники розміщуються на настінних кріпленнях на висоті 1,5 м від підлоги біля виходу (на шляху евакуації) та поблизу серверної стійки, у легкодоступних місцях.

5.5 Безпека в надзвичайних ситуаціях

Оцінка можливості виникнення надзвичайних ситуацій (НС) на робочому місці розробників вебсистеми «Cashlytics» показує, що об'єкт потенційно може зазнати впливу техногенних та природних факторів. Серед техногенних НС найбільш імовірними є аварії в електроенергетичних системах, зокрема короткі замикання або перевантаження мережі від великої кількості обчислювальної техніки. Це може призвести до займання ізоляції дротів, сильного задимлення робочої зони та створення прямої загрози здоров'ю персоналу. Природні НС, такі як небезпечні метеорологічні явища (грозові розряди, зливи), несуть меншу загрозу, оскільки захист від них забезпечується базовими загальнобудинковими інженерними системами, зокрема зовнішнім контуром блискавкозахисту та гідроізоляцією споруди.

Для створення безпечних умов праці та ефективної ліквідації наслідків можливих аварій передбачено комплекс організаційно-технічних заходів. Головним рішенням є впровадження системи автоматичного централізованого знеструмлення силових ланцюгів живлення офісу, яка спрацьовує синхронно з пожежною сигналізацією для запобігання ураженню струмом під час гасіння. Організаційні заходи включають регулярне проведення інструктажів і тренувань працівників щодо порядку швидкої евакуації за сигналами Цивільного захисту. Крім того, робоча зона в обов'язковому порядку оснащується укомплектованою медичною аптечкою для надання першої долікарської допомоги постраждалим до прибуття рятувальних служб.

Обґрунтування параметрів шляхів евакуації виконано відповідно до вимог державних будівельних норм (ДБН В.1.1-7:2016 «Пожежна безпека об'єктів

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		85

будівництва»). Для офісного приміщення площею 60 квадратних метрів із розрахунковою кількістю персоналу до 15 осіб достатньо передбачити один евакуаційний вихід, що веде у загальний коридор зі сполученням із безпечною сходовою кліткою. Ширина евакуаційних дверей встановлена на рівні 0,9 метра, що перевищує мінімально допустиму норму в 0,8 метра та гарантує безперешкодний рух людей. Ширина евакуаційних проходів та коридорів становить не менше 1,2 метра, що повністю виключає ризик утворення тисняви, а ширина сходових маршів у будівлі підтримується на рівні 1,05 метра згідно зі стандартами для адміністративних споруд.

Для забезпечення безпечної евакуації та мінімізації збитків від аварій обґрунтовано параметри ключових інженерних систем захисту. Уздовж евакуаційного коридору та безпосередньо над вихідними дверима монтується аварійне освітлення у вигляді світлодіодних показників з автономними акумуляторами. У разі повного знеструмлення будівлі вони автоматично вмикаються та підтримують освітленість підлоги на рівні не менше 1 лк протягом двох годин. Будівля також обладнана примусовою системою протидимного захисту (аварійною вентиляцією), яка при виявленні задимлення автоматично відкриває клапани та видаляє токсичні продукти горіння зі шляхів евакуації. З огляду на наявність коштовної обчислювальної техніки під напругою, для захисту серверного вузла обрано локальну модульну систему автоматичного газового пожежогасіння. Вона використовує діоксид вуглецю або інертні гази, що дозволяє швидко ліквідувати вогнище без пошкодження електронних плат та мікросхем.

Висновки до розділу 5

Підсумовуючи п'ятий розділ, важливо наголосити, що питання охорони праці розглядалися не як формальність, а як невід'ємна умова успішної розробки проєкту «Cashlytics». Усі запропоновані заходи спираються на реальні потреби ІТ-фахівців та повністю адаптовані до вимог сучасного українського законодавства, санітарних і будівельних норм.

Оскільки специфіка роботи інженера-програміста виключає вплив важких промислових небезпек, головний акцент у розділі зроблено на прихованих

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		86

загрозах: ризиках від роботи з електроустановками, акустичному дискомфорту від серверів та надмірному зоровому навантаженні. Для протидії цим факторам підібрано максимально конкретні інженерні рішення. Зокрема, безпеку електромережі гарантує система заземлення TN-S у комбінації з чутливими пристроями захисного відключення (ПЗВ), що миттєво реагують на витoki струму. Ергономіку робочого процесу забезпечують безблисківі LED-панелі та інтеграція темної теми (Dark Mode) безпосередньо у функціонал самого застосунку, що значно знижує втому очей.

Питання пожежної безпеки та цивільного захисту вирішено з урахуванням того, що офіс належить до категорії В і переповнений дороговартісною обчислювальною технікою. Саме тому класичні засоби гасіння замінено на вуглекислотні вогнегасники, які ефективно долають вогонь, не пошкоджуючи мікросхеми. Водночас спроектовані шляхи евакуації, автоматичне знеструмлення та аварійне освітлення гарантують швидкий і безпечний вихід персоналу у разі будь-яких надзвичайних ситуацій.

У підсумку, розроблений комплекс заходів доводить, що створення безпечного та комфортного робочого середовища є цілком реалізованим завданням. Продумана ергономіка та надійний захист від виробничих ризиків дозволяють розробнику уникнути професійних захворювань і повністю зосередитися на створенні якісного та стабільного ІТ-продукту.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		87

ВИСНОВКИ

Розробка інтелектуальної інформаційної системи управління особистими фінансами на базі технологій Open Banking «Cashlytics» дозволила створити ефективну, безпечну та зручну платформу для автоматизованого обліку доходів і витрат, інтелектуальної категоризації транзакцій та предиктивного фінансового аналізу. Ця вебсистема розроблена безпосередньо для користувачів (насамперед молоді, студентів, а також фрілансерів та ІТ-спеціалістів), які прагнуть оптимізувати свій особистий чи сімейний бюджет, уникнути рутинного ручного введення даних та підвищити рівень своєї фінансової грамотності, що є надзвичайно актуальним у сучасному цифровому суспільстві. У процесі роботи були досягнуті наступні результати:

При аналізі предметної області було детально досліджено ринок систем управління особистими фінансами (Personal Finance Management) та обґрунтовано актуальність інтеграції концепції відкритого банкінгу. Було вивчено можливості, сильні та слабкі сторони існуючих конкурентів, а також визначено функції, яких не вистачає в аналогічних застосунках, щоб створити збалансоване фінтех-рішення з унікальною конкурентною перевагою – поєднанням автоматичної синхронізації рахунків та генеративного штучного інтелекту.

Для проектування системи було обрано сучасні технології, які посідають провідні позиції на ІТ-ринку та забезпечують високу швидкість розробки і масштабованість. Клієнтську частину реалізовано за допомогою фреймворку React та мови TypeScript із використанням адаптивних інструментів візуалізації даних. Серверну частину побудовано на базі платформи Node.js, Express.js та Prisma ORM у зв'язці з реляційною базою даних PostgreSQL (Supabase). Для забезпечення ізольованості процесів та зручності розгортання застосовано технологію контейнеризації Docker. Особливе місце в архітектурі посідає успішна інтеграція з API Monobank, Wise та великою мовною моделлю Google Gemini, яка виконує роль розумного діалогового асистента. Важливим підсумком розробки стало виведення системи у середовище реальної експлуатації (Production) на власному домені

cashlytics.site із налаштуванням зворотного проксі-сервера Nginx, презентаційної сторінки (Landing Page) та захищеного HTTPS-з'єднання.

Також під час створення програмного продукту було проведено комплексне тестування модулів системи, яке показало, що програма працює коректно та стабільно виконує всі функції, які були зазначені у функціональних вимогах і розписані у клієнт-серверній архітектурі. Система продемонструвала надійність криптографічного шифрування банківських токенів за стандартом AES-256 та швидку обробку запитів користувача. Додатково було успішно протестовано модуль мережевого захисту від DDoS-атак (Rate Limiter) та механізм дотримання стандартів конфіденційності GDPR (збір згоди на використання файлів cookie).

При розробці системи було виділено аспекти, в яких можна покращити програму та вдосконалити її під час подальшої експлуатації. За допомогою функціонально-вартісного аналізу (ФВА) оптимізовано поточні витрати на інфраструктуру, проаналізовано ризики чутливості фінансової моделі (SaaS підписки та афілійований маркетинг), а також розроблено інженерно-технічні заходи з охорони праці та цивільного захисту, включаючи проектування систем безпечного освітлення та автоматичного захисного відключення електромережі.

В результаті виконаної роботи створено ефективну, захищену та масштабовану систему, яка повністю відповідає всім сучасним вимогам та галузевим стандартам розробки програмного забезпечення. Програмний продукт може бути використаний як надійна архітектурна основа для створення інших складних фінансових агрегаторів або інтелектуальних систем аналізу даних із забезпеченням користувачам високого рівня безпеки та інтерактивного інтерфейсу.

					227202.472 ПЗ	Арк.
						89
Змн.	Арк.	№ докум.	Підпис	Дата		

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

- 1) React Documentation // React. URL: <https://react.dev/> (дата звернення: 25.04.2026).
- 2) API для розробників Monobank [Електронний ресурс]. – Режим доступу: <https://api.monobank.ua/> – Дата звернення: 25.04.2026.
- 3) Amazon Web Services (AWS) Documentation [Електронний ресурс]. – Режим доступу: <https://docs.aws.amazon.com/> – Дата звернення: 25.04.2026.
- 4) Crypto API. Node.js Documentation [Електронний ресурс]. – Режим доступу: <https://nodejs.org/api/crypto.html> – Дата звернення: 25.04.2026.
- 5) Docker Documentation [Електронний ресурс]. – Режим доступу: <https://docs.docker.com/> – Дата звернення: 25.04.2026.
- 6) Express: Fast, unopinionated, minimalist web framework for Node.js [Електронний ресурс]. – Режим доступу: <https://expressjs.com/> – Дата звернення: 25.04.2026.
- 7) Gemini API Documentation. Google AI for Developers [Електронний ресурс]. – Режим доступу: <https://ai.google.dev/docs> – Дата звернення: 25.04.2026.
- 8) GitHub [Електронний ресурс]. – Режим доступу: <https://github.com/> – Дата звернення: 25.04.2026.
- 9) Material-UI (MUI): A popular React UI framework [Електронний ресурс]. – Режим доступу: <https://mui.com/> – Дата звернення: 25.04.2026.
- 10) Namecheap: Domain Name Registrar & Web Hosting [Електронний ресурс]. – Режим доступу: <https://www.namecheap.com/> – Дата звернення: 25.04.2026.
- 11) Nginx Documentation [Електронний ресурс]. – Режим доступу: <https://nginx.org/en/docs/> – Дата звернення: 25.04.2026.
- 12) Node.js Documentation [Електронний ресурс]. – Режим доступу: <https://nodejs.org/en/docs/> – Дата звернення: 25.04.2026.
- 13) PlantUML: Open-source tool for UML diagrams [Електронний ресурс]. – Режим доступу: <https://plantuml.com/> – Дата звернення: 25.04.2026.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		90

14) Prisma: Next-generation ORM for Node.js & TypeScript [Электронный ресурс]. – Режим доступа: <https://www.prisma.io/docs> – Дата звернення: 25.04.2026.

15) React Documentation [Электронный ресурс]. – Режим доступа: <https://react.dev/> – Дата звернення: 25.04.2026.

16) Recharts: A composable charting library built on React components [Электронный ресурс]. – Режим доступа: <https://recharts.org/> – Дата звернення: 25.04.2026.

17) Spendee: See where your money goes [Электронный ресурс]. – Режим доступа: <https://www.spendee.com/> – Дата звернення: 25.04.2026.

18) Stack Overflow: Where Developers Learn, Share, & Build Careers [Электронный ресурс]. – Режим доступа: <https://stackoverflow.com/> – Дата звернення: 25.04.2026.

19) Supabase Documentation [Электронный ресурс]. – Режим доступа: <https://supabase.com/docs> – Дата звернення: 25.04.2026.

20) Toshl Finance: Personal finance manager [Электронный ресурс]. – Режим доступа: <https://toshl.com/> – Дата звернення: 25.04.2026.

21) TypeScript Documentation [Электронный ресурс]. – Режим доступа: <https://www.typescriptlang.org/docs/> – Дата звернення: 25.04.2026.

22) Wallet by BudgetBakers [Электронный ресурс]. – Режим доступа: <https://budgetbakers.com/> – Дата звернення: 25.04.2026.

23) Wise API Documentation [Электронный ресурс]. – Режим доступа: <https://docs.wise.com/> – Дата звернення: 25.04.2026.

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		91

ДОДАТКИ

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		92

App.tsx

```

import { useState, useEffect } from 'react';
import { Container, CssBaseline, CircularProgress, Box } from '@mui/material';
import axios from 'axios';
import { AppThemeProvider, useThemeMode } from './components/Themecontext.tsx';
import Header from './components/Header';
import Footer from './components/Footer';
import Dashboard from './components/Dashboard';
import TransactionList from './components/TransactionList';
import AuthPage from './components/AuthPage';
import SettingsPage from './components/SettingsPage';
import ProfilePage from './components/ProfilePage';
import BudgetsPage from './components/BudgetsPage';
import SubscriptionsPage from './components/SubscriptionsPage';

```

```

axios.defaults.withCredentials = true;
axios.defaults.baseURL = 'http://localhost:3000';

```

```

const GridOverlay = () => (
  <Box sx={{
    position: 'fixed',
    inset: 0,
    zIndex: 0,
    pointerEvents: 'none',
    backgroundImage: `
      linear-gradient(rgba(148,163,184,0.025) 1px, transparent 1px),
      linear-gradient(90deg, rgba(148,163,184,0.025) 1px, transparent 1px)
    `,
    backgroundSize: '60px 60px',
  }} />
);

```

```

function LoadingScreen() {
  return (
    <Box sx={{
      display: 'flex', justifyContent: 'center', alignItems: 'center',
      minHeight: '100vh', flexDirection: 'column', gap: 3,

```

					227202.472 ПЗ	Арк.
						93
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    }}>
    <Box sx={{
      width: 60, height: 60, borderRadius: '16px',
      background: 'linear-gradient(135deg, #22D3EE, #818CF8)',
      display: 'flex', alignItems: 'center', justifyContent: 'center',
      fontSize: '28px',
      boxShadow: '0 0 40px rgba(34,211,238,0.3)',
      animation: 'breathe 2s ease-in-out infinite',
      '@keyframes breathe': {
        '0%, 100%': { transform: 'scale(1)', boxShadow: '0 0 40px rgba(34,211,238,0.3)' },
        '50%': { transform: 'scale(1.06)', boxShadow: '0 0 60px rgba(34,211,238,0.45)' },
      },
    }}>
    
    </Box>
    <CircularProgress size={20} thickness={4} sx={{ color: '#22D3EE', opacity: 0.6 }} />
  </Box>
);
}

```

```

function AppInner() {
  const { isDarkMode } = useThemeMode();

  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const [isLoading, setIsLoading] = useState(true);
  const [currentPage, setCurrentPage] = useState<'dashboard' | 'settings' | 'profile' | 'budgets' |
'subscriptions'>('dashboard');
  const [currentCurrency, setCurrentCurrency] = useState('UAH');

  useEffect(() => {
    axios.get('/api/auth/me')
      .then(() => setIsAuthenticated(true))
      .catch(() => setIsAuthenticated(false))
      .finally(() => setIsLoading(false));
  }, []);
}

```

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		94

```
const glowA = isDarkMode ? 'rgba(34,211,238,0.05)' : 'rgba(8,145,178,0.06)';
const glowB = isDarkMode ? 'rgba(129,140,248,0.05)' : 'rgba(99,102,241,0.05)';
const bgBase = isDarkMode ? '#070B14' : '#F0F6FA';
```

```
return (
  <Box sx={{
    minHeight: '100vh',
    position: 'relative',
    background: bgBase,
    transition: 'background 0.3s ease',
    '&::before': {
      content: '""',
      position: 'fixed',
      top: '-20%', left: '-10%',
      width: '50%', height: '60%',
      background: `radial-gradient(ellipse, ${glowA} 0%, transparent 70%)`,
      pointerEvents: 'none',
      zIndex: 0,
      transition: 'background 0.3s ease',
    },
    '&::after': {
      content: '""',
      position: 'fixed',
      bottom: '-20%', right: '-10%',
      width: '50%', height: '60%',
      background: `radial-gradient(ellipse, ${glowB} 0%, transparent 70%)`,
      pointerEvents: 'none',
      zIndex: 0,
      transition: 'background 0.3s ease',
    },
  }}>
  <GridOverlay />

  <Box sx={{ position: 'relative', zIndex: 1 }}>
    {isLoading ? (
      <LoadingScreen />
    ) : !isAuthenticated ? (
      <Box sx={{
```

					227202.472 ПЗ	Арк.
						95
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        minHeight: '100vh',
        display: 'flex',
        alignItems: 'center',
        justifyContent: 'center',
        px: 2,
      }}>
      <Container maxWidth="sm">
        <AuthPage onLoginSuccess={() => setIsAuthenticated(true)} />
      </Container>
    </Box>
  ): (
    <
      <Header
        onLogout={() => setIsAuthenticated(false)}
        onNavigate={(page) => setCurrentPage(page)}
        currentPage={currentPage}
        currentCurrency={currentCurrency}
        onCurrencyChange={setCurrentCurrency}
      />
      <Container maxWidth="lg" sx={{ pt: 5, pb: 2 }}>
        {currentPage === 'dashboard' && (
          <
            <Dashboard currentCurrency={currentCurrency} />
            <TransactionList currentCurrency={currentCurrency} />
          </>
        )}
        {currentPage === 'budgets' && <BudgetsPage currentCurrency={currentCurrency} />}
        {currentPage === 'subscriptions' && <SubscriptionsPage currentCurrency={currentCurrency} />}
      </Container>
      <Footer />
    </Box>
  )}
</Box>
</Box>
);

```

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		96


```
}  
  
export default function App() {  
  return (  
    <AppThemeProvider>  
      <CssBaseline />  
      <AppInner />  
    </AppThemeProvider>  
  );  
}
```

					227202.472 ПЗ	Арк.
						97
Змн.	Арк.	№ докум.	Підпис	Дата		

server.ts

```
import express, { Request, Response, NextFunction } from 'express';
import cors from 'cors';
import dotenv from 'dotenv';
import axios from 'axios';
import { GoogleGenAI } from '@google/genai';
import { Pool } from 'pg';
import { PrismaPg } from '@prisma/adapters-pg';
import { PrismaClient } from '@prisma/client';
import { encrypt, decrypt } from './encryption';
import rateLimit from 'express-rate-limit';
import * as argon2 from 'argon2';
import jwt from 'jsonwebtoken';
import cookieParser from 'cookie-parser';

dotenv.config();

console.log("🔑 Статус ключа Gemini:", process.env.GEMINI_API_KEY ? "Знайдено!" : "ПОРОЖНЬО");
console.log("🗄 Статус підключення до БД:", process.env.DATABASE_URL ? "Знайдено!" :
"ПОРОЖНЬО");

const app = express();
const PORT = 3000;
const JWT_SECRET = process.env.JWT_SECRET || 'fallback_secret_key';

app.use(cors({
  origin: 'http://localhost:5173',
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));
app.use(express.json());
app.use(cookieParser());

const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY as string });
```

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		98

```

const connectionString = process.env.DATABASE_URL;
const pool = new Pool({ connectionString });
const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL });
export const prisma = new PrismaClient({ adapter });
const aiForecastCache: Record<string, { hash: string, data: any }> = {};
const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 200,
  message: { error: 'Забагато запитів. Будь ласка, зачекайте трохи.' }
});
app.use('/api/', apiLimiter);

```

```

let cachedRates: Record<string, number> | null = null;
let lastFetchTime = 0;

```

```

async function getExchangeRates() {
  const now = Date.now();
  if (cachedRates && (now - lastFetchTime < 3600000)) {
    return cachedRates;
  }
  try {
    const response = await axios.get('https://open.er-api.com/v6/latest/USD');
    cachedRates = response.data.rates;
    lastFetchTime = now;
    return cachedRates;
  } catch (error) {
    console.error("❌ Помилка завантаження курсів валют", error);
    return cachedRates || { USD: 1, UAH: 39.5, EUR: 0.92 };
  }
}

```

```

function convertCurrency(amount: number, fromCurrency: string, toCurrency: string, rates: Record<string, number>) {
  if (fromCurrency === toCurrency) return amount;
  const rateFrom = rates[fromCurrency] || 1;
  const rateTo = rates[toCurrency] || 1;

```

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		99

```

    return (amount / rateFrom) * rateTo;
}

function getStartDate(range: string): Date {
    const d = new Date();
    if (range === 'day') return new Date(d.setHours(0, 0, 0, 0));
    if (range === 'week') return new Date(d.setDate(d.getDate() - (d.getDay() || 7) + 1));
    if (range === 'month') return new Date(d.getFullYear(), d.getMonth(), 1);
    if (range === '3months') { d.setMonth(d.getMonth() - 3); return d; }
    if (range === '6months') { d.setMonth(d.getMonth() - 6); return d; }
    if (range === '1year') { d.setFullYear(d.getFullYear() - 1); return d; }
    if (range === '2years') { d.setFullYear(d.getFullYear() - 2); return d; }
    if (range === '5years') { d.setFullYear(d.getFullYear() - 5); return d; }
    return new Date(0);
}

const authenticateUser = async (req: Request, res: Response, next: NextFunction): Promise<any> => {
    const token = req.cookies.token;
    if (!token) return res.status(401).json({ error: "Доступ заборонено. Немає токена." });

    try {
        const decoded = jwt.verify(token, JWT_SECRET) as { userId: string };
        (req as any).userId = decoded.userId;
        next();
    } catch (err) {
        res.status(401).json({ error: "Доступ заборонено. Токен недійсний." });
    }
};

app.post('/api/auth/register', async (req: Request, res: Response): Promise<any> => {
    try {
        const { email, password } = req.body;

        const existingUser = await prisma.user.findUnique({ where: { email } });
        if (existingUser) {
            return res.status(400).json({ error: "Користувач з таким email вже існує" });
        }
    }
}

```

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		100

```

const passwordHash = await argon2.hash(password, {
  type: argon2.argon2id,
  memoryCost: 2 ** 16,
  timeCost: 3,
  parallelism: 1
});

const newUser = await prisma.user.create({
  data: { email, passwordHash }
});

const token = jwt.sign({ userId: newUser.id }, JWT_SECRET, { expiresIn: '7d' });

res.cookie('token', token, {
  httpOnly: true,
  secure: process.env.NODE_ENV === 'production',
  sameSite: 'lax',
  maxAge: 7 * 24 * 60 * 60 * 1000
});

res.json({ message: "Реєстрація успішна", user: { email: newUser.email } });
} catch (error) {
  console.error(error);
  res.status(500).json({ error: "Помилка сервера при реєстрації" });
}
});

app.post('/api/auth/login', async (req: Request, res: Response): Promise<any> => {
  try {
    const { email, password } = req.body;

    const user = await prisma.user.findUnique({ where: { email } });
    if (!user) {
      return res.status(401).json({ error: "Невірний email або пароль" });
    }

    const isValid = await argon2.verify(user.passwordHash, password);

```

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		101

```

    if (!isPasswordValid) {
        return res.status(401).json({ error: "Невірний email або пароль" });
    }

    const token = jwt.sign({ userId: user.id }, JWT_SECRET, { expiresIn: '7d' });

    res.cookie('token', token, {
        httpOnly: true,
        secure: process.env.NODE_ENV === 'production',
        sameSite: 'lax',
        maxAge: 7 * 24 * 60 * 60 * 1000
    });

    res.json({ message: "Вхід успішний", user: { email: user.email } });
} catch (error) {
    console.error(error);
    res.status(500).json({ error: "Помилка сервера при вході" });
}
});

app.post('/api/auth/logout', (req, res) => {
    res.clearCookie('token', {
        httpOnly: true,
        secure: process.env.NODE_ENV === 'production',
        sameSite: 'lax'
    });
    res.json({ message: "Вийшли успішно" });
});

app.get('/api/auth/me', (req, res) => {
    const token = req.cookies.token;
    if (!token) return res.status(401).json({ error: "Не авторизовано" });

    try {
        const decoded = jwt.verify(token, JWT_SECRET);
        res.json({ userId: (decoded as any).userId });
    } catch (err) {
        res.status(401).json({ error: "Недійсний токен" });
    }
}

```

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		102

```

    }
  });

app.use('/api', authenticateUser);

app.get('/api/transactions', async (req: Request, res: Response) => {
  try {
    const userId = (req as any).userId;
    const targetCurrency = (req.query.currency as string) || 'UAH';

    const range = (req.query.range as string) || 'month';
    const startDate = getStartDate(range);

    const rates = await getExchangeRates();
    const dbTransactions = await prisma.transaction.findMany({
      where: { userId: userId, date: { gte: startDate } },
      orderBy: { date: 'desc' }
    });

    const convertedTransactions = dbTransactions.map(tx => ({
      ...tx,
      originalAmount: tx.amount,
      originalCurrency: tx.currency,
      amount: Number(convertCurrency(tx.amount, tx.currency, targetCurrency, rates!).toFixed(2)),
      currency: targetCurrency
    }));

    res.json(convertedTransactions);
  } catch (error) {
    res.status(500).json({ error: "Помилка читання з БД" });
  }
});

app.get('/api/sync-monobank', async (req: Request, res: Response): Promise<any> => {
  try {
    const userId = String((req as any).userId);
    const user = await prisma.user.findUnique({ where: { id: userId } });

```

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		103

```

if (!user || !user.monoToken) {
  return res.status(400).json({ error: "Додайте токен Monobank у налаштуваннях" });
}

const decryptedToken = decrypt(user.monoToken);

console.log("🏠 Йдемо в Monobank за інформацією про рахунки...");

const clientInfoResponse = await axios.get('https://api.monobank.ua/personal/client-info', {
  headers: { 'X-Token': decryptedToken }
});

const monoAccounts = clientInfoResponse.data.accounts || [];

for (const monoAcc of monoAccounts) {
  const realBalance = monoAcc.balance / 100;

  let cardName = "Monobank Картка";
  if (monoAcc.type === 'black') cardName = "Monobank Чорна 🖤";
  else if (monoAcc.type === 'white') cardName = "Monobank Біла ⬜";
  else if (monoAcc.type === 'platinum') cardName = "Monobank Platinum 💎";
  else if (monoAcc.type === 'fop') cardName = "Monobank ФОП 📁";
  else if (monoAcc.type === 'eAid') cardName = "єПідтримка UA";

  let currencyCode = "UAH";
  if (monoAcc.currencyCode === 840) currencyCode = "USD";
  else if (monoAcc.currencyCode === 978) currencyCode = "EUR";
  else if (monoAcc.currencyCode === 985) currencyCode = "PLN";

  await prisma.account.upsert({
    where: { id: monoAcc.id },
    update: { balance: realBalance, name: cardName },
    create: {
      id: monoAcc.id,
      name: cardName,
      balance: realBalance,
      currency: currencyCode,
    }
  });
}

```

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		104


```

        source: "MONO",
        userId: userId
    }
});
}

console.log('☑ Збережено/оновлено баланси для ${monoAccounts.length} карток Monobank.');
```

const currentUnixTime = Math.floor(Date.now() / 1000);
const maxMonobankPeriod = currentUnixTime - (31 * 24 * 60 * 60);

```

console.log("🏠 Йдемо в Monobank за випискою...");
const bankResponse = await axios.get(
    `https://api.monobank.ua/personal/statement/0/${maxMonobankPeriod}/${currentUnixTime}`,
    { headers: { 'X-Token': decryptedToken } }
);

const transactionsFromBank = bankResponse.data;
console.log('☑ Отримано з банку: ${transactionsFromBank.length} транзакцій.');
```

let addedCount = 0;

```

for (const item of transactionsFromBank.slice(0, 5)) {
    const existingTx = await prisma.transaction.findUnique({ where: { id: item.id } });
    if (existingTx) continue;

    const amountInUah = item.amount / 100;
    const description = item.description || "Невідомо";

    const prompt = `Категоризуй транзакцію: "${description}". Поверни ЛИШЕ ОДНЕ слово/фразу з
емодзі з цього списку: [🛒 Продукти, ☕ Кафе, 🚗 Транспорт, 🏠 Дім, 🛍 Шопінг, 🏥 Здоров'я, 🎁
Розваги, 📖 Освіта, ✈ Подорожі, 📬 Перекази, 💰 Зарплата, 📦 Подарунок, 🏧 Кешбек, 📦 Інше].
Переказ 📬* - також може бути просто якщо написаний номер картки.`;

    const response = await ai.models.generateContent({ model: 'gemini-3.1-flash-lite', contents: prompt });
    const aiCategory = response.text?.trim() || "📦 Інше";

    const parts = aiCategory.split(' ');

```

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		105

```

const icon = parts[0];
const cleanName = parts.slice(1).join(' ').trim() || aiCategory;

const dbCategory = await prisma.category.upsert({
  where: { name_userId: { name: cleanName, userId: userId } },
  update: {}, create: { name: cleanName, icon: icon, userId: userId }
});

await prisma.transaction.create({
  data: {
    id: item.id, date: new Date(item.time * 1000), description: description,
    amount: amountInUah, currency: "UAH", source: "MONOBANK", userId: userId,
    categoryId: dbCategory.id, categoryName: aiCategory
  }
});

addedCount++;
await new Promise(resolve => setTimeout(resolve, 1500));
}

res.json({ message: "Синхронізація успішна!", syncedCount: addedCount });

} catch (error: any) {
  console.error("❌ Помилка синхронізації Monobank:", error.message || error);
  res.status(500).json({ error: "Не вдалося отримати дані" });
}
});
app.post('/api/ai-chat', async (req: Request, res: Response): Promise<any> => {
  try {
    const userId = String((req as any).userId);
    const { message, history } = req.body;

    const dbTransactions = await prisma.transaction.findMany({
      where: { userId }, orderBy: { date: 'desc' }, take: 50
    });

    const budgets = await prisma.budget.findMany({ where: { userId } });

    const txContext = dbTransactions.map((t: any) => `${t.date.toISOString().split('T')[0]} | ${t.description} |

```

					227202.472 ПЗ	Арк.
						106
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    ${t.amount} ${t.currency} `).join('\n');

    const budgetContext = budgets.map((b: any) => `${b.categoryName}: ${b.amount}
    ${b.currency} `).join('\n');

    let prompt = `Ти – фінансовий AI-асистент додатку Cashlytics. Відповідай українською мовою,
    коротко, дружньо, без складного форматування.\n\n`;

    prompt += `Останні транзакції користувача:\n${txContext} || 'Немає'\n\n`;
    prompt += `Бюджети користувача:\n${budgetContext} || 'Немає'\n\n`;

    if (history && history.length > 0) {
        prompt += `Історія чату:\n`;
        history.slice(-6).forEach((h: any) => {
            prompt += `${h.role === 'ai' ? 'AI' : 'Користувач'}: ${h.text}\n`;
        });
        prompt += '\n';
    }

    prompt += `Запитання: ${message}\nAI:`;

    const aiRes = await ai.models.generateContent({ model: 'gemini-3.1-flash-lite', contents: prompt });
    res.json({ reply: aiRes.text?.trim() || "Не можу відповісти зараз 😊" });
} catch (error) {
    console.error("Помилка ШІ-чату:", error);
    res.status(500).json({ error: "Помилка зв'язку з AI" });
}
});

app.listen(PORT, () => {
    console.log(`☑ Сервер запущено на http://localhost:${PORT}`);

```

					227202.472 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		107